



RS6x_7x SDK Reference Manual

Number of Pages:169
Document Version:1.2
Release Date:2025-08-31

Revision History

Version	Revision Description	Date
1.2	Corrected a typo in Section 3.3.3.4 and removed the redundant closing brace from Table 4-9.	2025-10-29
1.1	Updated the HIF API descriptions.	2025-10-15
1.0	Initial release	2025-09-10

Table of Contents

Revision History	ii
Table of Contents	iii
List of Figures	ix
List of Tables	xi
1 Overview	1
2 SDK Introduction	2
2.1 Architecture Overview	2
2.1.1 RS6130 Hardware Architecture	2
2.1.2 RS6240 Hardware Architecture	3
2.1.3 SDK Software Architecture	4
2.2 Directory Structure	4
2.2.1 Build Tools	4
2.2.2 Software Source Code	4
2.3 References	5
3 Create a Project	7
3.1 Project Directory Structure	7
3.2 Project Creation	7
3.2.1 Add Source Files	9
3.2.2 Project Configuration	14
3.2.3 Board Selection	18
3.3 Create a Development Template	20
3.3.1 Create a Board Based on RS6130	20
3.3.2 Create a Board Based on RS6240	22
3.3.3 Configuration Changes	24
3.3.3.1 Clock Configuration	24
3.3.3.2 Pinmux Configuration	24
3.3.3.3 FLASH/XIP Operating Frequency	25
3.3.3.4 Other Configuration	26
4 Subsystems	28
4.1 CPU	28
4.1.1 Cache	30
4.1.2 Floating Point	31
4.1.3 Dual Core	31
4.1.4 Code Optimization Guide	31
4.1.4.1 SRAM Usage Analysis	31
4.1.4.2 Heap Usage Analysis	33
4.2 DSP	33
4.3 Flash and PSRAM	34
4.3.1 Code in XIP or SRAM	35
4.4 Memory Map	36
4.4.1 Img Format	36
4.4.2 Map	37
4.5 Startup and Configuration	39
4.5.1 Initialization	39
4.5.2 Configuration	39
4.6 Clock	40
4.7 Pinmux	41
4.8 LOG	42
4.9 Console Shell	44
4.10 Power Management	45
4.10.1 Introduction	45

4.10.2 Interface	46
4.10.2.1 pm_device_register	46
4.10.2.2 pm_device_unregister	46
4.10.2.3 pm_notifier_register	46
4.10.2.4 pm_notifier_unregister	47
4.10.2.5 pm_policy_latency_update	47
4.10.2.6 pm_policy_wake_lock_is_active	47
4.10.2.7 pm_policy_wake_lock	48
4.10.2.8 pm_policy_wake_unlock	48
4.10.2.9 pm_wake_lock_timeout	48
4.10.2.10 pm_device_runtime_enable	49
4.10.2.11 pm_device_runtime_disable	49
4.10.2.12 pm_device_runtime_get	49
4.10.2.13 pm_device_runtime_put	50
4.11 KVR & KVF	50
4.11.1 Introduction	50
4.11.2 Interface	50
4.11.2.1 KVR_Save	50
4.11.2.2 KVR_Load	51
4.11.2.3 KVR_Delete	51
4.11.2.4 KVF_Save	51
4.11.2.5 KVF_Load	52
4.11.2.6 KVF_Delete	52
4.12 Version	53
4.12.1 Introduction	53
4.12.2 Interface	54
4.12.2.1 VER_GetSdkVersion	54
4.12.2.2 VER_GetSubimgVersion	54
4.12.2.3 VER_GetBootVersion	55
4.12.2.4 VER_GetBuildTime	55
4.13 HIF	56
4.13.1 Function Overview	56
4.13.2 Macro Switches	56
4.13.3 API Description	56
4.13.3.1 HIF_Init	56
4.13.3.2 HIF_DeInit	57
4.13.3.3 HIF_DefaultInitParam	58
4.13.3.4 HIF_ExtControl	58
4.13.3.5 HIF_ParamGet	59
4.13.3.6 HIF_MsgHdl_Regist	60
4.13.3.7 HIF_MsgResp	60
4.13.3.8 HIF_MsgReport	61
4.13.3.9 HIF_MsgReport_ListAddData	62
4.13.3.10 :HIF_MsgReport_ListStart	62
4.14 Other	63
4.14.1 Interface	63
4.14.1.1 HAL_BOARD_GetFreq	63
4.14.1.2 HAL_BOARD_GetTime	64
4.14.1.3 HAL_BOARD_BlkDelay	64
4.14.1.4 HAL_BOARD_GetRand	65
4.14.1.5 HAL_BOARD_Reset	65
5 Peripheral Drivers	66
5.1 GPIO	66
5.1.1 Function Overview	66

5.1.1.1 Pull-up/Pull-down Resistors	66
5.1.1.2 Output Drive Capability	67
5.1.1.3 Functional Modes	67
5.1.1.4 GPIO and Power-Management WKIO	67
5.1.1.5 Debounce	68
5.1.2 Interface	68
5.1.2.1 HAL_GPIO_Init	68
5.1.2.2 HAL_GPIO_DeInit	69
5.1.2.3 HAL_GPIO_SetPinParam	69
5.1.2.4 HAL_GPIO_GetPinParam	70
5.1.2.5 HAL_GPIO_DisablePin	70
5.1.2.6 HAL_GPIO_SetPinMode	71
5.1.2.7 HAL_GPIO_SetPinPull	71
5.1.2.8 HAL_GPIO_SetPinPush	72
5.1.2.9 HAL_GPIO_SetPinDriving	73
5.1.2.10 HAL_GPIO_ReadPort	73
5.1.2.11 HAL_GPIO_WritePort	74
5.1.2.12 HAL_GPIO_ReadPin	74
5.1.2.13 HAL_GPIO_WritePin	75
5.1.2.14 HAL_GPIO_TogglePin	75
5.1.2.15 HAL_GPIO_SetIRQParam	75
5.1.2.16 HAL_GPIO_SetIRQDebounce	76
5.1.2.17 HAL_GPIO_EnableIRQ	77
5.1.2.18 HAL_GPIO_DisableIRQ	77
5.1.2.19 HAL_WKIO_Init	77
5.1.2.20 HAL_WKIO_DeInit	78
5.1.2.21 HAL_WKIO_SetPinMode	78
5.1.2.22 HAL_WKIO_GetPinMode	79
5.1.2.23 HAL_WKIO_SetPinPull	80
5.1.2.24 HAL_WKIO_GetPinPull	80
5.1.2.25 HAL_WKIO_WritePin	81
5.1.2.26 HAL_WKIO_WritePort	81
5.1.2.27 HAL_WKIO_SetWakeParam	82
5.1.2.28 HAL_WKIO_EnableWake	82
5.1.2.29 HAL_WKIO_DisableWake	83
5.1.2.30 HAL_WKIO_SetWakeEvent	83
5.1.2.31 HAL_WKIO_GetWakeEvent	84
5.1.2.32 HAL_WKIO_ReadWakePin	84
5.1.2.33 HAL_WKIO_SetWakeDebounce	85
5.2 UART	85
5.2.1 Function Overview	85
5.2.1.1 Features	85
5.2.1.2 Usage	85
5.2.2 Interface	86
5.2.2.1 HAL_UART_Init	86
5.2.2.2 HAL_UART_DeInit	87
5.2.2.3 HAL_UART_ExtControl	87
5.2.2.4 HAL_UART_Open	88
5.2.2.5 HAL_UART_Close	88
5.2.2.6 HAL_UART_SetTransferMode	89
5.2.2.7 HAL_UART_Receive	89
5.2.2.8 HAL_UART_Transmit	90
5.2.2.9 HAL_UART_GetRxData	90
5.2.2.10 HAL_UART_PutTxData	91

5.2.2.11 HAL_UART_IsTxReady	91
5.2.2.12 HAL_UART_IsTxEmpty	91
5.2.2.13 HAL_UART_IsRxReady	91
5.2.2.14 HAL_UART_RegisterIRQ	92
5.2.2.15 HAL_UART_UnregisterIRQ	92
5.2.2.16 HAL_UART_EnableTxReadyIRQ	93
5.2.2.17 HAL_UART_EnableRxReadyIRQ	93
5.2.2.18 HAL_UART_DisableTxReadyIRQ	93
5.2.2.19 HAL_UART_DisableRxReadyIRQ	94
5.2.2.20 HAL_UART_RegisterDMA	94
5.2.2.21 HAL_UART_UnregisterDMA	95
5.2.2.22 HAL_UART_StartTransmitDMA	95
5.2.2.23 HAL_UART_StopTransmitDMA	96
5.2.2.24 HAL_UART_StartReceiveDMA	96
5.2.2.25 HAL_UART_StopReceiveDMA	96
5.3 SPI	97
5.3.1 Function Overview	97
5.3.1.1 Features	97
5.3.1.2 Usage	98
5.3.2 Mode Description	101
5.3.2.1 SPI_MODE1	101
5.3.2.2 SPI_MODE2	102
5.3.3 Interface	103
5.3.3.1 HAL_SPI_Init	103
5.3.3.2 HAL_SPI_DeInit	103
5.3.3.3 HAL_SPI_Open	104
5.3.3.4 HAL_SPI_Close	104
5.3.3.5 HAL_SPI_ExtControl	104
5.3.3.6 HAL_SPI_SetTransferMode	105
5.3.3.7 HAL_SPI_Receive	106
5.3.3.8 HAL_SPI_Transmit	106
5.3.3.9 HAL_SPI_TransmitReceive	107
5.3.3.10 HAL_SPI_GetRxData	108
5.3.3.11 HAL_SPI_PutTxData	108
5.3.3.12 HAL_SPI_StartTrans	108
5.3.3.13 HAL_SPI_StopTrans	109
5.3.3.14 HAL_SPI_IsTxReady	109
5.3.3.15 HAL_SPI_IsTxEmpty	109
5.3.3.16 HAL_SPI_IsRxReady	109
5.3.3.17 HAL_SPI_IsBusy	110
5.3.3.18 HAL_SPI_GetFifoLevel	110
5.3.3.19 HAL_SPI_IsCsInterrupt	110
5.3.3.20 HAL_SPI_EnableDMA	111
5.3.3.21 HAL_SPI_DisableDMA	111
5.3.3.22 HAL_SPI_EnableCS	111
5.3.3.23 HAL_SPI_DisableCS	112
5.3.3.24 HAL_SPI_RegisterIRQ	112
5.3.3.25 HAL_SPI_UnregisterIRQ	113
5.3.3.26 HAL_SPI_EnableIRQ	113
5.3.3.27 HAL_SPI_DisableIRQ	114
5.3.3.28 HAL_SPI_RegisterDMA	114
5.3.3.29 HAL_SPI_UnregisterDMA	115
5.3.3.30 HAL_SPI_StartDMA	115
5.3.3.31 HAL_SPI_StopDMA	115

5.3.3.32 HAL_SPI_PM_Suspend	116
5.3.3.33 HAL_SPI_PM_Resume	116
5.4 I2C	118
5.4.1 Function Overview	118
5.4.1.1 Features	118
5.4.1.2 Usage	118
5.4.2 Interface	119
5.4.2.1 HAL_I2C_Init	119
5.4.2.2 HAL_I2C_DeInit	120
5.4.2.3 HAL_I2C_Open	120
5.4.2.4 HAL_I2C_Open	121
5.4.2.5 HAL_I2C_RecoverBus	121
5.4.2.6 HAL_I2C_SetTransferMode	121
5.4.2.7 HAL_I2C_Transmit	122
5.4.2.8 HAL_I2C_Receive	122
5.4.2.9 HAL_I2C_GetRxData	124
5.4.2.10 HAL_I2C_PutTxData	124
5.4.2.11 HAL_I2C_IsTxReady	125
5.4.2.12 HAL_I2C_IsRxReady	125
5.4.2.13 HAL_I2C_IsTxEmpty	125
5.4.2.14 HAL_I2C_RegisterIRQ	125
5.4.2.15 HAL_I2C_UnregisterIRQ	126
5.4.2.16 HAL_I2C_EnableIRQ	126
5.4.2.17 HAL_I2C_DisableIRQ	127
5.4.2.18 HAL_I2C_ClearIRQ	127
5.4.2.19 HAL_I2C_GetIRQStatus	128
5.5 DMA	129
5.5.1 Function Overview	129
5.5.1.1 Block Transfer	129
5.5.1.2 Linked-list Transfer	132
5.5.2 Interface	133
5.5.2.1 HAL_DMA_Init	133
5.5.2.2 HAL_DMA_DeInit	134
5.5.2.3 HAL_DMA_Open	134
5.5.2.4 HAL_DMA_Close	135
5.5.2.5 HAL_DMA_ConfigBlockTransfer	135
5.5.2.6 HAL_DMA_InitNodeCtrl	136
5.5.2.7 HAL_DMA_AddNextNode	136
5.5.2.8 HAL_DMA_ConfigListTransfer	137
5.5.2.9 HAL_DMA_ConfigScatterGatherTransfer	137
5.5.2.10 HAL_DMA_StartTransfer	138
5.5.2.11 HAL_DMA_AbortTransfer	139
5.5.2.12 HAL_DMA_RegisterIRQ	139
5.5.2.13 HAL_DMA_UnRegisterIRQ	140
5.5.2.14 HAL_DMA_EnableIRQ	140
5.5.2.15 HAL_DMA_DisableIRQ	141
5.5.2.16 HAL_DMA_GetChanStatus	141
5.6 FLASH	143
5.6.1 Function Overview	143
5.6.2 Interface	146
5.6.2.1 HAL_FLASH_Init	146
5.6.2.2 HAL_FLASH_DeInit	146
5.6.2.3 HAL_FLASH_Open	147
5.6.2.4 HAL_FLASH_Close	147

5.6.2.5 HAL_FLASH_Read	148
5.6.2.6 HAL_FLASH_Write	148
5.6.2.7 HAL_FLASH_Erase	149
5.6.2.8 HAL_FLASH_ExtControl	149
5.6.2.9 HAL_FLASH_Suspend	151
5.6.2.10 HAL_FLASH_Resume	151
5.7 WDG	151
5.7.1 Function Overview	151
5.7.2 Interface	152
5.7.2.1 HAL_WDG_Init	152
5.7.2.2 HAL_WDG_DeInit	152
5.7.2.3 HAL_WDG_Open	153
5.7.2.4 HAL_WDG_Close	153
5.7.2.5 HAL_WDG_Feed	154

List of Figures

Figure 2-1	RS6130 System Block Diagram	2
Figure 2-2	RS6130 Application Subsystem	2
Figure 2-3	RS6240 System Block Diagram	3
Figure 2-4	RS6240 Application Subsystem	3
Figure 2-5	RS Series SDK Framework	4
Figure 3-1	new_project User Interface	8
Figure 3-2	Rename Project Menu	8
Figure 3-3	Enter the Project Name	9
Figure 3-4	Add a Source Folder	9
Figure 3-5	Select the kernel Folder	10
Figure 3-6	Select the Option Not to Copy the Folder	10
Figure 3-7	Exclude a Folder from the Build	11
Figure 3-8	Icon for a Folder Excluded from the Build	11
Figure 3-9	Create a Virtual Folder	12
Figure 3-10	Enter the Virtual Folder Name	12
Figure 3-11	Add Source Files to a Virtual Folder	13
Figure 3-12	Select Source Files for the Current Project	13
Figure 3-13	Select Source Files for the Current Project	14
Figure 3-14	Target Configuration Screen	14
Figure 3-15	Output Configuration Screen	15
Figure 3-16	User Configuration Screen	15
Figure 3-17	Compiler Configuration Screen	16
Figure 3-18	Assembler Configuration Screen	16
Figure 3-19	Linker Configuration Screen	17
Figure 3-20	Select the Linker File	17
Figure 3-21	Device Configuration Screen	18
Figure 3-22	Select the Board in Options for myproject	19
Figure 3-23	Add Application Code to the main Function	19
Figure 3-24	Build the Project	20
Figure 3-25	Add the New Board to CDK	22
Figure 3-26	New Board Added to CDK	22
Figure 3-27	myboard Pinmux Configuration	24
Figure 3-28	myboard Pinmux Configuration Function	25
Figure 3-29	myboard SPI Flash Configuration	26
Figure 4-1	Select the CPU Instruction Set in CDK	29
Figure 4-2	Configure the Linker Script in CDK	29
Figure 4-3	Configure the Radar Symbol Table in CDK	30
Figure 4-4	HEAP Usage	33
Figure 4-5	Configure the DSP Library in CDK	34
Figure 4-6	Startup Process	36
Figure 4-7	Img Organization	37
Figure 4-8	Img Format	37
Figure 4-9	Img Memory Map	38
Figure 4-10	Startup Flow	39
Figure 4-11	Configuration Relationships	40
Figure 4-12	Power Management Flow	45
Figure 5-1	GPIO Framework Diagram	66
Figure 5-2	UART Integrated-API Workflow	86
Figure 5-3	SPI Mode Diagram	98
Figure 5-4	SPI Integrated-API Workflow	99
Figure 5-5	SPI FIFO Usage	100

Figure 5-6	SPI Mode Selection	101
Figure 5-7	SPI Rising-Edge Sampling Error Caused by Delay	101
Figure 5-8	SPI Data Sampling Using CLK_0	102
Figure 5-9	FIFO Pop Operations During SPI Transfers	102
Figure 5-10	I2C Integrated-API Workflow	118
Figure 5-11	DMA Block-Transfer Workflow	130
Figure 5-12	DMA Memory-to-Memory Transfer with 32-bit Width	131
Figure 5-13	DMA Memory-to-Memory Transfer with 8-bit Width	131
Figure 5-14	DMA UART0-RX-to-Memory Transfer	132
Figure 5-15	DMA UART0-RX-to-Memory Transfer	132
Figure 5-16	DMA Linked-List Transfer Workflow	133
Figure 5-17	Flash Chip ID Example	143
Figure 5-18	Dual SPI Transfer Example	144
Figure 5-19	Quad SPI Transfer Example	144
Figure 5-20	SPI Format Support in the Specification	145
Figure 5-21	QER Example in the Specification	145
Figure 5-22	Maximum SPI Frequency in the Specification	146
Figure 5-23	Sleep Latency in the Specification	146
Figure 5-24	Watchdog Usage	152

List of Tables

Table 4-1	RS6130 CPU Specifications	28
Table 4-2	RS6240 CPU Specifications	28
Table 4-3	Heap Shell Return Values	33
Table 4-4	Flash XIP Address Map	34
Table 4-5	PSRAM XIP Address Map	34
Table 4-6	Address Remap	38
Table 4-7	RS6130 Clock Configuration	40
Table 4-8	RS6240 Clock Configuration	41
Table 4-9	pm_device_register	46
Table 4-10	pm_device_unregister	46
Table 4-11	pm_notifier_register	46
Table 4-12	pm_notifier_unregister	47
Table 4-13	pm_policy_latency_update	47
Table 4-14	pm_policy_wake_lock_is_active	47
Table 4-15	pm_policy_wake_lock	48
Table 4-16	pm_policy_wake_unlock	48
Table 4-17	pm_wake_lock_timeout	48
Table 4-18	pm_device_runtime_enable	49
Table 4-19	pm_device_runtime_disable	49
Table 4-20	pm_device_runtime_get	49
Table 4-21	pm_device_runtime_put	50
Table 4-22	KVR_Save	50
Table 4-23	KVR_Load	51
Table 4-24	KVR_Delete	51
Table 4-25	KVF_Save	51
Table 4-26	KVF_Load	52
Table 4-27	KVF_Delete	52
Table 4-28	VER_GetSubimgVersion	54
Table 4-29	VER_GetSdkVersion	54
Table 4-30	VER_GetBootVersion	55
Table 4-31	VER_GetBuildTime	55
Table 4-32	CONFIG_HIF	56
Table 4-33	HIF_Init	56
Table 4-34	HIF_Delnit	57
Table 4-35	HIF_DefaultInitParam	58
Table 4-36	HIF_ExtControl	58
Table 4-37	HIF_ParamGet	59
Table 4-38	HIF_MsgHdl_Regist	60
Table 4-39	HIF_MsgResp	60
Table 4-40	HIF_MsgReport	61
Table 4-41	HIF_MsgReport_ListAddData	62
Table 4-42	HIF_MsgReport_ListStart	62
Table 4-43	HAL_BOARD_GetFreq	63
Table 4-44	HAL_BOARD_GetTime	64
Table 4-45	HAL_BOARD_BlkDelay	64
Table 4-46	HAL_BOARD_GetRand	65
Table 4-47	HAL_BOARD_Reset	65
Table 5-1	HAL_GPIO_Init	68
Table 5-2	HAL_GPIO_Delnit	69
Table 5-3	HAL_GPIO_SetPinParam	69
Table 5-4	HAL_GPIO_GetPinParam	70

Table 5-5	HAL_GPIO_DisablePin	70
Table 5-6	HAL_GPIO_SetPinMode	71
Table 5-7	HAL_GPIO_SetPinPull	71
Table 5-8	HAL_GPIO_SetPinPush	72
Table 5-9	HAL_GPIO_SetPinDriving	73
Table 5-10	HAL_GPIO_ReadPort	73
Table 5-11	HAL_GPIO_WritePort	74
Table 5-12	HAL_GPIO_ReadPin	74
Table 5-13	HAL_GPIO_WritePin	75
Table 5-14	HAL_GPIO_TogglePin	75
Table 5-15	HAL_GPIO_SetIRQParam	75
Table 5-16	HAL_GPIO_SetIRQDebounce	76
Table 5-17	HAL_GPIO_EnableIRQ	77
Table 5-18	HAL_GPIO_DisableIRQ	77
Table 5-19	HAL_WKIO_Init	77
Table 5-20	HAL_WKIO_DeInit	78
Table 5-21	HAL_WKIO_SetPinMode	78
Table 5-22	HAL_WKIO_GetPinMode	79
Table 5-23	HAL_WKIO_SetPinPull	80
Table 5-24	HAL_WKIO_GetPinPull	80
Table 5-25	HAL_WKIO_WritePin	81
Table 5-26	HAL_WKIO_WritePort	81
Table 5-27	HAL_WKIO_SetWakeParam	82
Table 5-28	HAL_WKIO_EnableWake	82
Table 5-29	HAL_WKIO_DisableWake	83
Table 5-30	HAL_WKIO_SetWakeEvent	83
Table 5-31	HAL_WKIO_GetWakeEvent	84
Table 5-32	HAL_WKIO_ReadWakePin	84
Table 5-33	HAL_WKIO_SetWakeDebounce	85
Table 5-34	HAL_UART_Init	86
Table 5-35	HAL_UART_DeInit	87
Table 5-36	HAL_UART_ExtControl	87
Table 5-37	HAL_UART_Open	88
Table 5-38	HAL_UART_Close	88
Table 5-39	HAL_UART_SetTransferMode	89
Table 5-40	HAL_UART_Receive	89
Table 5-41	HAL_UART_Transmit	90
Table 5-42	HAL_UART_GetRxData	90
Table 5-43	HAL_UART_PutTxData	91
Table 5-44	HAL_UART_IsTxReady	91
Table 5-45	HAL_UART_IsTxEmpty	91
Table 5-46	HAL_UART_IsRxReady	91
Table 5-47	HAL_UART_RegisterIRQ	92
Table 5-48	HAL_UART_UnregisterIRQ	92
Table 5-49	HAL_UART_EnableTxReadyIRQ	93
Table 5-50	HAL_UART_EnableRxReadyIRQ	93
Table 5-51	HAL_UART_DisableTxReadyIRQ	93
Table 5-52	HAL_UART_DisableRxReadyIRQ	94
Table 5-53	HAL_UART_RegisterDMA	94
Table 5-54	HAL_UART_UnregisterDMA	95
Table 5-55	HAL_UART_StartTransmitDMA	95
Table 5-56	HAL_UART_StopTransmitDMA	96
Table 5-57	HAL_UART_StartReceiveDMA	96
Table 5-58	HAL_UART_StopReceiveDMA	96

Table 5-59	SPI Pin Functions	97
Table 5-60	HAL_SPI_Init	103
Table 5-61	HAL_SPI_DeInit	103
Table 5-62	HAL_SPI_Open	104
Table 5-63	HAL_SPI_Close	104
Table 5-64	HAL_SPI_ExtControl	104
Table 5-65	HAL_SPI_SetTransferMode	105
Table 5-66	HAL_SPI_Receive	106
Table 5-67	HAL_SPI_Transmit	106
Table 5-68	HAL_SPI_TransmitReceive	107
Table 5-69	HAL_SPI_GetRxData	108
Table 5-70	HAL_SPI_PutTxData	108
Table 5-71	HAL_SPI_StartTrans	108
Table 5-72	HAL_SPI_StopTrans	109
Table 5-73	HAL_SPI_IsTxReady	109
Table 5-74	HAL_SPI_IsTxEmpty	109
Table 5-75	HAL_SPI_IsRxReady	109
Table 5-76	HAL_SPI_IsBusy	110
Table 5-77	HAL_SPI_GetFifoLevel	110
Table 5-78	HAL_SPI_IsCsInterrupt	110
Table 5-79	HAL_SPI_EnableDMA	111
Table 5-80	HAL_SPI_DisableDMA	111
Table 5-81	HAL_SPI_EnableCS	111
Table 5-82	HAL_SPI_DisableCS	112
Table 5-83	HAL_SPI_RegisterIRQ	112
Table 5-84	HAL_SPI_UnregisterIRQ	113
Table 5-85	HAL_SPI_EnableIRQ	113
Table 5-86	HAL_SPI_DisableIRQ	114
Table 5-87	HAL_SPI_RegisterDMA	114
Table 5-88	HAL_SPI_UnregisterDMA	115
Table 5-89	HAL_SPI_StartDMA	115
Table 5-90	HAL_SPI_StopDMA	115
Table 5-91	HAL_SPI_PM_Suspend	116
Table 5-92	HAL_SPI_PM_Resume	116
Table 5-93	HAL_I2C_init	119
Table 5-94	HAL_I2C_Deinit	120
Table 5-95	HAL_I2C_Open	120
Table 5-96	HAL_I2C_Close	121
Table 5-97	HAL_I2C_RecoverBus	121
Table 5-98	HAL_I2C_SetTransferMode	121
Table 5-99	HAL_I2C_Transmit	122
Table 5-100	HAL_I2C_Receive	122
Table 5-101	HAL_I2C_GetRxData	124
Table 5-102	HAL_I2C_PutTxData	124
Table 5-103	HAL_I2C_IsTxReady	125
Table 5-104	HAL_I2C_IsRxReady	125
Table 5-105	HAL_I2C_IsTxEmpty	125
Table 5-106	HAL_I2C_RegisterIRQ	125
Table 5-107	HAL_I2C_UnregisterIRQ	126
Table 5-108	HAL_I2C_EnableIRQ	126
Table 5-109	HAL_I2C_DisableIRQ	127
Table 5-110	HAL_I2C_ClearIRQ	127
Table 5-111	HAL_I2C_GetIRQStatus	128
Table 5-112	HAL_DMA_Init	133

Table 5-113	HAL_DMA_DeInit	134
Table 5-114	HAL_DMA_Open	134
Table 5-115	HAL_DMA_Close	135
Table 5-116	HAL_DMA_ConfigBlockTransfer	135
Table 5-117	HAL_DMA_InitNodeCtrl	136
Table 5-118	HAL_DMA_AddNextNode	136
Table 5-119	HAL_DMA_ConfigListTransfer	137
Table 5-120	HAL_DMA_ConfigScatterGatherTransfer	137
Table 5-121	HAL_DMA_StartTransfer	138
Table 5-122	HAL_DMA_AbortTransfer	139
Table 5-123	HAL_DMA_RegisterIRQ	139
Table 5-124	HAL_DMA_UnRegisterIRQ	140
Table 5-125	HAL_DMA_EnableIRQ	140
Table 5-126	HAL_DMA_DisableIRQ	141
Table 5-127	HAL_DMA_DisableIRQ	141
Table 5-128	QER Type	145
Table 5-129	HAL_FLASH_Init	146
Table 5-130	HAL_FLASH_DeInit	146
Table 5-131	HAL_FLASH_Open	147
Table 5-132	HAL_FLASH_Close	147
Table 5-133	HAL_FLASH_Read	148
Table 5-134	HAL_FLASH_Write	148
Table 5-135	HAL_FLASH_Erase	149
Table 5-136	HAL_FLASH_ExtControl	149
Table 5-137	HAL_FLASH_Suspend	151
Table 5-138	HAL_FLASH_Resume	151
Table 5-139	HAL_WDG_Init	152
Table 5-140	HAL_WDG_DeInit	152
Table 5-141	HAL_WDG_Open	153
Table 5-142	HAL_WDG_Close	153
Table 5-143	HAL_WDG_Feed	154

1 Overview

The RS6 software development kit (SDK) is provided by Possumic technology for developing products based on RS6 series chips.

This document provides detailed instructions for the development stages that are generally required when using this SDK, such as creating projects, porting modules, and adding peripherals. It helps developers become familiar with the SDK more quickly and complete project development more easily and efficiently.

2 SDK Introduction

The RS6130 and RS6240 radar sensor chips from Possumic technology are ultra-low-power, high-performance 60 GHz mmWave radar sensor chips. They integrate a 32-bit RISC-V central processing unit (CPU) and a wide range of peripherals, and support execute in place (XIP), making them an ideal solution for low-power, high-performance intelligent sensing applications.

This SDK series uses the open-source FreeRTOS real-time embedded operating system (OS) to provide lighter, more flexible, and more comprehensive application services.

2.1 Architecture Overview

2.1.1 RS6130 Hardware Architecture

The RS6130 supports 1T3R radar and 2.4 GHz wireless communication (Bluetooth Low Energy (BLE) and 802.15.4). It integrates a RISC-V CPU with floating-point support and a maximum clock frequency of 128 MHz, provides 256 KB of system random-access memory (RAM) and a 128 KB Data Cube, supports Serial Peripheral Interface (SPI) Flash XIP, and supports common peripherals such as the Universal Asynchronous Receiver/Transmitter (UART), SPI, and Inter-Integrated Circuit (I2C).

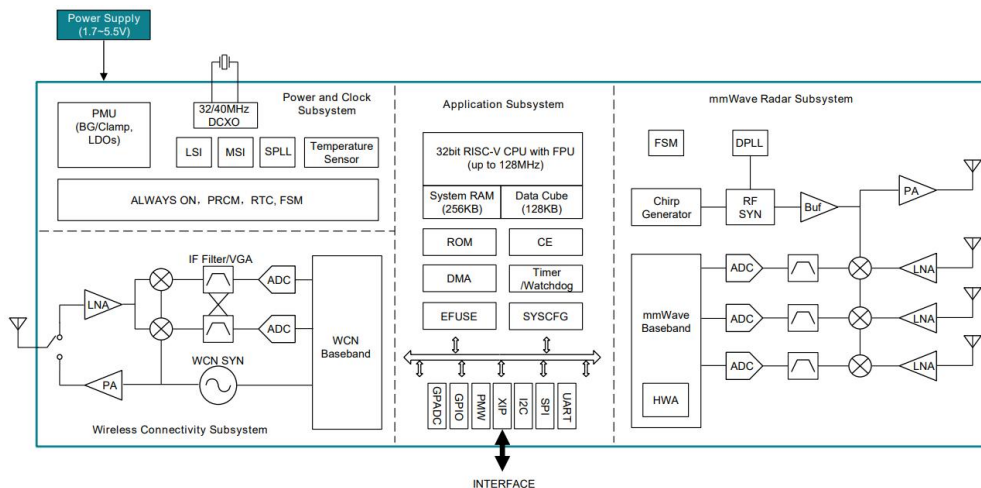


Figure 2-1 RS6130 System Block Diagram

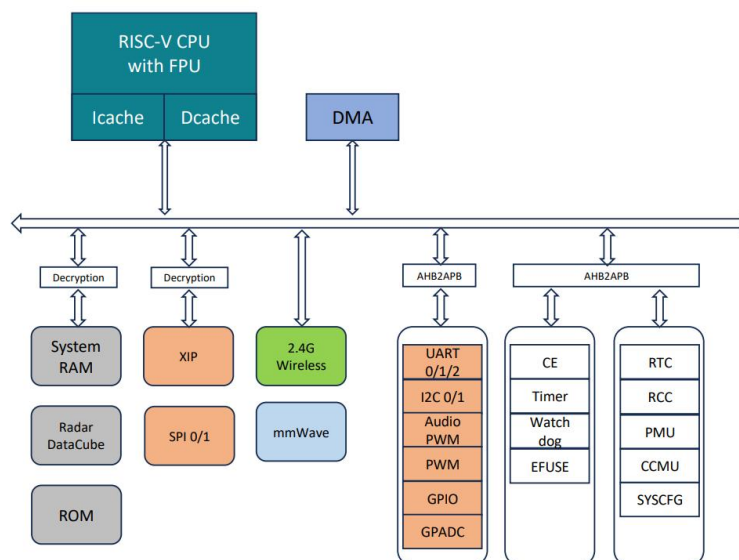


Figure 2-2 RS6130 Application Subsystem

2.1.2 RS6240 Hardware Architecture

The RS6240 supports 2T4R radar and 2.4 GHz wireless communication (BLE and IEEE 802.15.4). It integrates two RISC-V CPUs, one supporting floating-point instructions and the other supporting floating-point and digital signal processing (DSP) instructions, with maximum clock frequencies of 128 MHz and 256 MHz, respectively. It provides 295 KB of system RAM and a 256 KB Data Cube, supports SPI Flash/pseudo-static RAM (PSRAM) XIP and data storage, and includes common peripherals such as UART, SPI, and I2C.

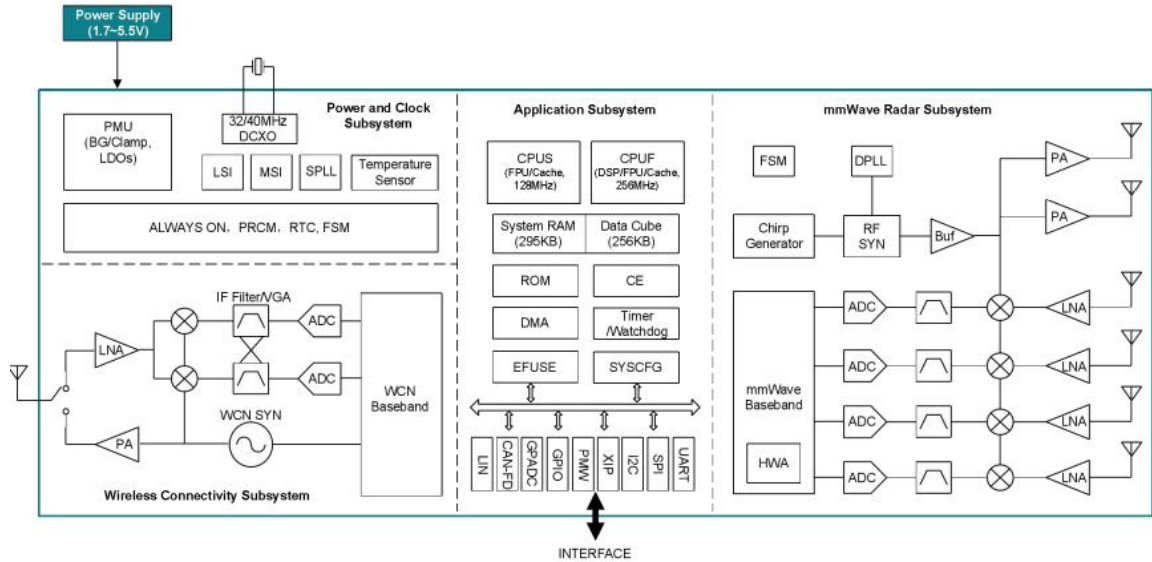


Figure 2-3 RS6240 System Block Diagram

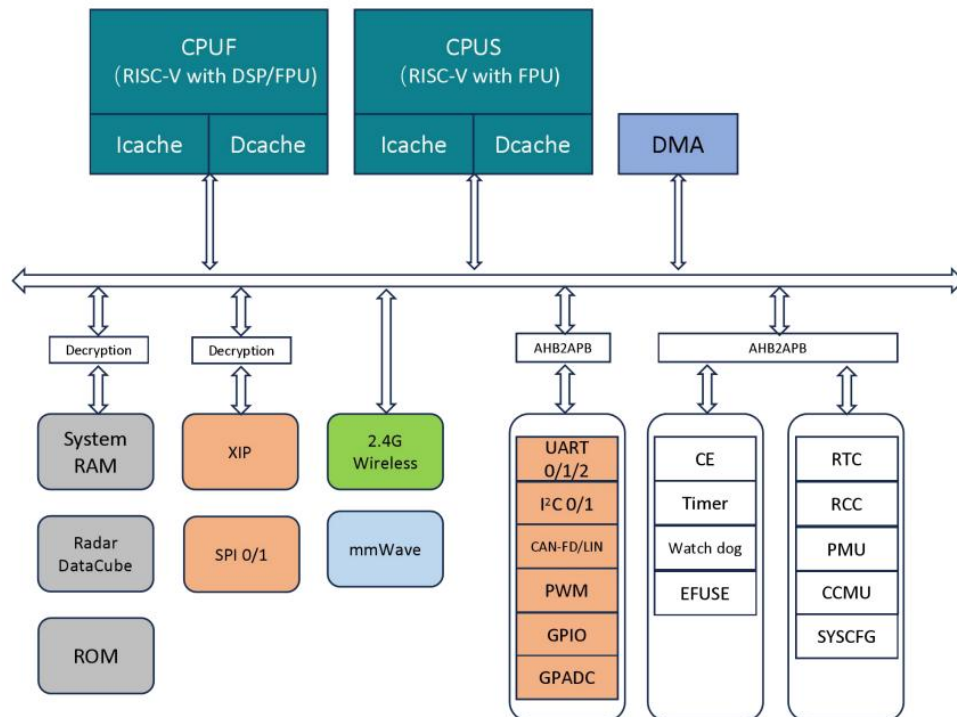


Figure 2-4 RS6240 Application Subsystem

2.1.3 SDK Software Architecture

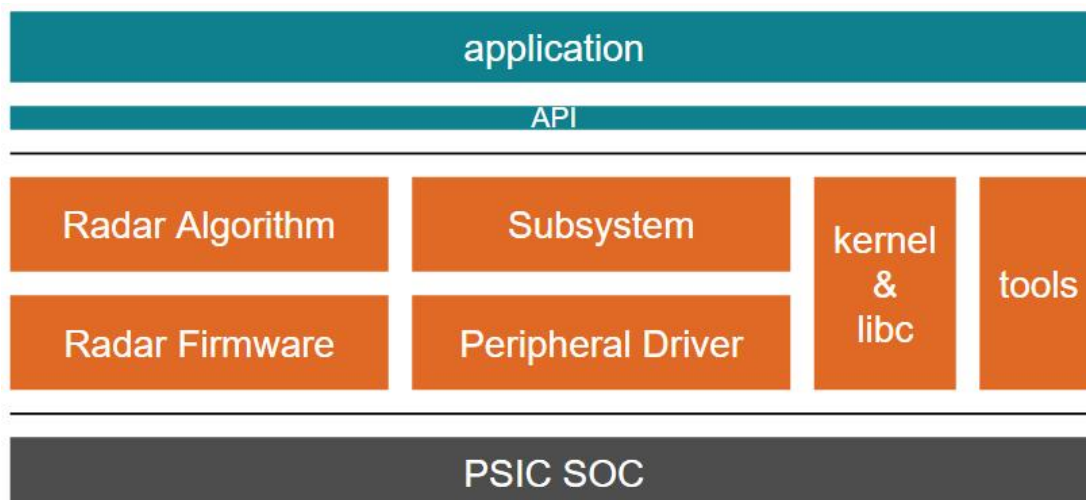


Figure 2-5 RS Series SDK Framework

The SDK is based on the PSIC SoC hardware platform and consists primarily of three layers:

1. Driver layer: Includes the radar controller driver and various peripheral drivers. The radar controller driver is provided as a separate binary file, while peripheral drivers are provided as source code. The current version does not support wireless communication drivers.
2. Middleware layer: Provides application components, including radar algorithms, file systems, the test framework, and the OTA subsystem.
3. Application layer: Contains radar applications and basic SoC samples implemented using the middleware application programming interfaces (APIs).

Below the application layer are also the operating system, C library, and various build helper scripts used throughout the SDK.

2.2 Directory Structure

The SDK consists of build tools and source code.

2.2.1 Build Tools

Build operations are performed using the integrated CDK tool. CDK provides code editing, compilation and linking, debugging, and analysis functions.

CDK must be installed before use. For instructions, see ['RS6xxx mmWave Radar Development Quick Start Guide.pdf'](#).

Note: 1. Online debugging through J-Link is not currently supported.

2.2.2 Software Source Code

The SDK source includes C code and scripts, organized as follows:

```

<home>/Software Kit/02 SDK/
psdf_sdk                               /* Software source code */
|__ kernel                             /* RTOS source code and OS abstraction layer */
|  |__ freeRTOS                        /* FreeRTOS source code */
|  |__ inc                             /* OSI header files for the OS abstraction layer */
|  |__ src                             /* OSI source files for the OS abstraction layer */
|
|__ libs                               /* Stores static libraries (.a) and binary files (.img) */
|  |__ bootloader                     /* bootloader binary file */
|  |__ dsp                            /* DSP static library */
|  |__ mmw                            /* radar control binary file and symbol table */

```

```

|   |__ spiflash                /* Configuration files for external SPI Flash/PSRAM chips */
|
|__ platform                    /* Drivers and configuration files for the chip platform */
|   |__ arch                    /* Files for the chip CPU architecture, such as startup assembly and
interrupt handling assembly */
|   |__ boards                  /* Drivers and configuration for solution boards, such as pin-mux */
|   |__ drivers                 /* Chip peripheral drivers, such as UART and GPIO */
|   |__ soc                     /* Chip configuration, such as address-region allocation and interrup
t numbers */
|   |__ utils                   /* Common utilities, such as version numbers, linker scripts, and pa
ckaging templates */
|
|__ project                     /* Application projects */
|   |__ basic                   /* Basic projects, such as a minimal system */
|   |__ driver                  /* HAL drivers, such as UART and FLASH */
|   |__ hello_word              /* hello_word project */
|   |__ mmwave                  /* Radar projects */
|   |   |__ radar_analysis      /* Radar analysis project used with the radar analysis tool */
|   |
|   |__ subsys                  /* Subsystem projects */
|       |__ hif                 /* HIF communication subsystem demo project */
|       |__ kvf                 /* Key-Value Flash storage subsystem demo project */
|       |__ kvr                 /* Key-Value RAM storage subsystem demo project */
|       |__ pm                  /* Power management subsystem demo project */
|       |__ shell               /* shell control subsystem demo project */
|
|__ subsys                      /* Functionally independent subsystems */
|   |__ cmd                     /* shell command set */
|   |__ log                     /* Printed LOG output */
|   |__ dsp                     /* DSP library */
|   |__ hif                     /* HIF (Host Interface) communication protocol */
|   |__ kv                      /* Key-Value storage */
|   |   |__ kvr                 /* Key-Value RAM storage; data is lost after power-off */
|   |   |__ kvf                 /* Key-Value Flash storage; data is retained after power-off */
|   |
|   |__ libc                    /* C library adaptation */
|   |__ mmw                     /* Radar driver and algorithm subsystem */
|   |__ pm                      /* PM power management subsystem */
|   |__ test                    /* Test project */
|   |__ testsuite               /* Test framework */
|   |__ shell                   /* shell subsystem */
|
|__ tools                       /* Tool scripts */
|   |__ python                  /* Portable Python package */
|   |__ scripts                 /* Scripts, such as img packaging */

```

2.3 References

For more information about FreeRTOS usage and conventions, see the following website:

<https://www.freertos.org>

For more information about CDK usage and conventions, refer to CDK Help:

[Open CDK, click Help on the toolbar, select "CDK help", and then select "xxx Doc".](#)

You can also refer to the following document:

['RS6xxx 7x mmWave Radar Development Quick Start Guide.pdf'](#)

3 Create a Project

The SDK architecture and directory structure have now been introduced. This chapter describes how to create a project and a board.

This section uses the creation of "myproject" under the project directory as an example to describe how to create an application project.

3.1 Project Directory Structure

Before creating a project, become familiar with the project directory structure. The sample project folder path is as follows:

`"<home>\MRS2xx SDK\Software Kit\02 SDK\project\basic\new_project"`

Open [project.cdkproj](#) in this directory. After a successful build, the project directory structure is as follows:

```
new_project
|__ app                /* Project source directory */
|  |__ inc             /* Project header files */
|  |  |__ prj_config.h /* Project configuration file */
|  |  |__ src          /* Project C files */
|  |  |__ main.c       /* Project application entry file */
|  |  |__ xxx          /* Additional directories and files can be added */
|  |
|  |__ cdg             /* CDK configuration and working directory */
|  |  |__ after_build.sh /* Startup script for packaging output img files*/
|  |  |__ project.cdkproj /* CDK configuration file */
|  |  |__ Lst          /* Startup script for packaging output img files*/
|  |  |  |__ psdf.asm   /* Generated assembly file */
|  |  |  |__ psdf.map   /* Generated symbol-to-address mapping file */
|  |  |__ Obj          /* Generated files */
|  |  |__ output       /* Startup script for packaging output img files*/
```

prj config is the project configuration file used to centrally manage hardware parameters, software features, global variables, and other project settings.

3.2 Project Creation

Step A: Copy the [project\basic\new_project](#) folder to the project directory and rename it "myproject".

Note: 1. new_project is an empty project template.

Step B: Go to `project\myproject\cdk` and double-click `project.cdkproj` to open CDK.

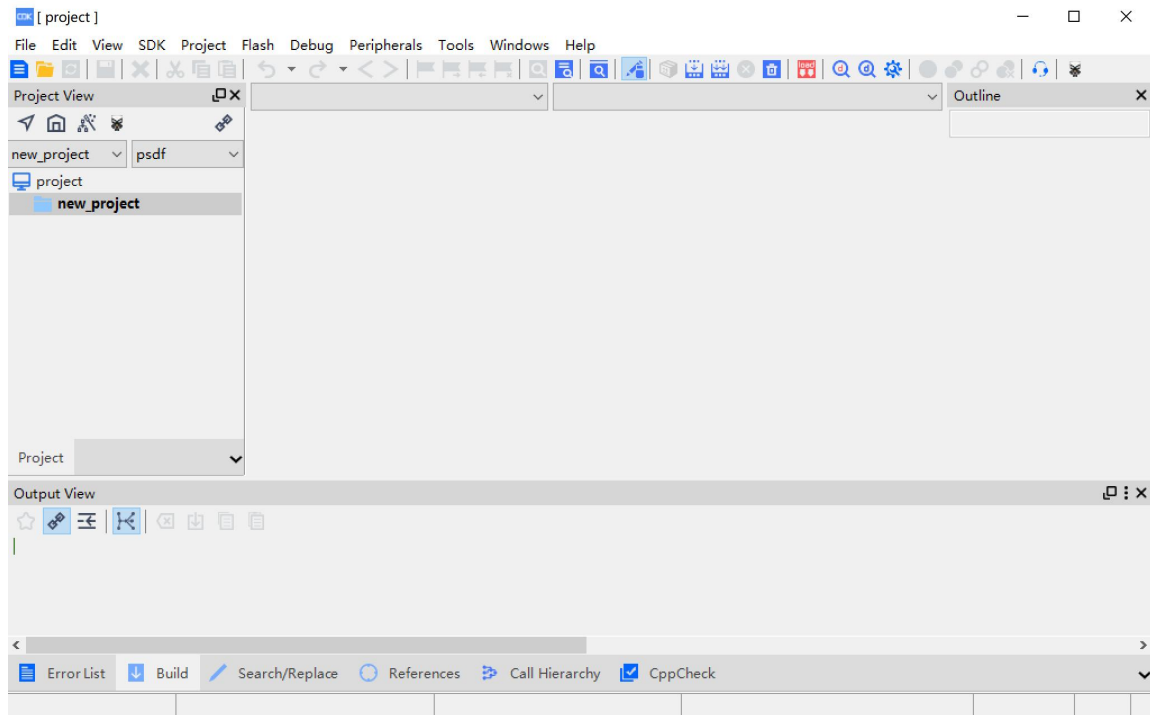


Figure 3-1 new_project User Interface

Step C: Right-click new_project, select "Rename Project" from the menu, enter "myproject" in the dialog box, and click "OK".

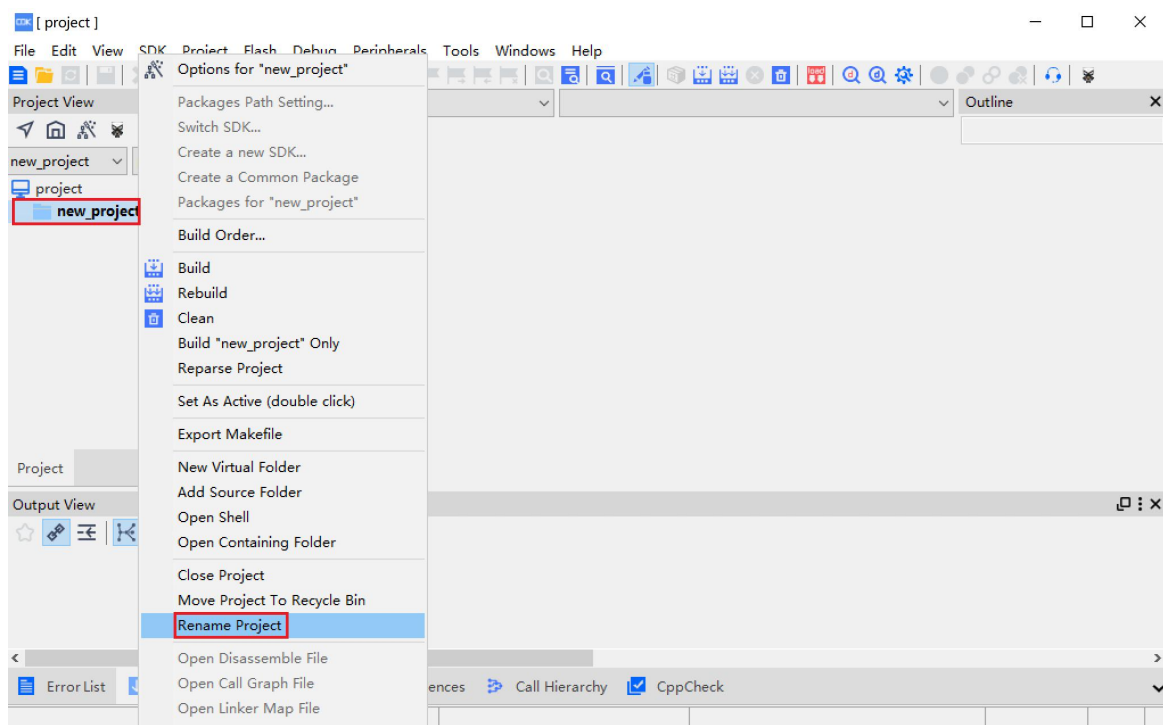


Figure 3-2 Rename Project Menu

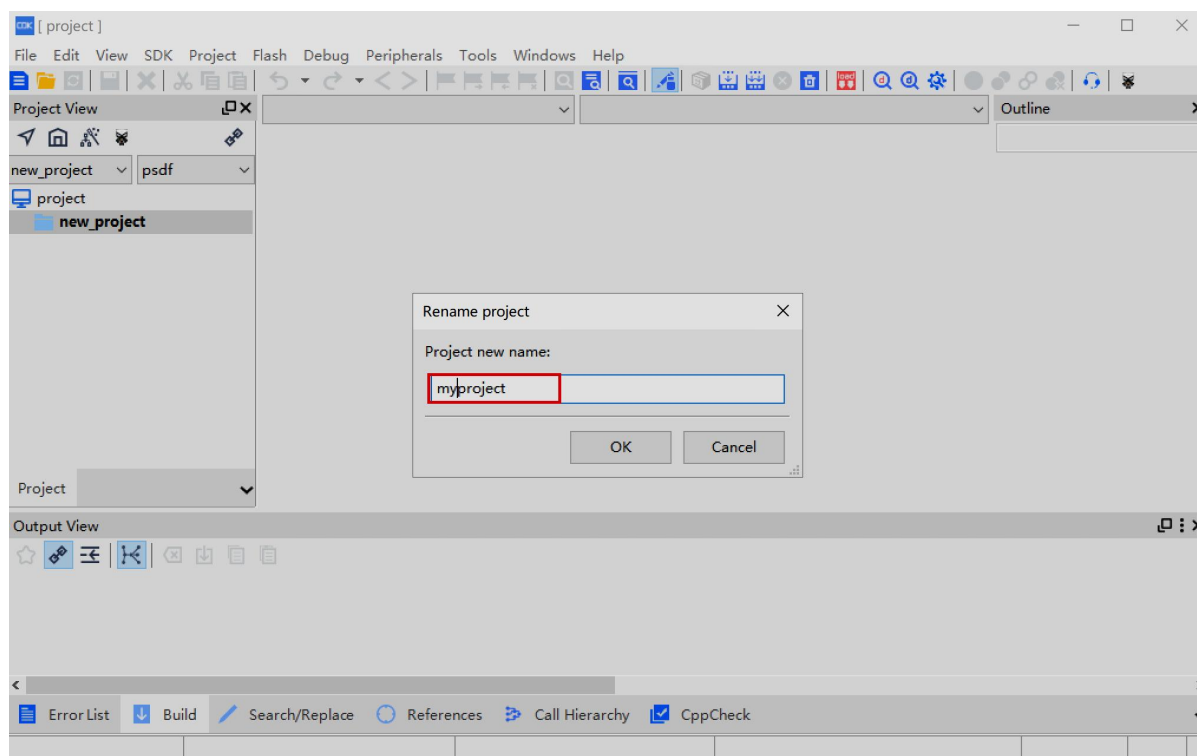


Figure 3-3 Enter the Project Name

3.2.1 Add Source Files

Add source files and header-file paths to the project.

Step D-1: Right-click myproject and select "Add Source Folder" from the menu.

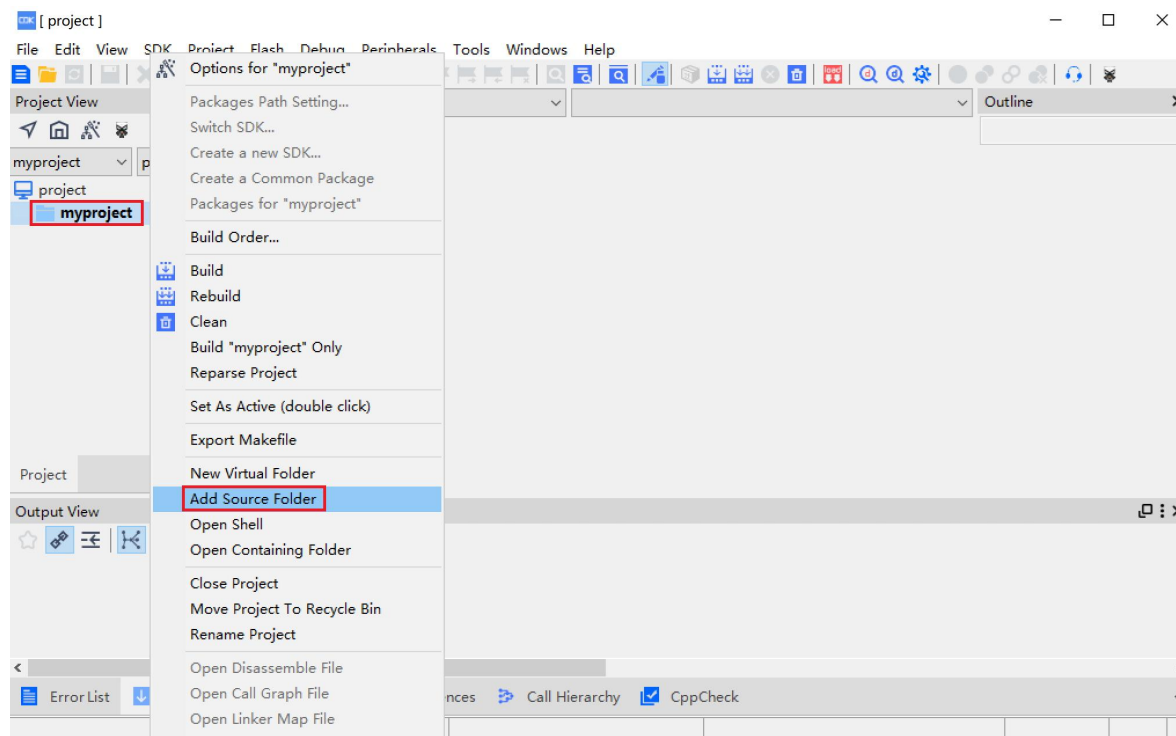
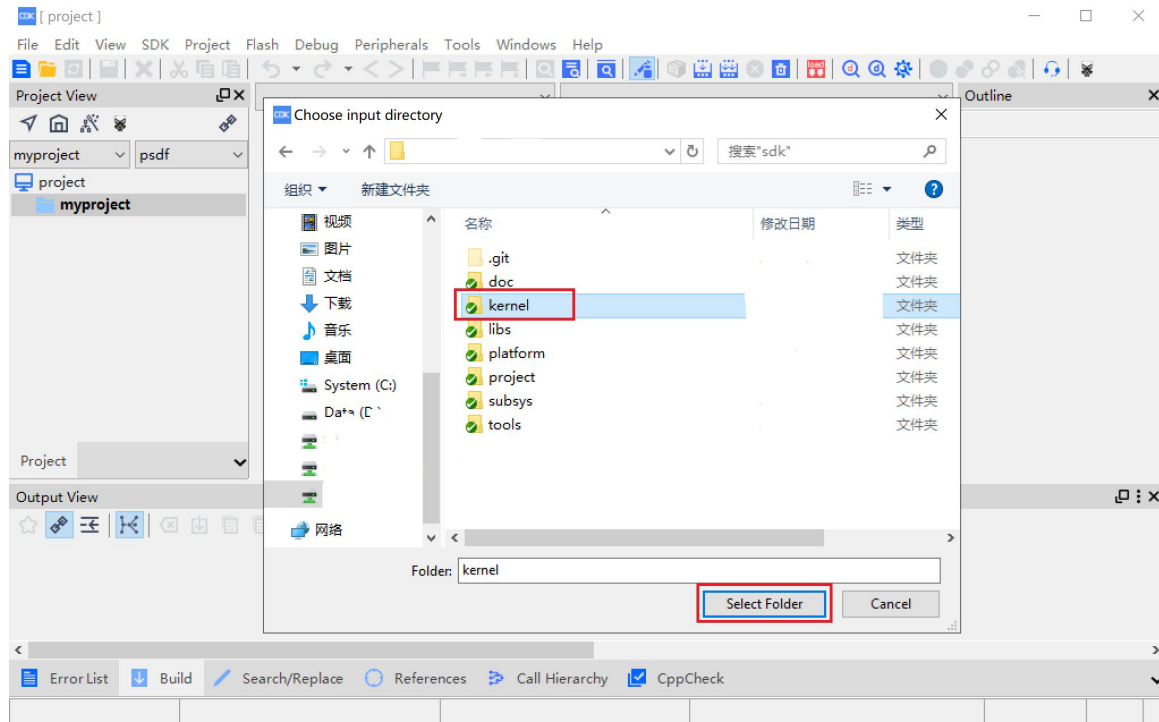
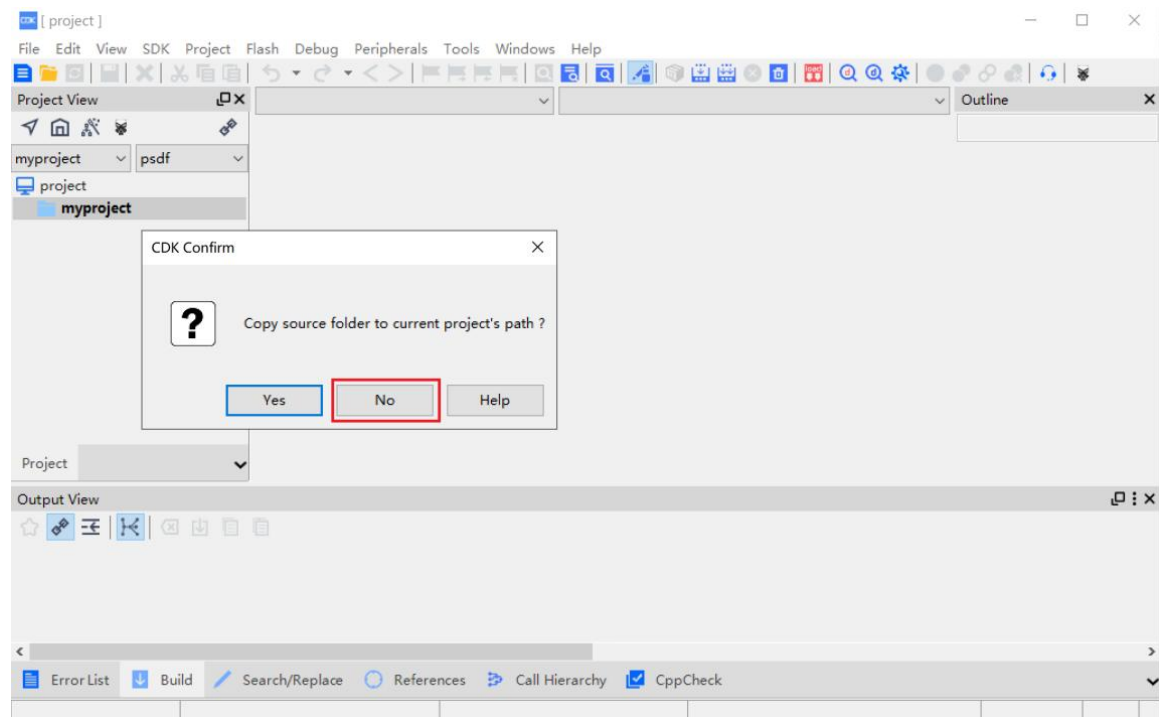


Figure 3-4 Add a Source Folder

Step D-2: Select the kernel folder in the SDK.**Figure 3-5 Select the kernel Folder****Step D-3: Select "No" in the dialog box.****Figure 3-6 Select the Option Not to Copy the Folder**

Step D-4: Repeat Steps D-2 and D-3 to add the platform and subsys virtual folders. Unnecessary source files or folders under subsys can be excluded from the build. No files under kernel or platform need to be excluded.

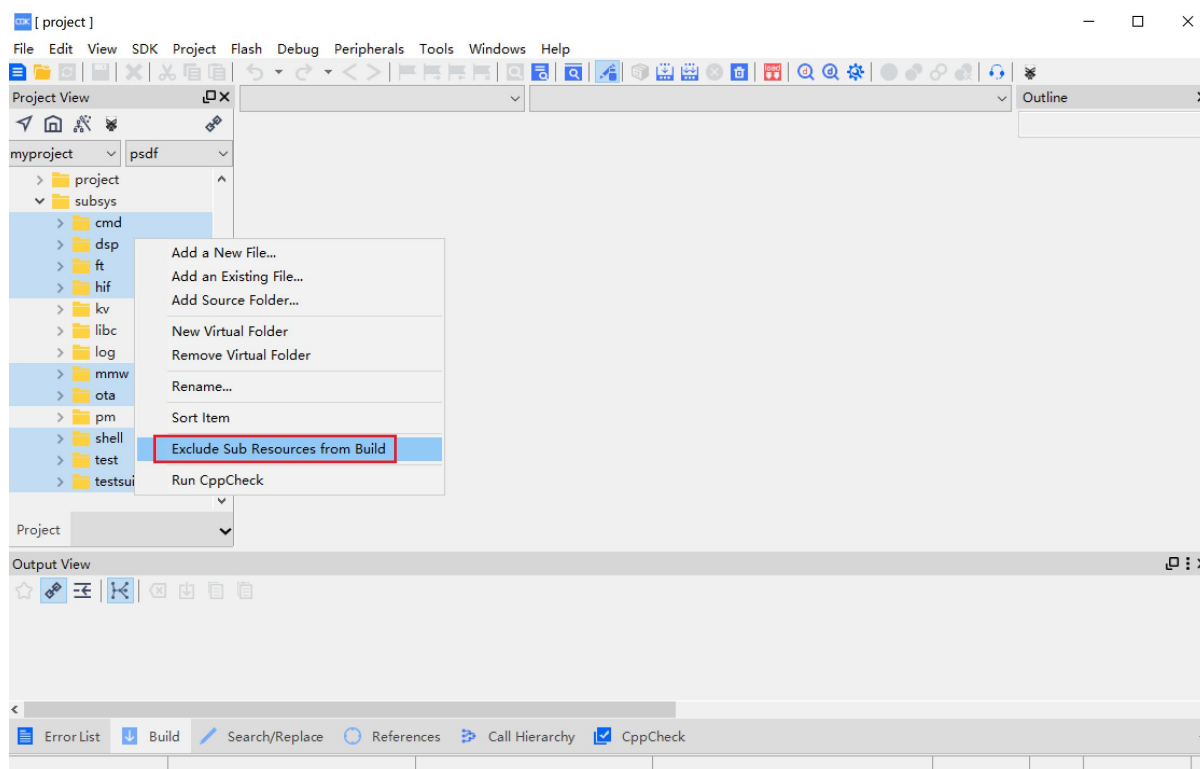


Figure 3-7 Exclude a Folder from the Build

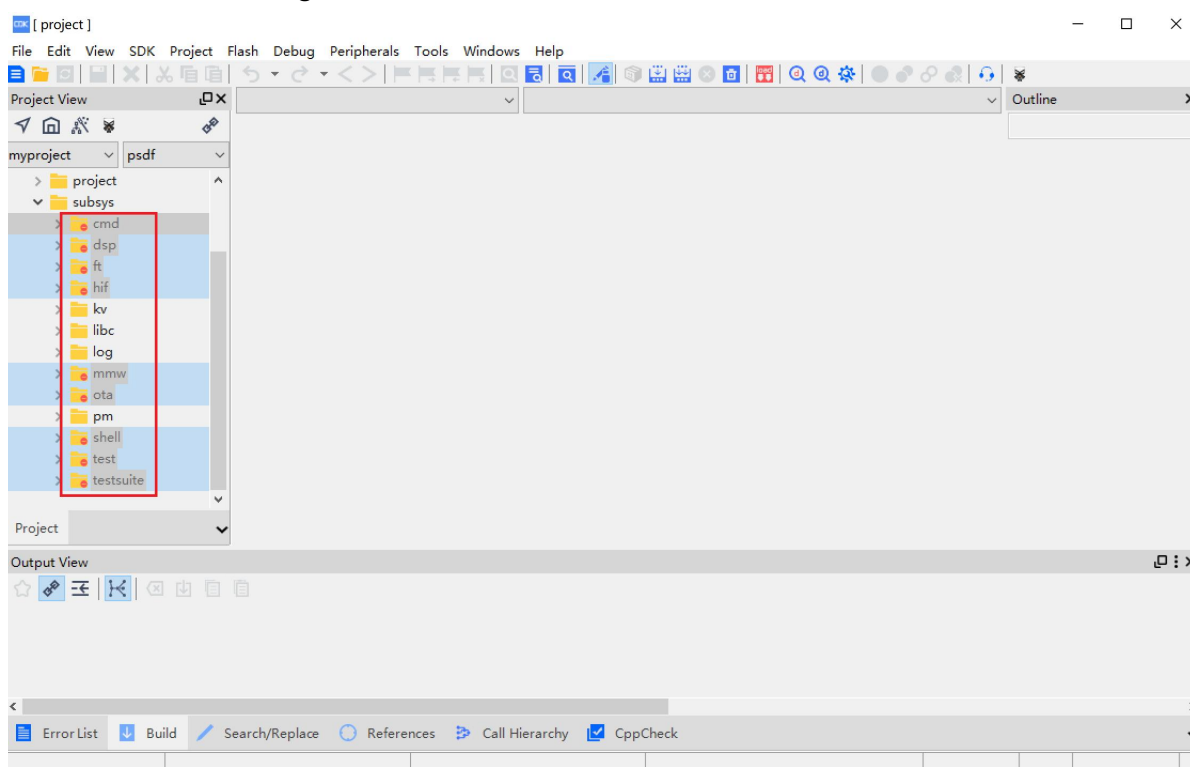


Figure 3-8 Icon for a Folder Excluded from the Build

Step D-5: Add the project application files. Right-click myproject, select "New Virtual Folder," and enter "project" as the folder name.

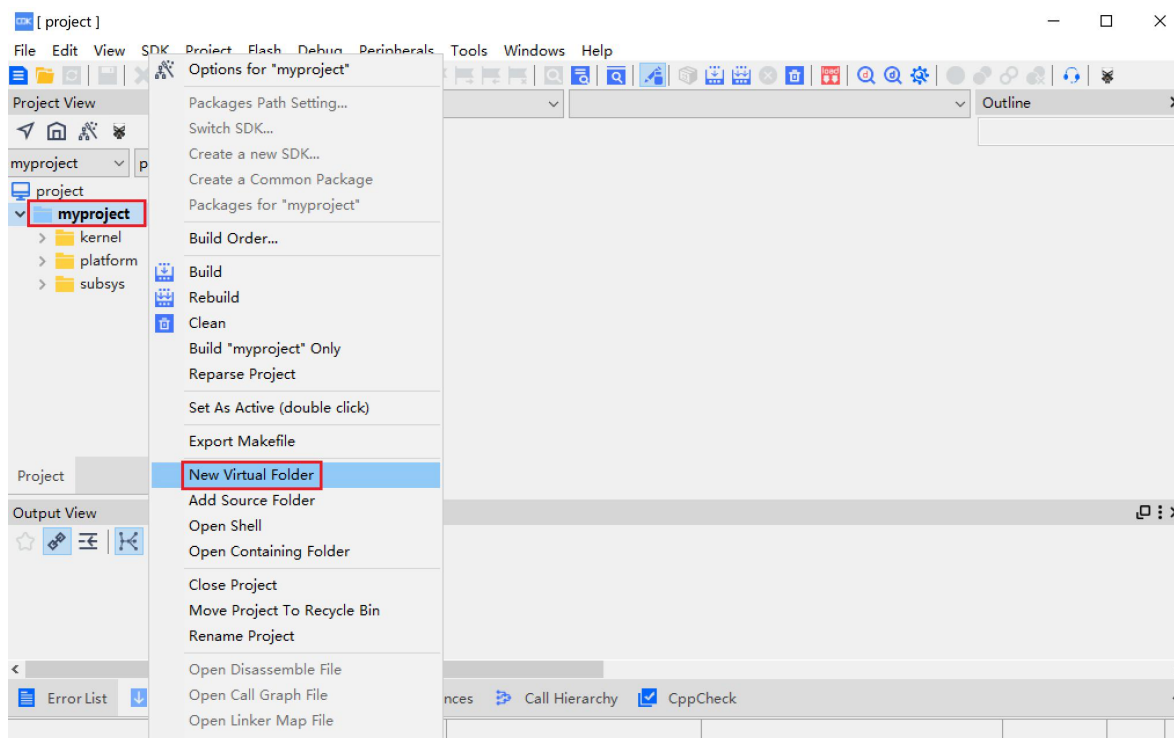


Figure 3-9 Create a Virtual Folder

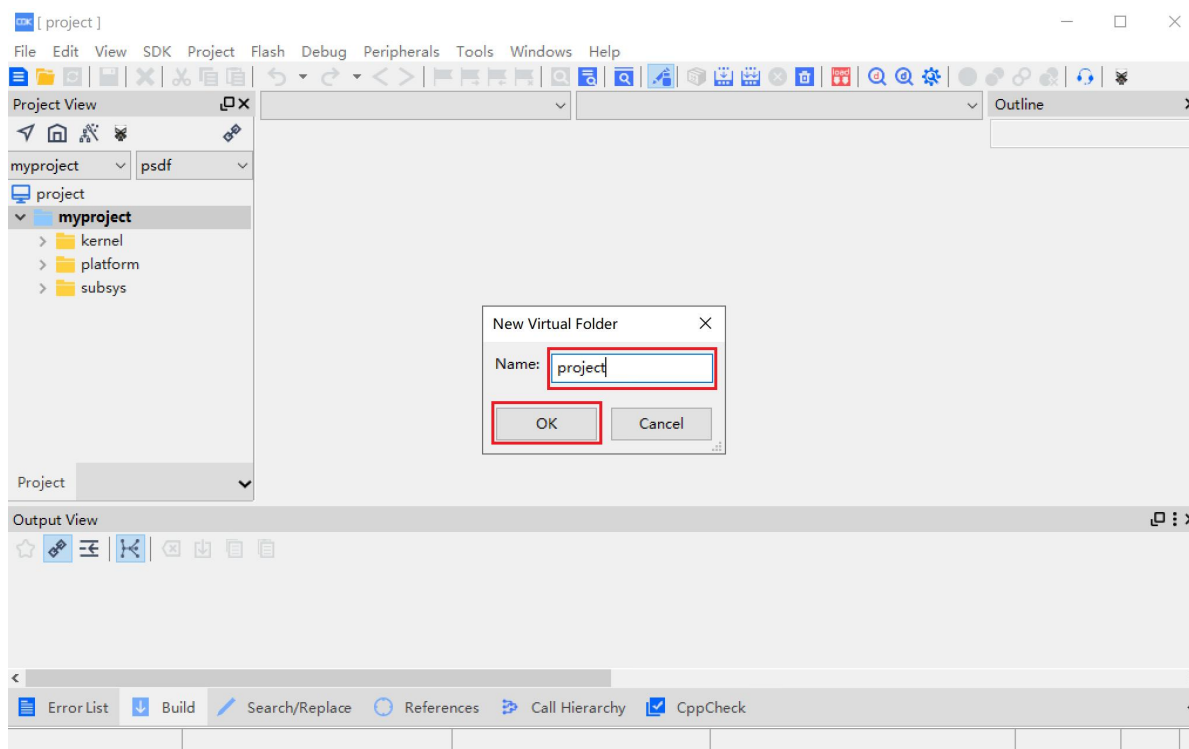


Figure 3-10 Enter the Virtual Folder Name

Step D-6: Right-click project, select "Add Source Folder," and select the app folder.

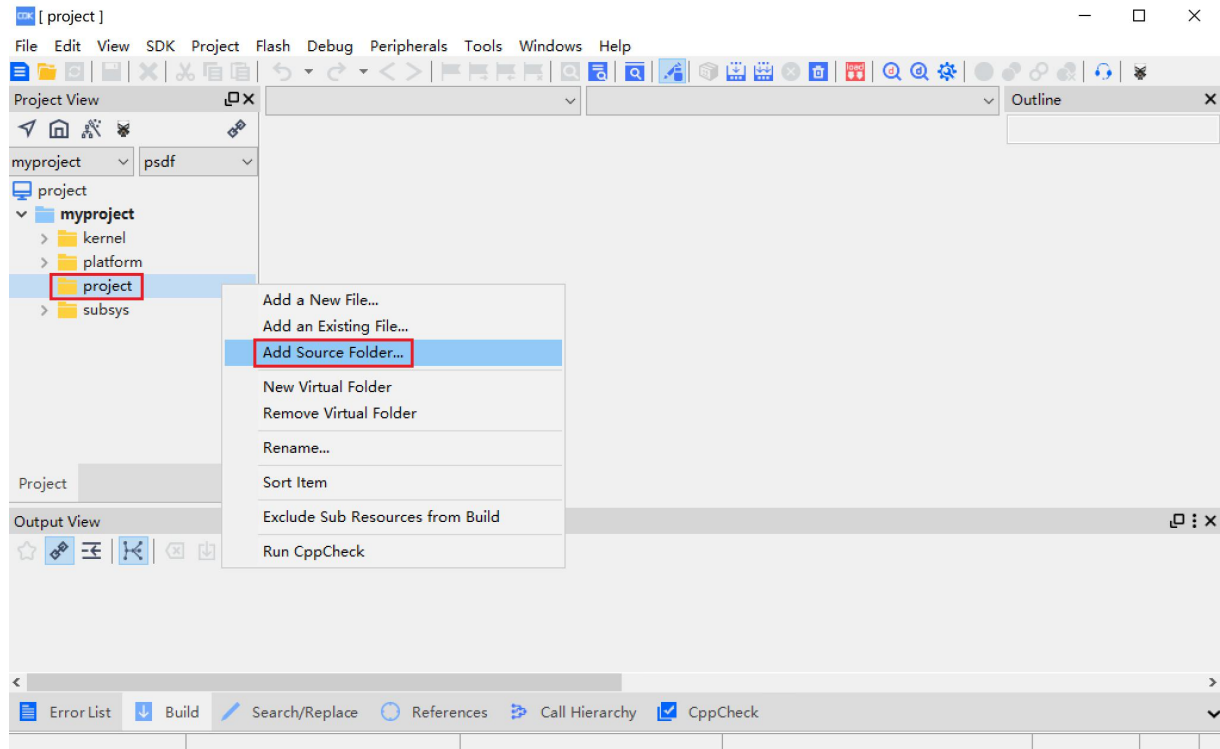


Figure 3-11 Add Source Files to a Virtual Folder

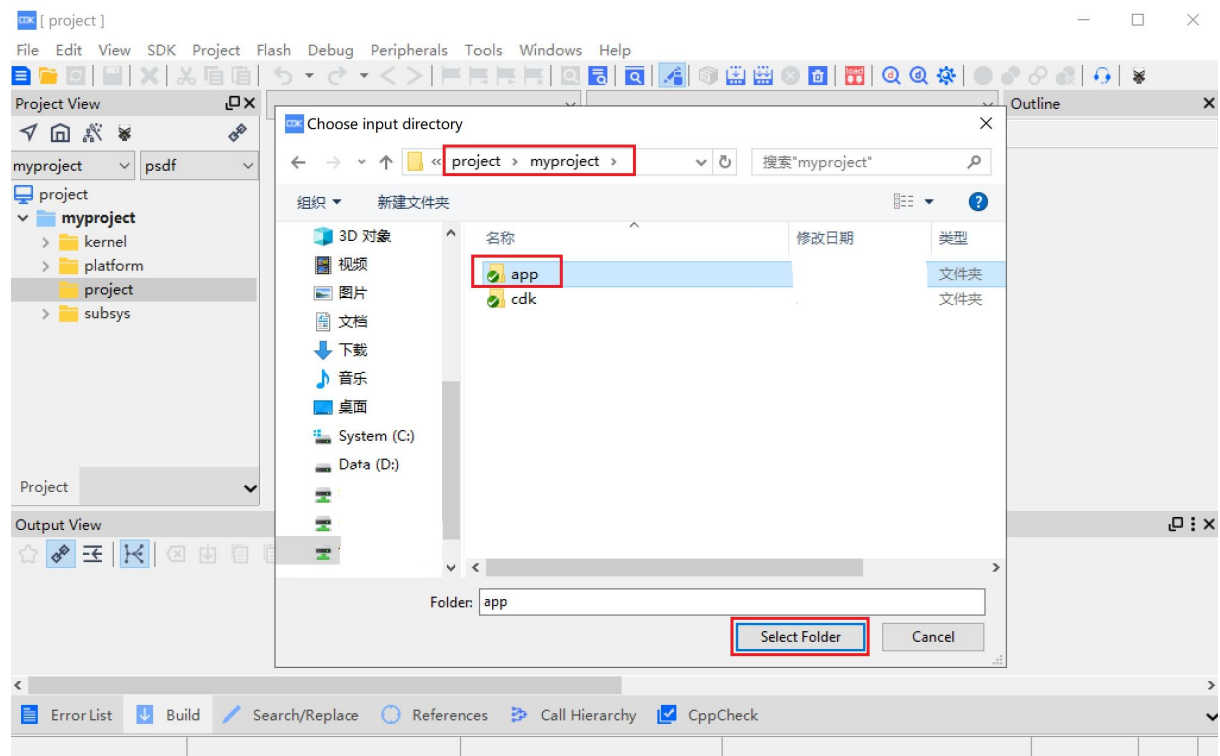


Figure 3-12 Select Source Files for the Current Project

3.2.2 Project Configuration

Step E: Modify the project configuration.

Step E-1: Right-click myproject and select "Options for 'myproject'". Project Setting is a graphical configuration interface provided by CDK for the build process and basic project properties, including chip selection, toolchain configuration, and basic linker configuration.

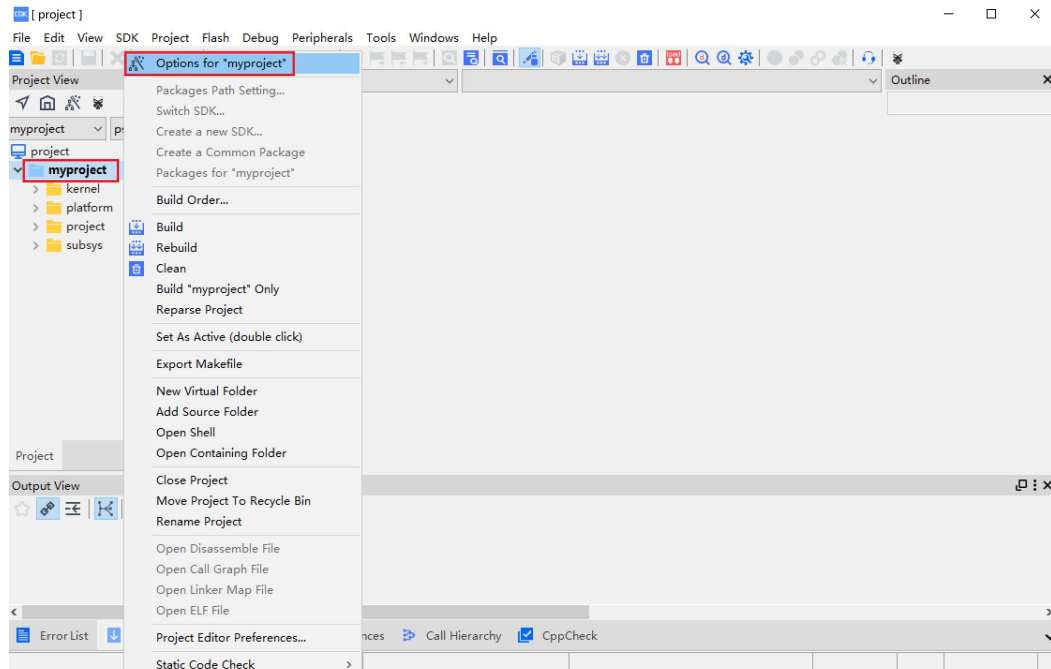


Figure 3-13 Select Source Files for the Current Project

Step E-2: Configure the toolchain in the Target menu as shown below. Do not modify these settings. Select "Use Stdlib."

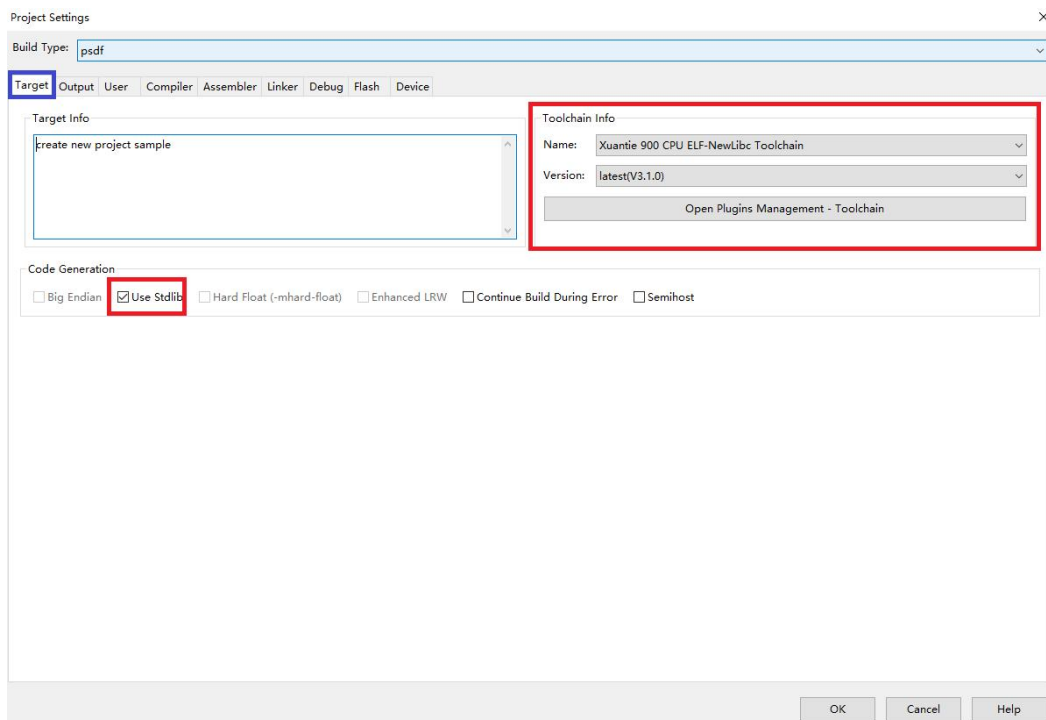


Figure 3-14 Target Configuration Screen

Step E-3: Configure the Output menu as shown below. Do not change the output name "psdf"; otherwise, img packaging may fail. Enable ELF output. Enabling ASM and MAP output is recommended for debugging.

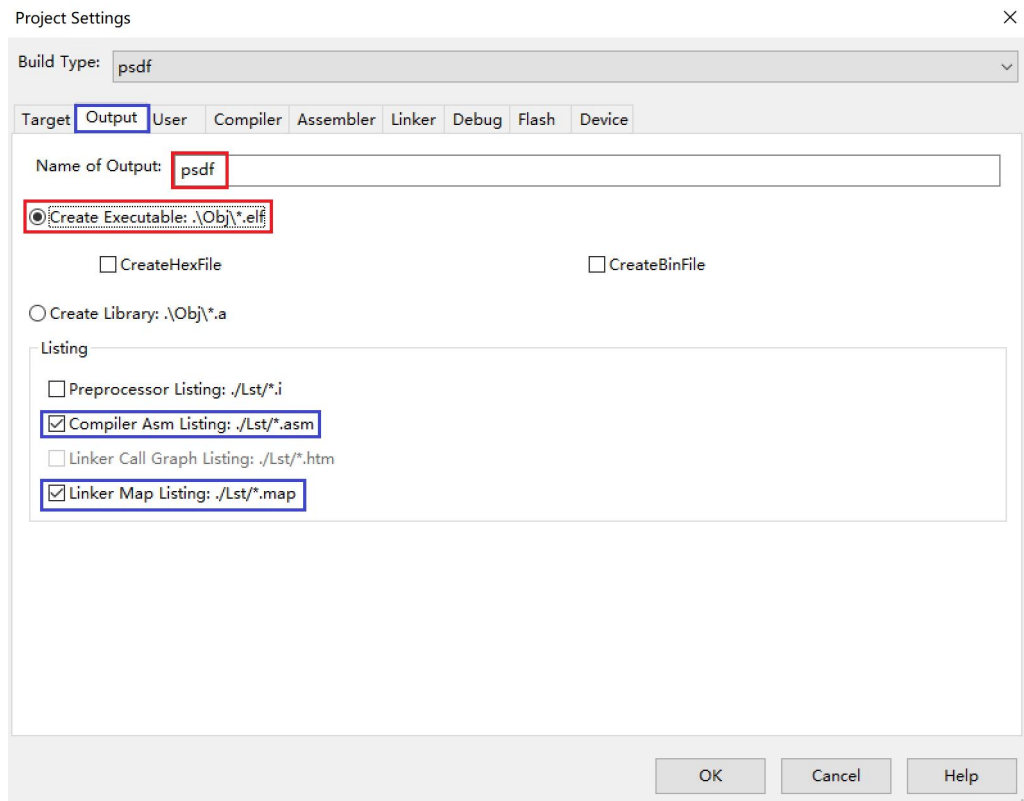


Figure 3-15 Output Configuration Screen

Step E-4: Configure the User menu as shown below. Enable the post-build script to start img packaging.

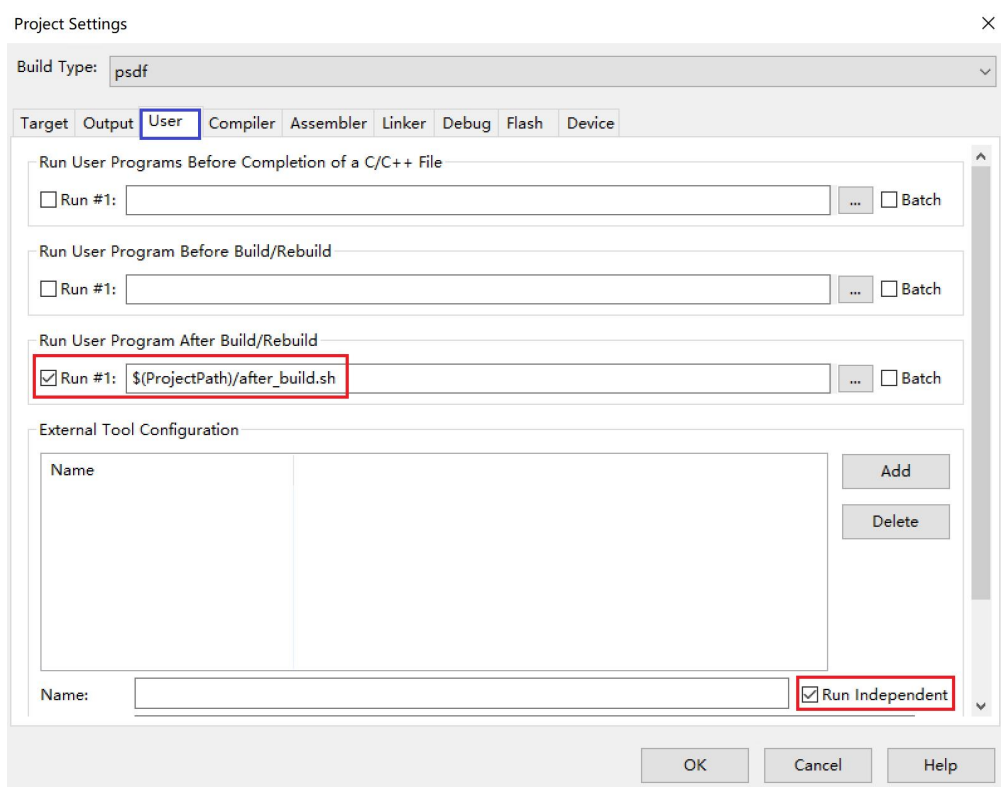


Figure 3-16 User Configuration Screen

Step E-5: Check the Compiler and Assembler settings shown below. Do not modify the Flag fields. Header-file paths were added automatically in Step D. Select -Os for Optimization. Use -O3 with caution because it may cause problems.

-MP -MMD -Wno-main -mmodel=medlow -Wno-unused-function -Wpointer-arith -Wno-undef -Wall -ffunction-sections -fdata-sections -fno-inline-functions -fno-builtin -fno-strict-aliasing

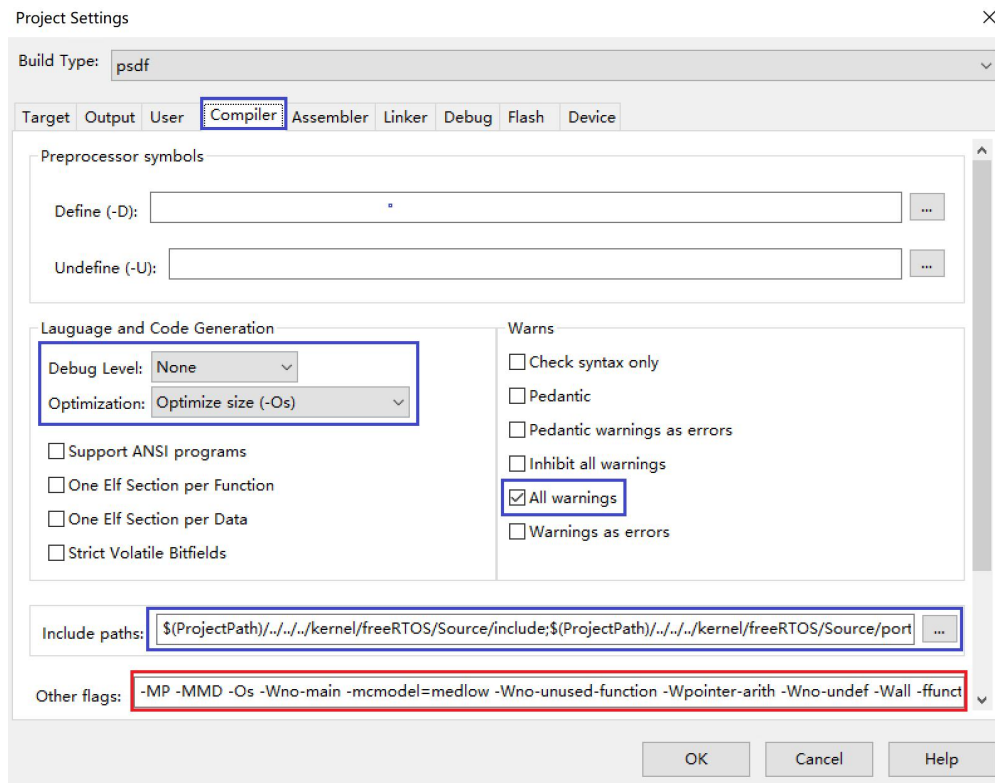


Figure 3-17 Compiler Configuration Screen

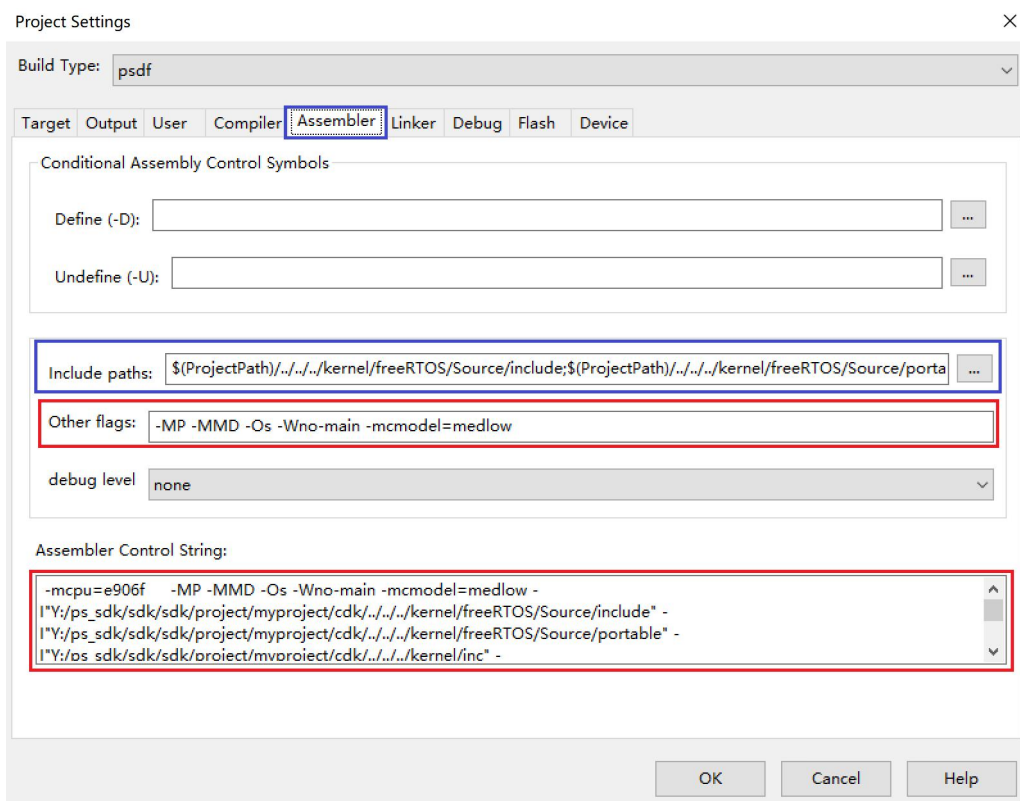


Figure 3-18 Assembler Configuration Screen

Step E-6: Check the Linker settings shown below. Do not modify the Flag field. Select the Linker File path according to the required linker script.

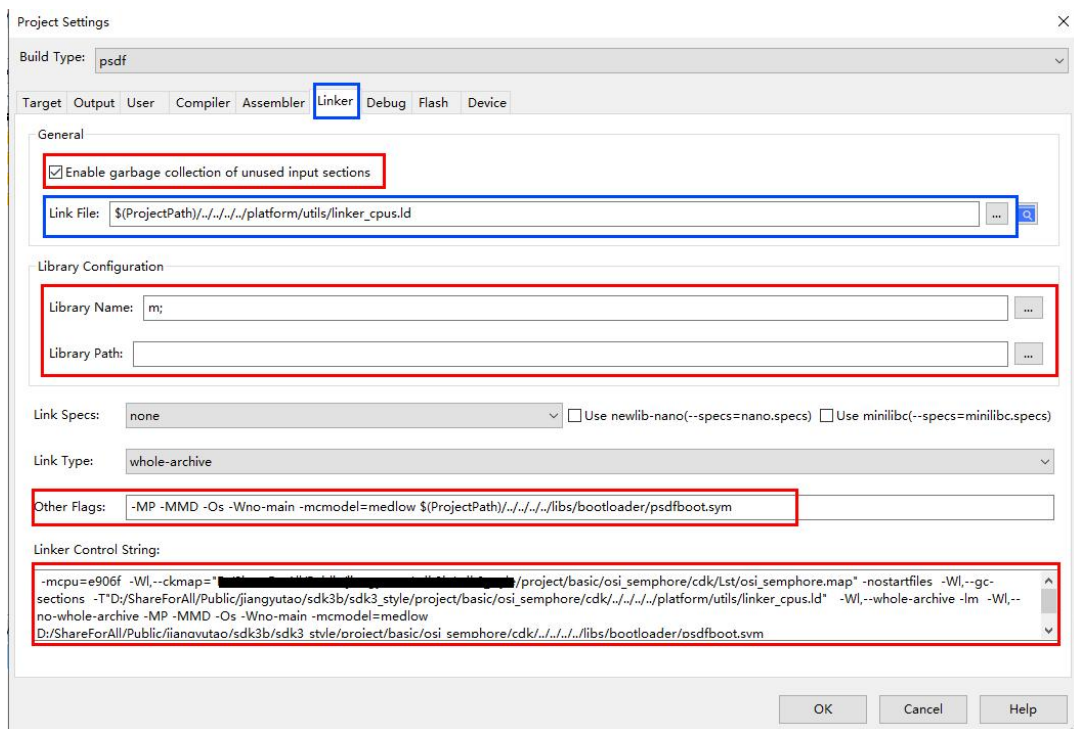


Figure 3-19 Linker Configuration Screen

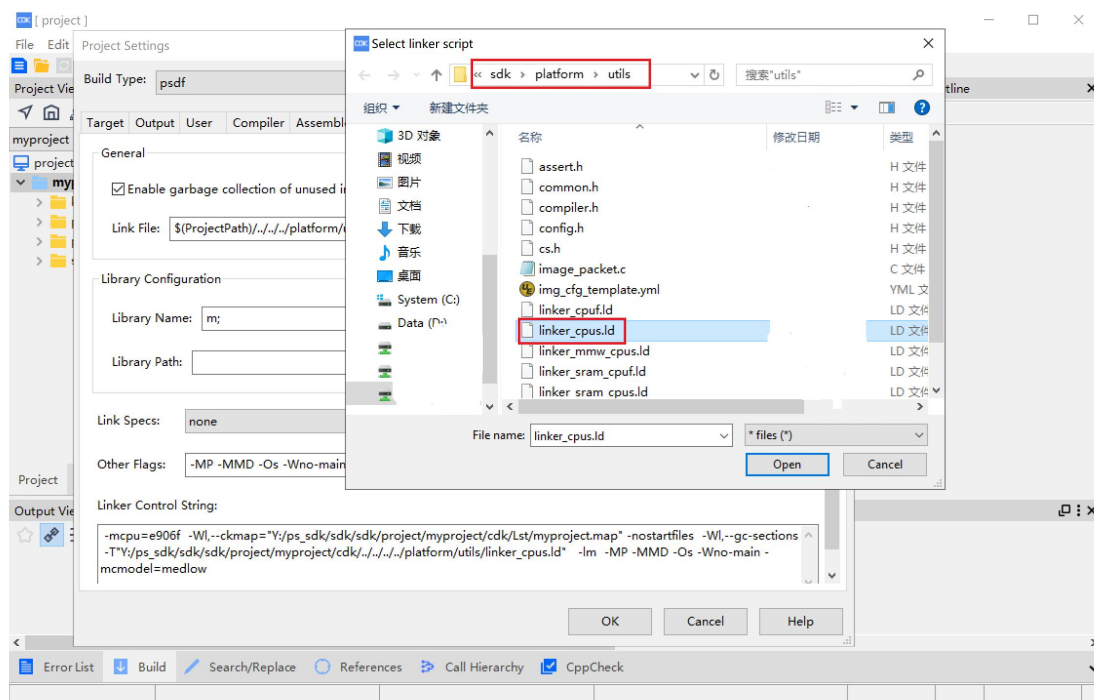


Figure 3-20 Select the Linker File

Step E-7: Configure the Device menu as shown below. Select "E906F" for CPUS or "E906FP" for C PUF.

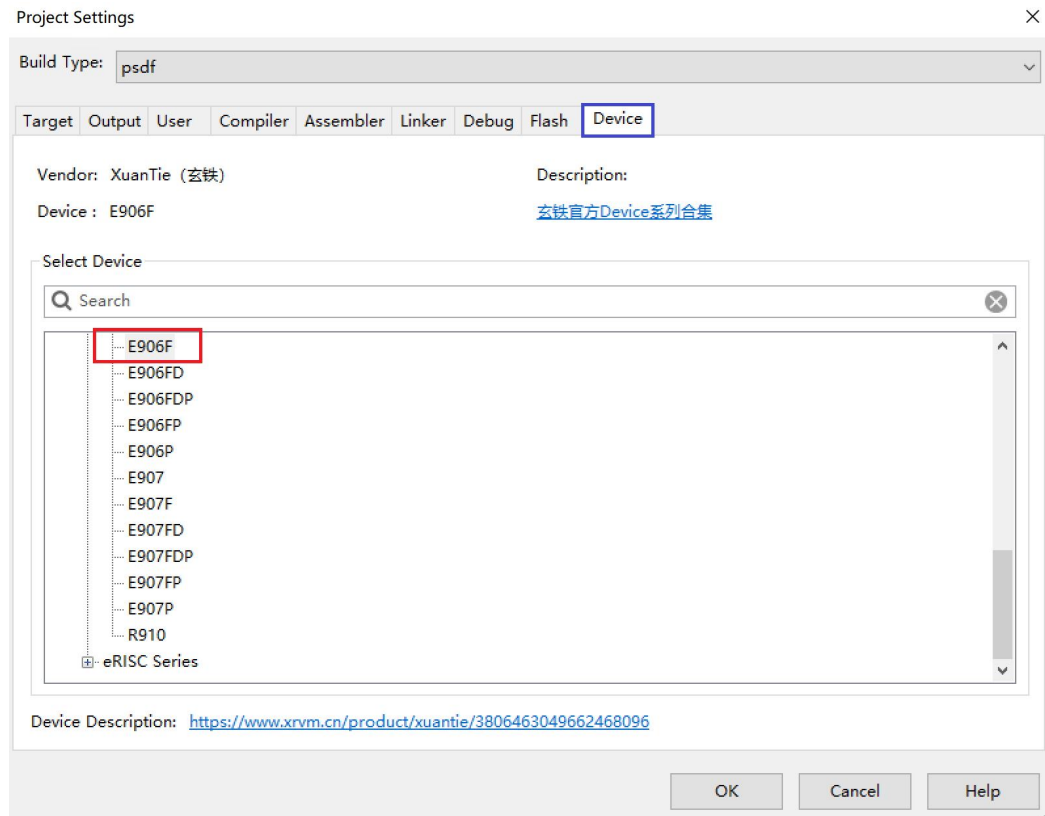


Figure 3-21 Device Configuration Screen

3.2.3 Board Selection

Step F: Select the required board in "Options for 'myproject'" and add it on both the Compiler and Assembler pages (see Figure 3-22). The available boards are:

CONFIG_BOARD_MRS6130_P1806

CONFIG_BOARD_MRS6130_P1812

CONFIG_BOARD_MRS6240_P2512_CPUS

CONFIG_BOARD_MRS6240_P2512_CPUF

Then add the application code to the main function (the application entry point).

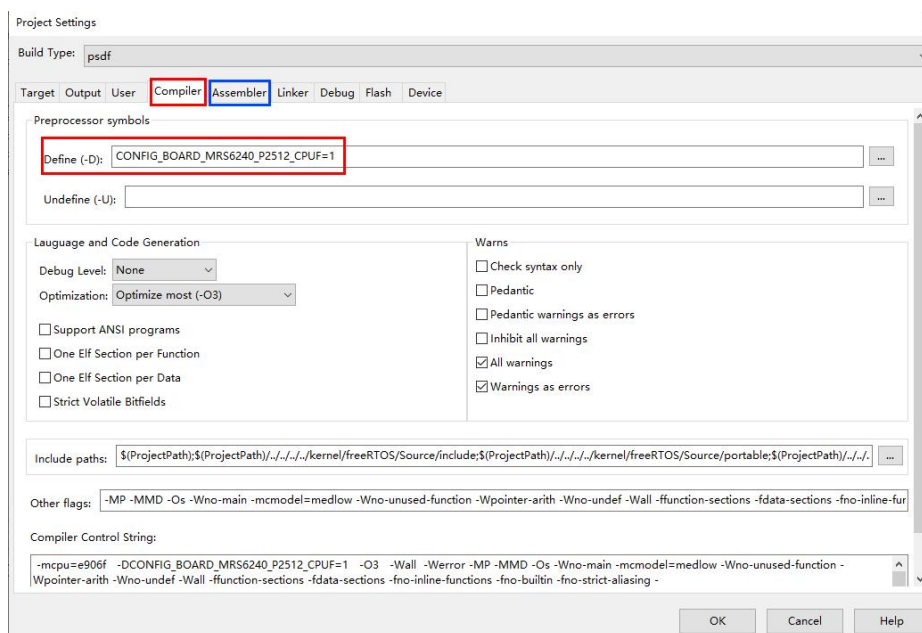


Figure 3-22 Select the Board in Options for myproject

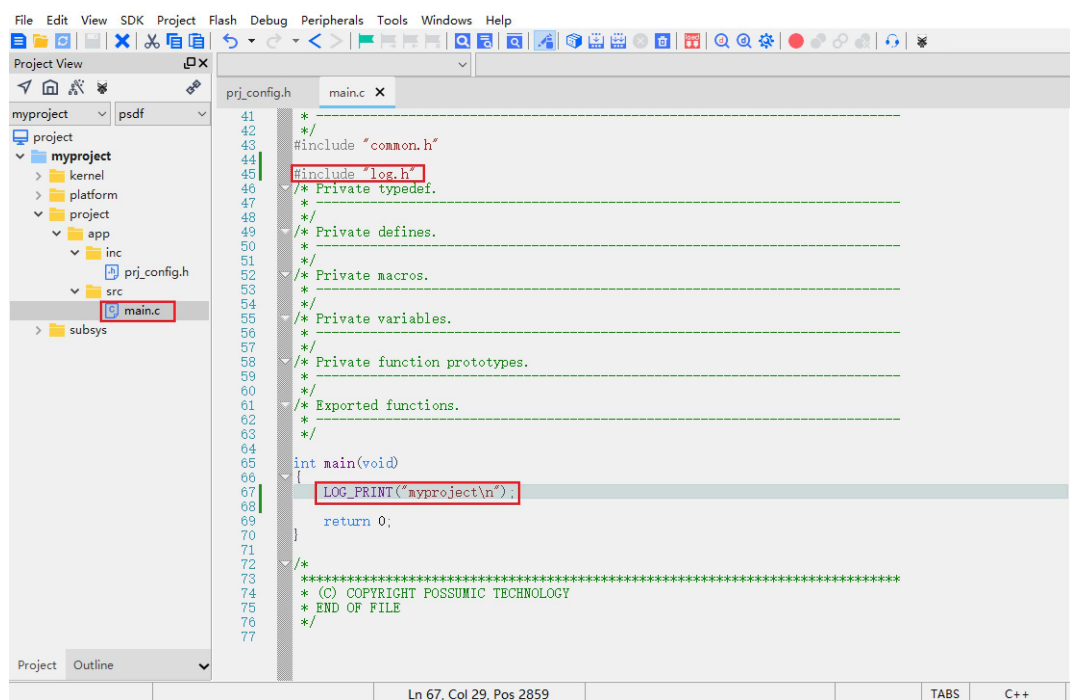


Figure 3-23 Add Application Code to the main Function

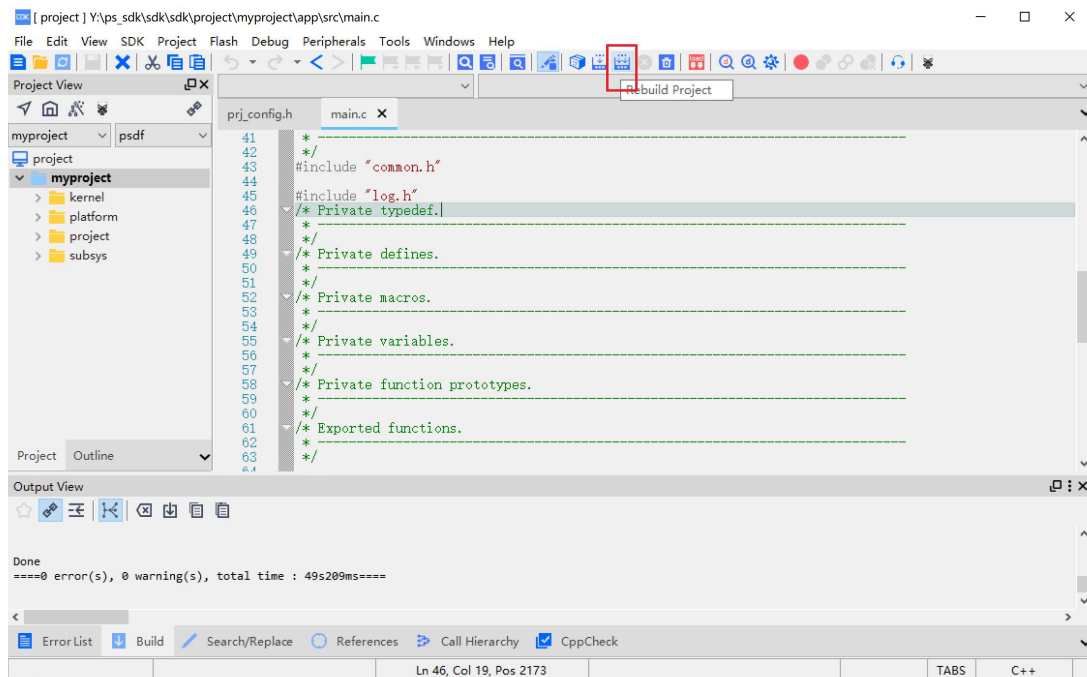


Figure 3-24 Build the Project

3.3 Create a Development Template

During project implementation, pinmux, operating frequency, and subsystems may need to be configured for the solution. The following example creates a board named "myboard" and selects rs6130 and rs6240 as the SoCs.

3.3.1 Create a Board Based on RS6130

Step A: Add the "myboard" folder and its files.

Step A-1: Go to [platform/boards](#), copy the mrs6130_p1812 folder, and rename it myboard.

Step A-2: Go to myboard and rename [mrs6130_p1812.c](#) and [mrs6130_p1812_config.h](#) to [myboard.c](#) and [myboard_config.h](#).

Step A-3: Open [myboard_config.h](#) and modify the following required items.

```
#ifndef _MYBOARD_CONFIG_H    /* Modify the header-file include guard */
#define _MYBOARD_CONFIG_H

#if CONFIG_BOARD_MYBOARD    /* BOARD configuration macro */

#define CONFIG_BOARD_STRING    "myboard"    /* Board name */
#define CONFIG_SOC_RS6130    1    /* Selected SoC */

#endif /* CONFIG_BOARD_MYBOARD */

#endif /* _BOARD_CONFIG_H */
```

Step A-4: Open [myboard.c](#) and modify the following required items.

```
#if (CONFIG_BOARD_MYBOARD)
```

```
#define LOG_MODULE                "myboard"
#define LOG_LEVEL                CONFIG_BOARD_LOG_LEVEL
#include "log.h"

#endif /* CONFIG_BOARD_MYBOARD */
```

Step B: Modify the highlighted section in [platform/boards/board_config.h](#).

```
#ifndef CONFIG_BOARD_MRS6242_P2512_CPUF
#define CONFIG_BOARD_MRS6242_P2512_CPUF    0
#endif

#ifndef CONFIG_BOARD_MYBOARD
#define CONFIG_BOARD_MYBOARD              0
#endif

#if CONFIG_BOARD_MRS6130_P1806
#include "mrs6130_p1806/mrs6130_p1806_config.h"
#elif CONFIG_BOARD_MRS6130_P1812
#include "mrs6130_p1812/mrs6130_p1812_config.h"
#elif (CONFIG_BOARD_MRS6240_P2512_CPUS || CONFIG_BOARD_MRS6240_P2512_CPUF)
#include "mrs6240_p2512/mrs6240_p2512_config.h"
#elif ((CONFIG_BOARD_MRS6242_P2512_CPUS) || (CONFIG_BOARD_MRS6242_P2512_CPUF))
#include "mrs6242_p2512/mrs6242_p2512_config.h"
#elif (CONFIG_BOARD_MYBOARD == 1)
#include "myboard/myboard_config.h"
#else
#error "No board has been selected !!!"
#endif
```

Step C: Add the board to the project.

Step C-1: Open a basic project, such as hello_word, and add myboard to CDK.

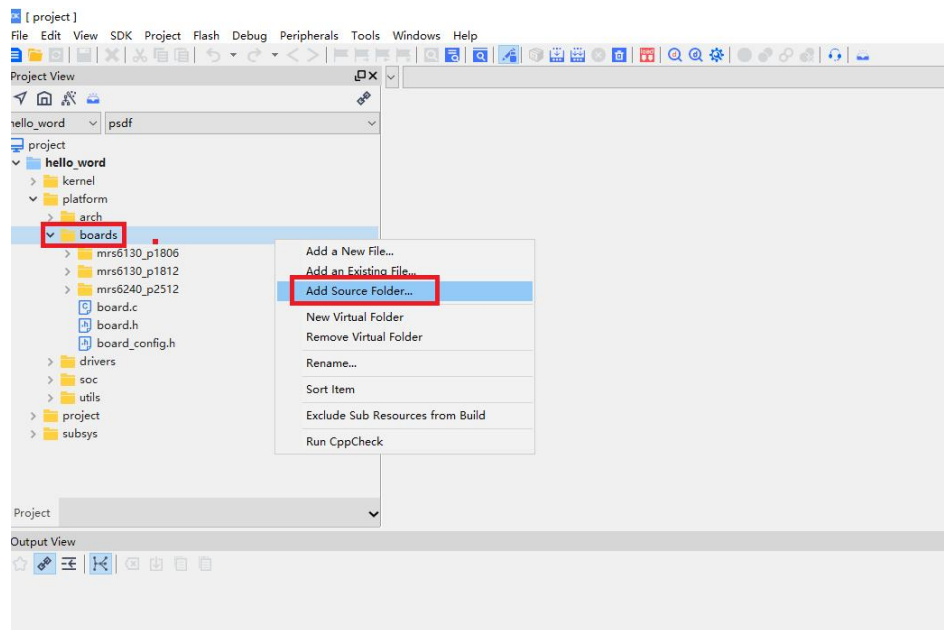


Figure 3-25 Add the New Board to CDK

After adding the board, build the project for verification as shown below.

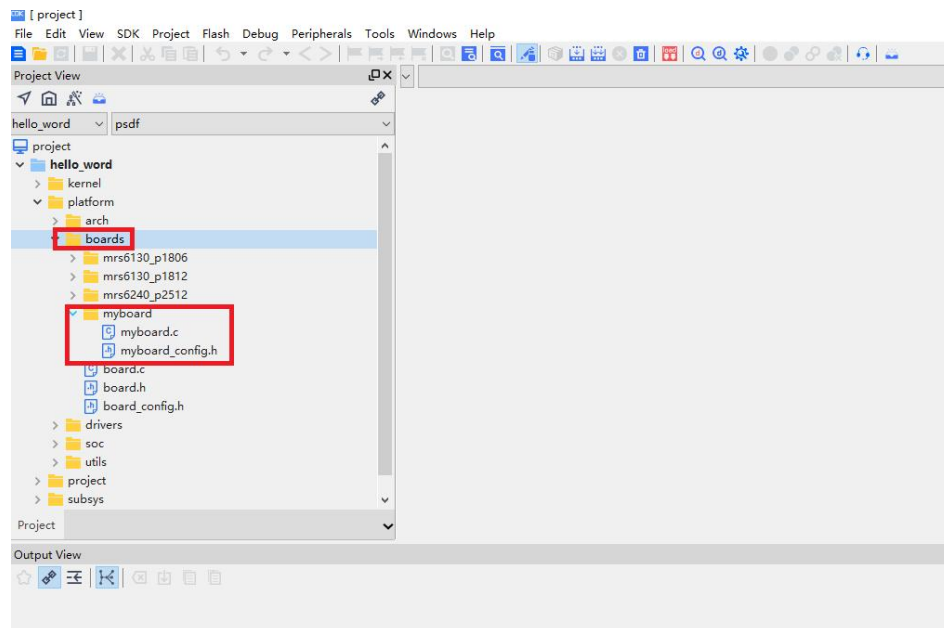


Figure 3-26 New Board Added to CDK

Step C-2: Follow Step C-1 to add the new board to CDK in the target project.

3.3.2 Create a Board Based on RS6240

Creating a board based on the RS6240 is generally the same as for the RS6130, but the CPUS and CPUF configurations must both be considered.

Note the following differences in [myboard_config.h](#).

```
#ifndef _MYBOARD_CONFIG_H
#define _MYBOARD_CONFIG_H

#if (CONFIG_BOARD_MYBOARD_CPUS || CONFIG_BOARD_MYBOARD_CPUF)
```

```
#define CONFIG_BOARD_STRING                "myboard"

#if CONFIG_BOARD_MYBOARD_CPUS
#define CONFIG_SOC_RS6240_CPUS            1
#endif

#if CONFIG_BOARD_MYBOARD_CPUF
#define CONFIG_SOC_RS6240_CPUF            1
#endif

#endif /* CONFIG_BOARD_MYBOARD_CPUS || CONFIG_BOARD_MYBOARD_CPUF */

#endif /* _MYBOARD_CONFIG_H */
```

Modify [Myboard.c](#) as follows.

```
#if (CONFIG_BOARD_MYBOARD_CPUS || CONFIG_BOARD_MYBOARD_CPUF)

#define LOG_MODULE                "myboard"
#define LOG_LEVEL                  CONFIG_BOARD_LOG_LEVEL
#include "log.h"

#endif /* CONFIG_BOARD_MYBOARD_CPUS || CONFIG_BOARD_MYBOARD_CPUF */
```

Modify [board_config.h](#) as follows.

```
#ifndef CONFIG_BOARD_MRS6242_P2512_CPUF
#define CONFIG_BOARD_MRS6242_P2512_CPUF            0
#endif

#ifndef CONFIG_BOARD_MYBOARD_CPUS
#define CONFIG_BOARD_MYBOARD_CPUS                  0
#endif

#ifndef CONFIG_BOARD_MYBOARD_CPUF
#define CONFIG_BOARD_MYBOARD_CPUF                  0
#endif

#if CONFIG_BOARD_MRS6130_P1806
#include "mrs6130_p1806/mrs6130_p1806_config.h"
#elif CONFIG_BOARD_MRS6130_P1812
#include "mrs6130_p1812/mrs6130_p1812_config.h"
#elif (CONFIG_BOARD_MRS6240_P2512_CPUS || CONFIG_BOARD_MRS6240_P2512_CPUF)
#include "mrs6240_p2512/mrs6240_p2512_config.h"
#elif ((CONFIG_BOARD_MRS6242_P2512_CPUS) || (CONFIG_BOARD_MRS6242_P2512_CPUF))
#include "mrs6242_p2512/mrs6242_p2512_config.h"
#elif ((CONFIG_BOARD_MYBOARD_CPUS) || (CONFIG_BOARD_MYBOARD_CPUF))
#include "myboard/myboard_config.h"
```

```
#else
#error "No board has been selected !!!"
#endif
```

3.3.3 Configuration Changes

The following sections describe how to modify commonly used settings.

3.3.3.1 Clock Configuration

[Myboard.c](#) allows selection of 32 MHz, 96 MHz, 112 MHz, or 128 MHz.

```
#if (CONFIG_BOARD_CLOCK_CFG == 32)
#include "rs6130_config_cpus_32mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 96)
#include "rs6130_config_cpus_96mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 112)
#include "rs6130_config_cpus_112mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 128)
#include "rs6130_config_cpus_128mhz.h"
#endif
```

The CONFIG_BOARD_CLOCK_CFG configuration macro is defined in [myboard_config.h](#).

```
/* 32: low
 * 96: medimu
 * 112: high
 * 128: highest
 */
#ifndef CONFIG_BOARD_CLOCK_CFG
#define CONFIG_BOARD_CLOCK_CFG 128
#endif
```

3.3.3.2 Pinmux Configuration

Defined in [myboard.c](#).

```
/* Private constants.
 *
 */
#ifdef CONFIG_DRIVER_UART_0
__attribute__((weak))const HAL_PinmuxConfig_t pinmux_uart0[] = {
    { GPIO_PORT_B, GPIO_PIN_0, {GPIOB_P0_F2_UART0_TX, GPIO_DRIVING_LEVEL_1, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* TX */
    { GPIO_PORT_B, GPIO_PIN_1, {GPIOB_P1_F2_UART0_RX, GPIO_DRIVING_LEVEL_1, GPIO_OPEN_DRAIN, GPIO_PULL_UP } }, /* RX */
};
#endif

#ifdef CONFIG_DRIVER_UART_1
__attribute__((weak))const HAL_PinmuxConfig_t pinmux_uart1[] = {
    { GPIO_PORT_A, GPIO_PIN_0, {GPIOA_P0_F3_UART1_TX, GPIO_DRIVING_LEVEL_1, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* TX */
    { GPIO_PORT_A, GPIO_PIN_1, {GPIOA_P1_F3_UART1_RX, GPIO_DRIVING_LEVEL_1, GPIO_OPEN_DRAIN, GPIO_PULL_UP } }, /* RX */
};
#endif

#ifdef CONFIG_DRIVER_UART_2
__attribute__((weak))const HAL_PinmuxConfig_t pinmux_uart2[] = {
    { GPIO_PORT_A, GPIO_PIN_2, {GPIOA_P2_F5_UART2_TX, GPIO_DRIVING_LEVEL_1, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* TX */
    { GPIO_PORT_A, GPIO_PIN_3, {GPIOA_P3_F5_UART2_RX, GPIO_DRIVING_LEVEL_1, GPIO_OPEN_DRAIN, GPIO_PULL_UP } }, /* RX */
};
#endif

#ifdef CONFIG_DRIVER_SPI
__attribute__((weak))const HAL_PinmuxConfig_t pinmux_spi0[] = {
    { GPIO_PORT_A, GPIO_PIN_0, {GPIOA_P0_F2_SPI_CS, GPIO_DRIVING_LEVEL_3, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* CS */
};
```

Figure 3-27 myboard Pinmux Configuration

The board_GetPinmux function defined in [myboard.c](#) is referenced by other drivers.

```

HAL_Status_t board_GetPinMux(
    HAL_DevType_t type,
    uint8_t idx,
    HAL_PinMuxType_t cfgType,
    HAL_Pinmux_t * pPinMux)
{
    HAL_Status_t status = HAL_STATUS_INVALID_STATE;

    switch (type) {
        case HAL_DEV_TYPE_UART:
            if (idx == UART0_ID) {
                #if CONFIG_DRIVER_UART_0
                if (cfgType == HAL_PINMUX_TYPE_INIT) {
                    pPinMux->pinConfig = (HAL_PinmuxConfig_t *)pinmux_uart0;
                    pPinMux->pinCnt = HAL_ARRAY_SIZE(pinmux_uart0);
                    status = HAL_STATUS_OK;
                }
                #endif
            } else if (idx == UART1_ID) {
                #if CONFIG_DRIVER_UART_1
                if (cfgType == HAL_PINMUX_TYPE_INIT) {
                    pPinMux->pinConfig = (HAL_PinmuxConfig_t *)pinmux_uart1;
                    pPinMux->pinCnt = HAL_ARRAY_SIZE(pinmux_uart1);
                    status = HAL_STATUS_OK;
                }
                #endif
            } else if (idx == UART2_ID) {
                #if CONFIG_DRIVER_UART_2
                if (cfgType == HAL_PINMUX_TYPE_INIT) {
                    pPinMux->pinConfig = (HAL_PinmuxConfig_t *)pinmux_uart2;
                    pPinMux->pinCnt = HAL_ARRAY_SIZE(pinmux_uart2);
                    status = HAL_STATUS_OK;
                }
                #endif
            }
    }
}

```

Figure 3-28 myboard Pinmux Configuration Function

3.3.3.3 FLASH/XIP Operating Frequency

Modify the SPI Flash operating frequency if required. For details, see Peripheral Drivers - Flash.

```

HAL_Status_t BOARD_InitXIP(void)
{
    HAL_Status_t status = HAL_STATUS_OK;
    HAL_Dev_t *pFlashDev = NULL;

    pFlashDev = HAL_FLASH_Init(FLASH_DEFT_ID);
    if (pFlashDev == NULL) {
        LOG_DBG("init fail");
        status = HAL_STATUS_ERROR;
    }

    uint32_t listMode = FLASH_LIST_TYPE_DEFAULT | FLASH_LIST_TYPE_LIST;
    status = HAL_FLASH_ExtControl(pFlashDev, FLASH_EXT_SET_CHIP_INFO_MODE, &listMode, sizeof(listMode));
    if (status != HAL_STATUS_OK) {
        LOG_DBG("set list mode fail");
        return status;
    }

    uint32_t freq = HAL_CLOCK_GetFreq(pBoardDev_Clock, CLOCK_SYS);
    FLASH_InitParam_t flashInitParam = {
        /* Configure the clock source and frequency in a template such as rs6130_config_cpus_96mhz.h */
        .clockSrc = CONFIG_XIP_CLK_SRC,
        .clockFreq = freq / CONFIG_XIP_CLK_DIV,
        .spiFormat = FLASH_SPI_FORMAT_QUAD,
        .psramEnable = false,
    };
}

```

```

status = HAL_FLASH_Open(pFlashDev, &flashInitParam);
if (status == HAL_STATUS_OK) {
    listMode = 0;
    status = HAL_FLASH_ExtControl(pFlashDev, FLASH_EXT_GET_USED_CHIP_INFO_MODE, &listMode, size
of(listMode));
    if (status != HAL_STATUS_OK) {
        LOG_DBG("get used list mode fail");
        return status;
    }

    if (listMode == FLASH_LIST_TYPE_DEFAULT) {
        status = HAL_STATUS_BUSY;
    }
}

return status;
}

```

Figure 3-29 myboard SPI Flash Configuration

3.3.3.4 Other Configuration

Other Common Configurations

```

/* Total system stack configuration */
#ifndef CONFIG_HEAP_SIZE
#define CONFIG_HEAP_SIZE                (1024 * 40)
#endif

/* Whether to enable printing */
#ifndef CONFIG_PRINTF
#define CONFIG_PRINTF                    1
#endif

/* If printing is enabled, configure the UART number and baud rate. */
#if CONFIG_PRINTF
#ifndef CONFIG_PRINTF_UART_NUM
#define CONFIG_PRINTF_UART_NUM          0
#endif

#ifndef CONFIG_PRINTF_UART_BAUDRATE
#define CONFIG_PRINTF_UART_BAUDRATE      115200
#endif

/* Whether to enable the shell */
#ifndef CONFIG_SHELL
#define CONFIG_SHELL                    0
#endif

/* If the shell is enabled, configure the UART number and baud rate. */
#if CONFIG_SHELL
#ifndef CONFIG_SHELL_UART_NUM
#define CONFIG_SHELL_UART_NUM           0
#endif
#endif

```

```
#ifndef CONFIG_SHELL_UART_BAUDRATE
#define CONFIG_SHELL_UART_BAUDRATE          115200
#endif
```

4 Subsystems

4.1 CPU

The RS6130 supports the following CPU specifications.

Table 4-1 RS6130 CPU Specifications

	run freq max	cache size	Instruction Set
CPUS	Max: 128MHz Default: 96MHz	i_cache : 4KB d_cache: 2KB	IMACF Integer, multiply/divide, atomic-operation, compressed, and single-precision floating-point instructions

The RS6240 supports the following CPUF and CPUS dual-core specifications.

Table 4-2 RS6240 CPU Specifications

	run freq max	cache size	Instruction Set
CPUS	Max:128MHz Default: 112MHz	i_cache : 4KB d_cache: 2KB	IMACF Integer, multiply/divide, atomic-operation, compressed, and single-precision floating-point instructions
CPUF	Max:256MHz Default: 224MHz	i_cache : 8KB d_cache: 16KB	IMACFP Integer, multiply/divide, atomic-operation, compressed, single-precision floating-point, and DSP instructions

Usage Notes:

1. When using a different CPU, configure CDK to select the correct instruction set.
In CDK, go to Project Setting > Device. Select E906F for CPUS or E906FP for CPUF.

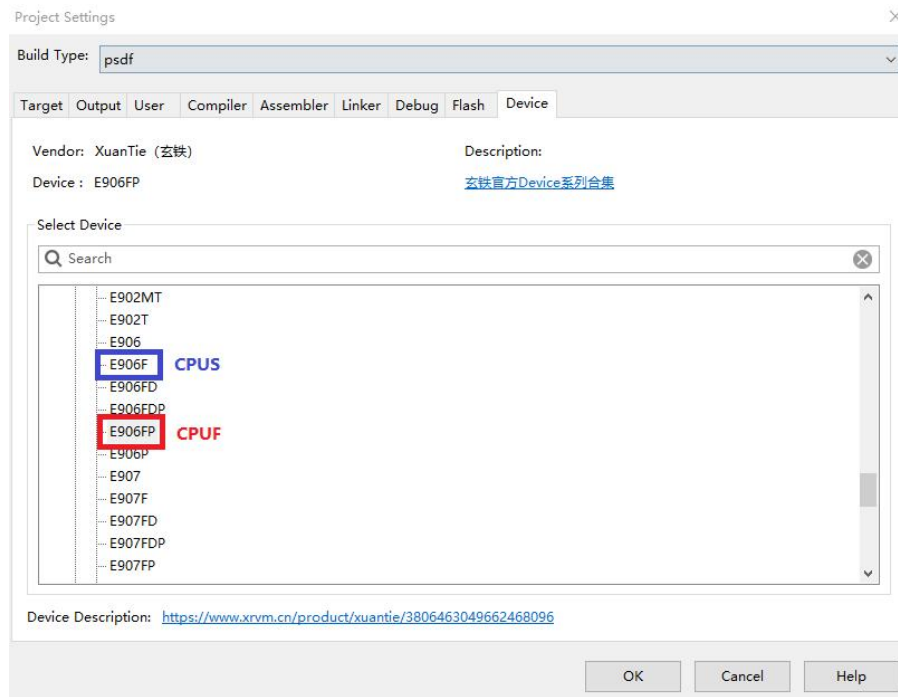


Figure 4-1 Select the CPU Instruction Set in CDK

- In CDK, select the corresponding linker script. For details, see the linker-script description in Memory Map. The main difference between the CPUS and CPUF linker scripts is the XIP address: CPUS uses 0x0900 0000 and CPUF uses 0x0940 0000.

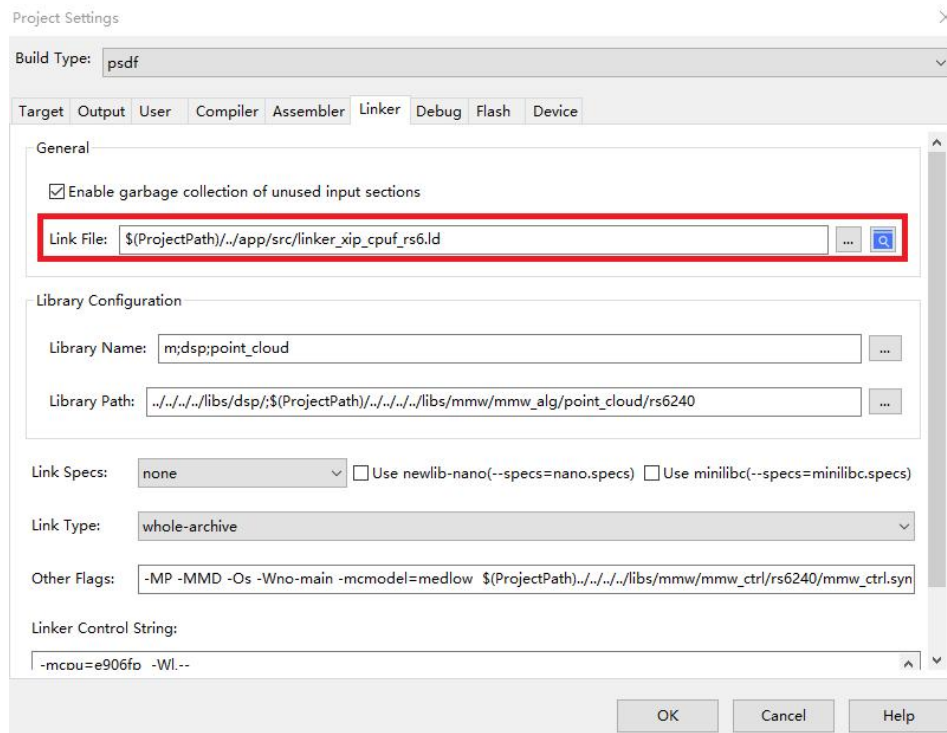


Figure 4-2 Configure the Linker Script in CDK

- When using radar, select the correct radar-firmware symbol table in CDK.

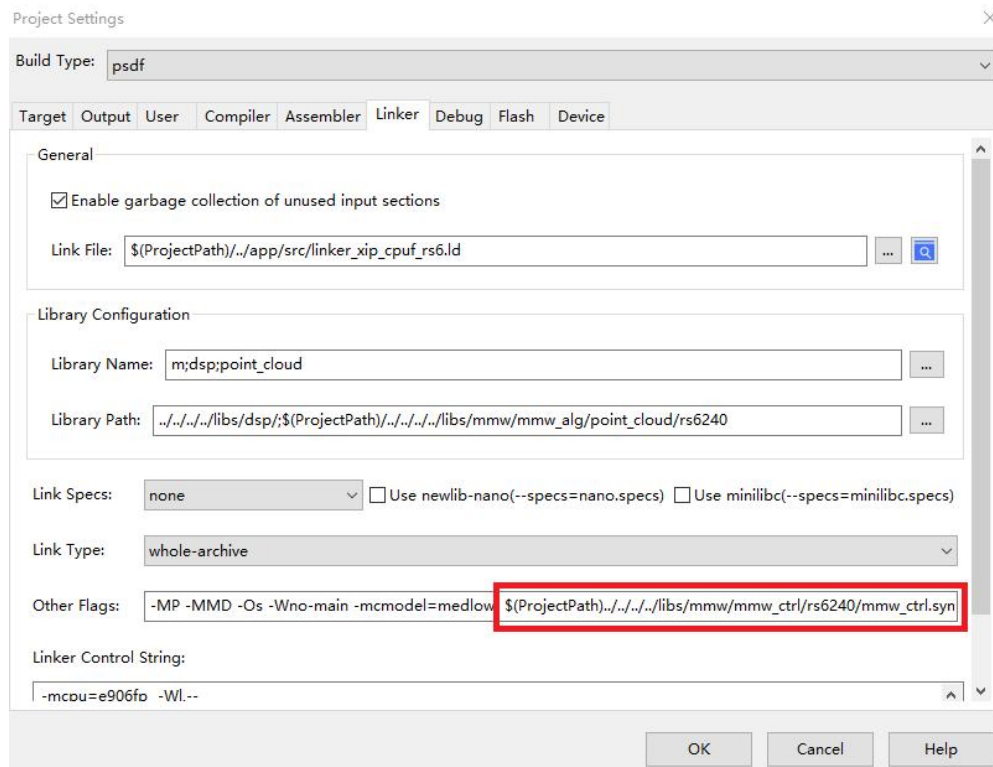


Figure 4-3 Configure the Radar Symbol Table in CDK

4.1.1 Cache

The CPU cache improves execution efficiency, especially for code and data running in XIP. Cache configuration is located in [platform/soc/rs613x/src/rs613x.c](#) and [platform/soc/rs624x/src/rs624x.c](#). As shown below, cacheable means that reads and writes to the address range pass through the cache. For example, `csi_sysmap_config_region(4, 0x18000000, SYMAP_SYMAPCFG_B_Msk | SYMAP_SYMAPCFG_C_Msk);` means that accesses from the preceding 0x1040 0000 address through 0x1800 0000 pass through the cache. This range contains data and therefore uses the D-cache.

```
/* 0> 0x0000 0000 - 0x0050 0000 : rom/ram-code,      memory, non-cacheable, bufferable */
csi_sysmap_config_region(0, 0x00500000, SYMAP_SYMAPCFG_B_Msk);
/* 1> 0x0050 0000 - 0x0800 0000 : remap sram-code,   memory, cacheable,      bufferable */
csi_sysmap_config_region(1, 0x08000000, SYMAP_SYMAPCFG_B_Msk | SYMAP_SYMAPCFG_C_Msk);
/* 2> 0x0800 0000 - 0x1000 0000 : xip-code,          memory, cacheable,      bufferable */
csi_sysmap_config_region(2, 0x10000000, SYMAP_SYMAPCFG_B_Msk | SYMAP_SYMAPCFG_C_Msk);
/* 3> 0x1000 0000 - 0x1040 0000 : ram-data,          memory, non-cacheable, bufferable */
csi_sysmap_config_region(3, 0x10400000, SYMAP_SYMAPCFG_B_Msk);
/* 4> 0x1040 0000 - 0x1800 0000 : sram-data,         memory, non-cacheable, bufferable */
csi_sysmap_config_region(4, 0x18000000, SYMAP_SYMAPCFG_B_Msk | SYMAP_SYMAPCFG_C_Msk);
/* 5> 0x1800 0000 - 0x2000 0000 : xip-data,          memory, cacheable,      bufferable */
csi_sysmap_config_region(5, 0x20000000, SYMAP_SYMAPCFG_B_Msk | SYMAP_SYMAPCFG_C_Msk);
/* 6> 0x2000 0000 - 0x4000 0000 : sbus-sram,         memory, non-cacheable, bufferable */
csi_sysmap_config_region(6, 0x40000000, SYMAP_SYMAPCFG_B_Msk);
/* 7> 0x4000 0000 - 0xFFFF FFFF : periph,           device, non-cacheable, non-bufferable */
csi_sysmap_config_region(7, 0xFFFFFFF, SYMAP_SYMAPCFG_SO_Msk);
```

The `csi_icache_invalid` API invalidates the entire instruction cache. Subsequent instructions must be reloaded into the cache.

The `csi_dcache_clean_invalid_range` API cleans and invalidates the data cache in the specified address range. Data in this range is written back and reloaded into the cache.

4.1.2 Floating Point

The CPUs in the RS6130 and RS6240 support hardware floating-point instructions, which are enabled by default. Floating-point operations improve calculation accuracy but increase operating power consumption and code size.

Note: The hardware supports only single-precision floating point. Using double-precision floating point invokes the software floating-point library and increases code size.

4.1.3 Dual Core

The RS6240 supports dual-core operation. Radar firmware and algorithm functions should run on CP UF, while application logic and communication can run on CPUS.

If CONFIG_BOARD_MYBOARD6240_CPUF is used in [Prj config.h](#), the code is built to run on COUF. Also select the correct linker file and CPU instruction set; see the CPU usage notes in Section 4.1. If

CONFIG_BOARD_MYBOARD6240_CPUS is defined, the code is built to run on CPUS. For dual-core packaging, enable CONFIG_SOC_DUAL_CORE when building the last core.

The current version does not support simultaneous dual-core operation.

4.1.4 Code Optimization Guide

4.1.4.1 SRAM Usage Analysis

The Radar Analysis project is used as an example to show how to analyze static RAM (SRAM) code usage.

The overall SRAM usage can be obtained from the build output, as shown below. Total SRAM is 294 KB (the data sheet specifies 295 KB; the remaining 1 KB is used by the KVR file system, as described in the KVR & KVF section). The current image uses 188.88 KB, or 64.24%.

Note: The following data is for example analysis only and does not represent the final SRAM usage of an actual project.

```
output image : mrs6240\_p2512.img
version : R0.0-M0.0.0-V2.0.0
image size (flash): 252876 (246.95 KB)
sram size: 188.88k (64.24%). total size: 294k
```

Open [output/img_cfg.yml](#) in the current project directory and locate [merge-img/remap-address](#).

```
merge-img:
  remap-address:
    - offset: 0x400400
      run-address: 0x500000 /* 0x500000 is the instruction run address of the app, including the radar-algorithm static library. */
      size: 0x12000 /* SRAM used by app code */
      type: sram-ibus-cpuf
    - offset: 0x419780
      run-address: 0x10500000 /* 0x10500000 is the app data run address. */
      size: 0x16000 /* SRAM used by app data */
      type: sram-dbus-cpuf
    - offset: 0x1000
      run-address: 0x9400000
      size: 0x20000 /* Flash used by app instructions and data */
      type: xip
    - offset: 0x412400
      run-address: 0x580000 /* 0x580000 is the radar-firmware instruction run address. */
      size: 0x6a80 /* SRAM used by radar-firmware instructions */
      type: sram-ibus-cpusf
    - offset: 0x418e80
```

```

run-address: 0x10580000 /* 0x10580000 is the radar-firmware data run address. */
size: 0x900             /* SRAM used by radar-firmware data */
type: sram-dbus-cpusf
- offset: 0x21000
  run-address: 0x9c00000
  size: 0x19000         /* Flash used by radar-firmware instructions and data */
  type: xip

```

Open [Lst/psdf.map](#) in the current project directory and [app/src/linker_xip_cpuf_rs6.ld](#) (open the file corresponding to the CDK configuration). To determine the SRAM occupied by algorithm code, for example, add markers before and after the relevant section and subtract their addresses in the generated map file.

[app/src/linker_xip_cpuf_rs6.ld](#) File

```

isram_text : {
    . = ALIGN(0x4) ;
    __s_isram_text = . ;

.....

    /* libc and libgcc */
    __s_libs_text = . ;
    *libc.a:(*.text*)
    *libgcc.a:(*.text*)
    *libdsp.a:(*.text*)
    *libm.a:(*.text*)
    *libc_newlib_stub*.o(*.text*)
    __e_libs_text = . ;

    /* mmw */
    __s_mmw_algo_text = . ;           /* Radar-algorithm start marker */
    *mmw_process*.o(*.text*)
    *mmw_psic_lib*.o(*.text*)
    *mmw_algorithm*.o(*.text*)
    *src_mmw*.o(*.text*)
    *mmw_mdsp*.o(*.text*)
    *mmw_mimo*.o(*.text*)
    *common_lib*.o(*.text*)
    __e_mmw_algo_text = . ;          /* Radar-algorithm end marker */

.....

    . = ALIGN(0x4) ;
    __e_isram_text = . ;
} > ISRAM

```

In [PsdF.map](#), search separately for `__e_mmw_algo_text` and `__s_mmw_algo_text` to obtain their addresses. Calculate the size as $0x0050d4e2 - 0x00508a14 = 0x4ACE = 18.7 \text{ KB}$.

[Lst/psdf.map](#) File

```

0x00508a14          __s_mmw_algo_text = .

0x0050d4e2          __e_mmw_algo_text = .

```

For a detailed function size, search for the function in [psdf.map](#). For example, `fast_db_trans_f32` has a size of `0x3E`.

[Lst/psdf.map](#) File

```
*mmw_process*.o(*.text*)
.text.fast_db_trans_f32
      0x00508a14      0x3e Obj/mmw\_process mmw\_point\_cloud\_process.o
      0x00508a14      fast_db_trans_f32
```

4.1.4.2 Heap Usage Analysis

The Radar Analysis project is used as an example to show how to analyze heap usage.

In [prj_config.h](#), modify `CONFIG_HEAP_SIZE` to configure the heap size (the default maximum is 130 KB).

Enter the shell command "heap" in the UART console to obtain the overall heap usage, as shown below.

```
psic:/$ heap
start addr      : 10502000
total size      : 130 KB (133120 B)
current free size : 115 KB (118136 B)
largest free size : 115 KB (118136 B)
smallest free size : 115 KB (118136 B)
number free blocks : 1
min free size in history : 114 KB (117688 B)
number call alloc : 29
number call free  : 1
Command Success
```

Figure 4-4 HEAP Usage

Table 4-3 Heap Shell Return Values

Heap shell	Description
start addr	Heap start address: 10501E30
total size	Heap size: 130 KB
current free size	Current remaining heap size: 115 KB
largest free size	Current largest free heap block: 115 KB
smallest free size	Current smallest free heap block: 115 KB
number free blocks	Current number of free heap blocks: 1
min free size in history	Historical minimum free heap: 114 KB
number call alloc	Number of memory allocations after program startup: 29
number call free	Number of memory releases after program startup: 1

4.2 DSP

The SoC provides a software DSP static library. To use it, configure CDK as shown below: add `dsp` to Library Name and add the [libs/dsp](#) directory to Library Path.

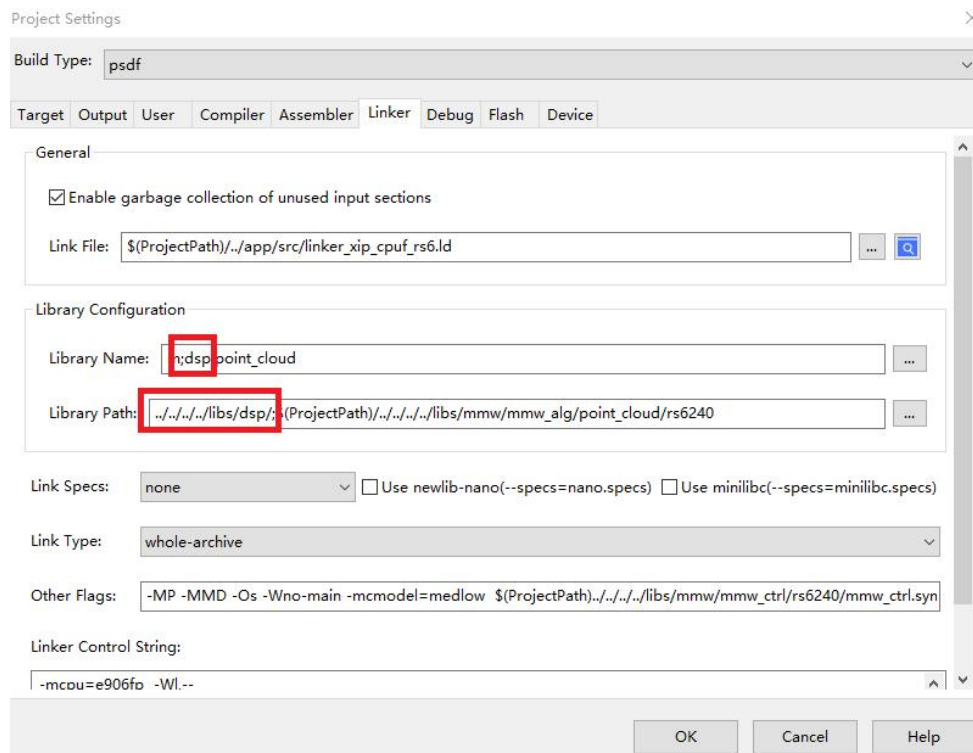


Figure 4-5 Configure the DSP Library in CDK

Header-file path: [subsys/dsp/inc/csi_math.h](#)

For API descriptions, see '[rs6xxx DSP API Reference Manual.pdf](#)'.

4.3 Flash and PSRAM

The SoC supports external SPI Flash and SPI PSRAM. Use HAL_FLASH_Open to open SPI Flash or SPI PSRAM. For details, see Flash under Peripheral Drivers.

Flash can contain executable code and read-only data.

Flash can be read directly through its XIP address or through the Flash Read API. To write Flash, first erase it through the API and then write it.

The addresses used for XIP reads are as follows:

Table 4-4 Flash XIP Address Map

	Address	Description
SYS 0	0x0900 0000	XIP address of the application running on CPUS
SYS 1	0x0940 0000	XIP address of the application running on CPUF
MMW CTRL	0x09C0 0000	XIP run address of the radar BB firmware, which can run on CPUS or CPUF
MMW ALG	0x0980 0000	XIP run address of the radar algorithm, which can run on CPUS or CPUF

PSRAM can store the large amount of intermediate data and logs generated by radar algorithms. It can theoretically execute XIP code, but doing so is not recommended. PSRAM can contain executable code and read/write data and can be accessed through XIP addresses or APIs. It is not supported in the current version.

Table 4-5 PSRAM XIP Address Map

Address	Description
---------	-------------

	Address	Description
SYS 0 Code	0x0D08 0000	PSRAM code address range of the application running on CPUS
SYS 0 Data	0x1D08 0000	PSRAM data address range of the application running on CPUS
SYS 1 Code	0x1D10 0000	PSRAM code address range of the application running on CPUF
SYS 1 Data	0x1D10 0000	PSRAM data address range of the application running on CPUF

4.3.1 Code in XIP or SRAM

When code runs from XIP by default, use either of the following methods to move code to SRAM:

Method 1: Modify the corresponding [linker cpu.ld](#) file.

For example, to place [add_new_file.c](#) in SRAM, add it to the isram_text section as shown below. All functions defined in [add_new_file.c](#) are then linked into isram_text.

```
isram_text : {
    . = ALIGN(0x4) ;
    __s_isram_text = . ;

    /* The asterisks before and after add_new_file are filename wildcards. The leading asterisk is recommended; the trailing asterisk is optional. */
    *add_new_file*.o(*.text*)

    . = ALIGN(0x4) ;
    __e_isram_text = . ;
} > ISRAM
```

Method 2: Add an attribute macro before the function.

For example, add `__sram_text` or `__keep_sram_text` before a function to link it into SRAM.

Both macros are defined in [platform/utls/compiler.h](#), which must be included when they are used. Before using `__sram_text`, confirm that `CONFIG_SECTION_ATTRIBUTE_SRAM` is enabled. A function marked with `__keep_sram_text` is linked into SRAM even if it is unreferenced, whereas a function marked with `__sram_text` is linked into SRAM only when it is referenced. An unreferenced function is not called by other functions, especially by the main flow, and may be removed by the compiler to reduce code size.

Macro definitions in [platform/utls/compiler.h](#)

```
#define __keep_sram_text    __attribute__((section (".keep_sram_text")))
#define __keep_sram_rodata __attribute__((section (".keep_sram_rodata")))
#define __keep_sram_data   __attribute__((section (".keep_sram_data")))
#define __keep_sram_bss    __attribute__((section (".keep_sram_bss")))

#if (CONFIG_SECTION_ATTRIBUTE_SRAM == 1)
#define __sram_text        __attribute__((section (".sram_text")))
#define __sram_rodata      __attribute__((section (".sram_rodata")))
#define __sram_data        __attribute__((section (".sram_data")))
#define __sram_bss         __attribute__((section (".sram_bss")))
#else
#define __sram_text
#define __sram_rodata
#define __sram_data
#define __sram_bss
#endif
```

Usage Example

```

__keep_sram_text int32_t HAL_CLOCK_GetFreq(HAL_Dev_t *clock, CLOCK_Type_t clkType)
{
    if ((clock == NULL) || (clkType >= CLOCK_MAX)) {
        return (int32_t)0;
    }

    Clock_Device_t *pClockDevice = (Clock_Device_t *)clock;

    return pClockDevice->freq[clkType];
}

```

The linker file links the .sram_text and .keep_sram_text sections

```

isram_text : {
    . = ALIGN(0x4) ;
    __s_isram_text = . ;

    KEEP(*(.keep_sram_text));
    *(.sram_text);

    . = ALIGN(0x4) ;
    __e_isram_text = . ;
} > ISRAM

```

4.4 Memory Map

The main software-development flow is as follows:

1. Use CDK to build the bin files, and use the packaging script to combine the related bin files into one img.
2. Use BurnTool to communicate with the read-only memory (ROM) program in the SoC and burn the img to external SPI Flash.
3. Reset the SoC. The CPU first runs the ROM program, which loads and runs the bootloader from SPI Flash. The bootloader then loads and runs the application.

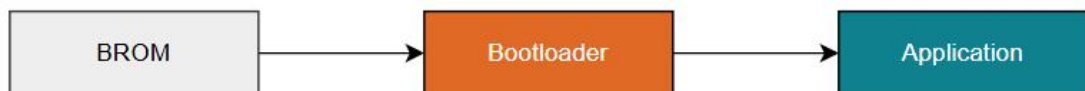


Figure 4-6 Startup Process

4.4.1 Img Format

The img produced by the build and programmed to SPI Flash, such as [mrs6240_p2512.img](#), is organized as shown below. It consists primarily of [bootloader.img](#), [merge.img](#), [flash_list.img](#), and [custom_cfg.img](#).

1. [bootloader.img](#): Second-stage boot program responsible for img verification, program loading, over-the-air (OTA) updates, and related functions. It is provided by the SDK as an img file.

2. [merge.img](#): Main application image containing sub-images such as [sys.img](#), [mmw_ctrl.img](#), and [mmw_alg.img](#).
- (1) [sys0.img/sys1.img](#): Application image. For example, [sys1.img](#) generated by the radar analysis sample contains the OS, peripheral drivers, subsystems, and radar application. The SDK provides the source code and generates [sys0.img/sys1.img](#) during the build. sys0 corresponds to CPUS and sys1 corresponds to CPUF; check the support available in each SoC series.
- (2) [mmw_ctrl.img](#): Radar BB driver image provided by the SDK.
- (3) [mmw_alg.img](#): Radar algorithm image. Some closed-source algorithms are provided by the SDK as img files.
2. [Flash_list.img](#): Stores parameters for the SPI Flash support list.
3. [custom_cfg.img](#): Optional image used to store application configuration.

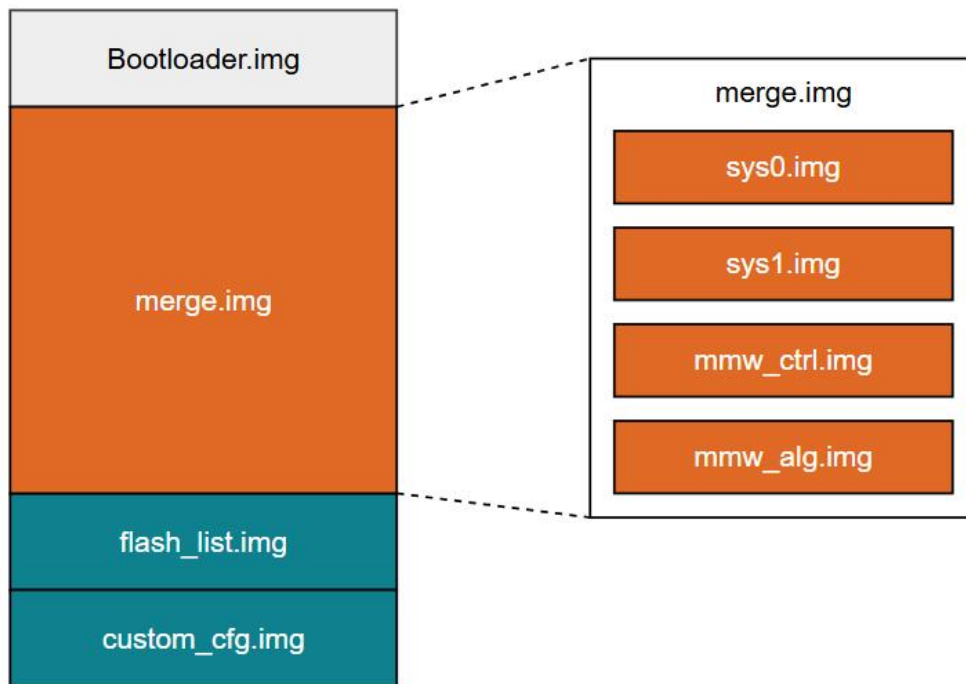


Figure 4-7 Img Organization

Each img and sub-image has the format shown below. img-head contains the version, img size, and related information; img-body contains the hash; and img-body contains the executable code or configuration body.

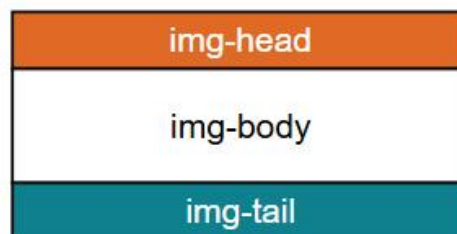


Figure 4-8 Img Format

4.4.2 Map

Each img uses the memory layout shown below, including XIP code and RO data and SRAM code, RO data, data, stack, and other sections. The img is programmed to SPI Flash in this layout. The bootloader copies isram_text, isram_rodata, and dsram_data to the corresponding SRAM locations and configures the virtual run addresses for ixip, isram, and dsram.

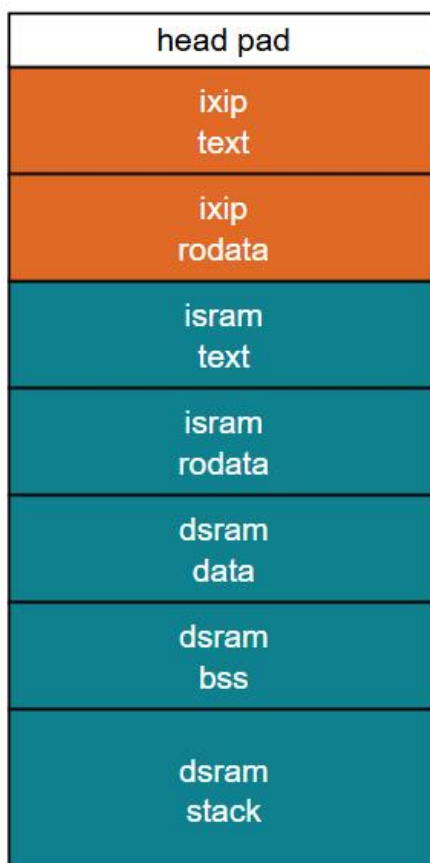


Figure 4-9 Img Memory Map

The memory remap addresses of the images are listed below.

Table 4-6 Address Remap

	SYS0	SYS1	MMW_CTRL	MMW_ALG
iXIP	0x0900 0000	0x0940 0000	0x09C0 0000	0x0980 0000
iSRAM	0x0050 0000	0x0050 0000	0x0058 0000	0x0054 0000
dSRAM	0x1050 0000	0x1050 0000	0x1058 0000	0x1054 0000

The bootloader maps run addresses to different physical addresses. For example, XIP addresses map to different offsets in SPI Flash, and SRAM addresses map to different physical SRAM locations.

The following linker scripts are provided under [platform/utlis](#) in the SDK source. Their sections correspond to the descriptions above.

1. [linker_cpus.ld](#): CPUS linker script. Code runs from XIP by default; time-critical code and code needed before XIP resumes from sleep are placed in SRAM.
2. [linker_cpuf.ld](#): CPUF linker script. Code runs from XIP by default; time-critical code and code needed before XIP resumes from sleep are placed in SRAM.
3. [linker_sram_cpus.ld](#): CPUS linker script with all code running from SRAM.
4. [linker_sram_cpuf.ld](#): CPUF linker script with all code running from SRAM.

4.5 Startup and Configuration

4.5.1 Initialization

As described above, after loading [merge.img](#), the bootloader jumps to [sys0.img](#) or [sys1.img](#) and transfers CPU control to the application.

The application startup flow is as follows (see Figure 4-5 for details):

1. `reset_handler`: Initializes the interrupt stack, interrupt vector table, and CPU attributes. It is implemented in [platform/arch/riscv/startup.s](#).
2. `soc_init`: Clears BSS, configures memory features, and enables cache. Data-section initialization is performed by the bootloader. The common entry is in [platform/soc/common/src/soc.c](#), and SoC-specific implementation is provided in each SoC file, such as [platform/soc/rs6130/rs6130.c](#).
3. `board_init`: Initializes solution-specific configuration, including OS startup and system-driver initialization. The common entry is in [platform/boards/board.c](#), and board-specific implementation is provided in each board file, such as [platform/boards/mrs6240_p2512/mrs6240_p2512.c](#).
4. `main`: Application entry point implemented in [main.c](#) of each project.

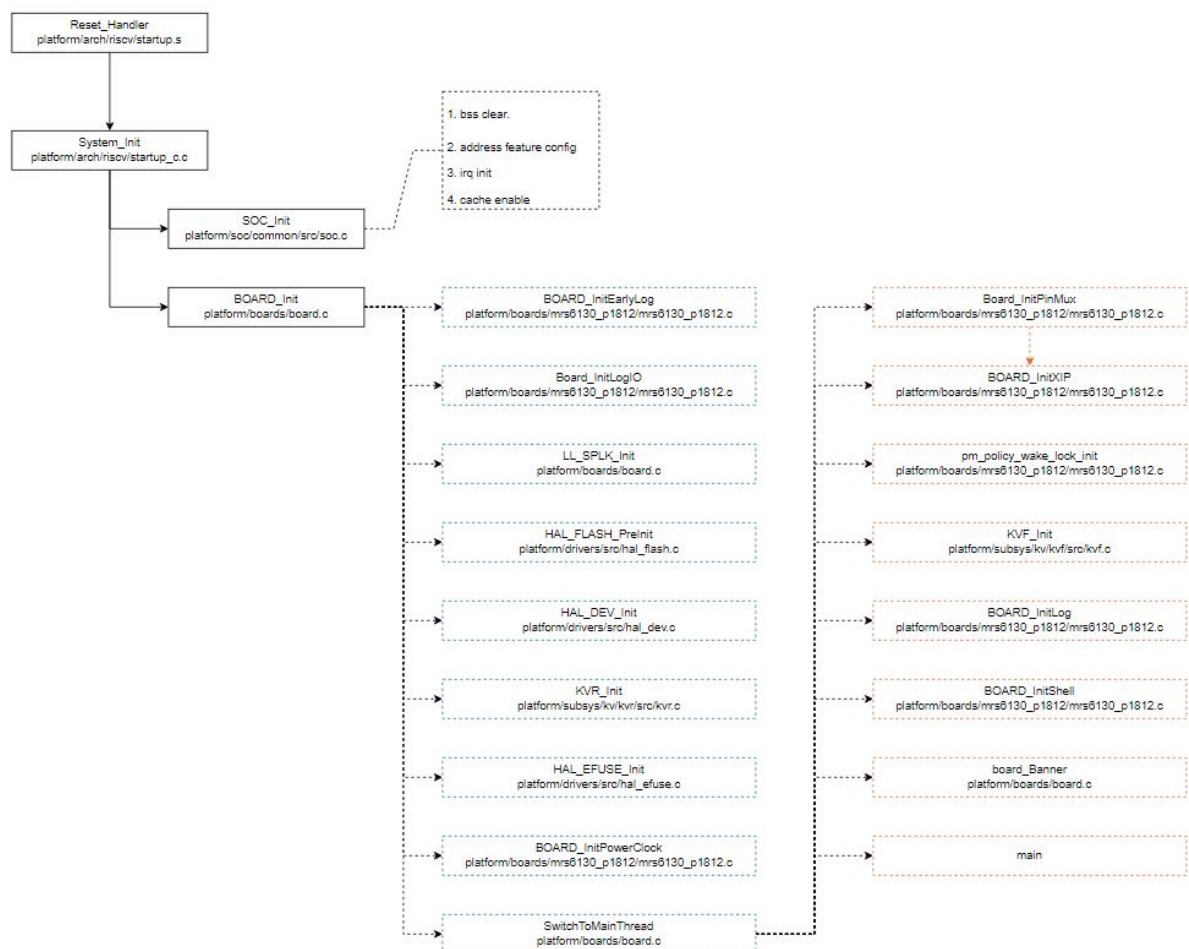


Figure 4-10 Startup Flow

4.5.2 Configuration

System functions and parameters can be configured through macros. The configuration relationship is shown below. [prj_config.h](#) is the top-level project configuration and can override default settings in [board_config.h](#) and [soc_config.h](#).

Include [config.h](#) in driver and application code. Do not include only [prj_config.h](#) or similar headers, because that may omit required configuration.

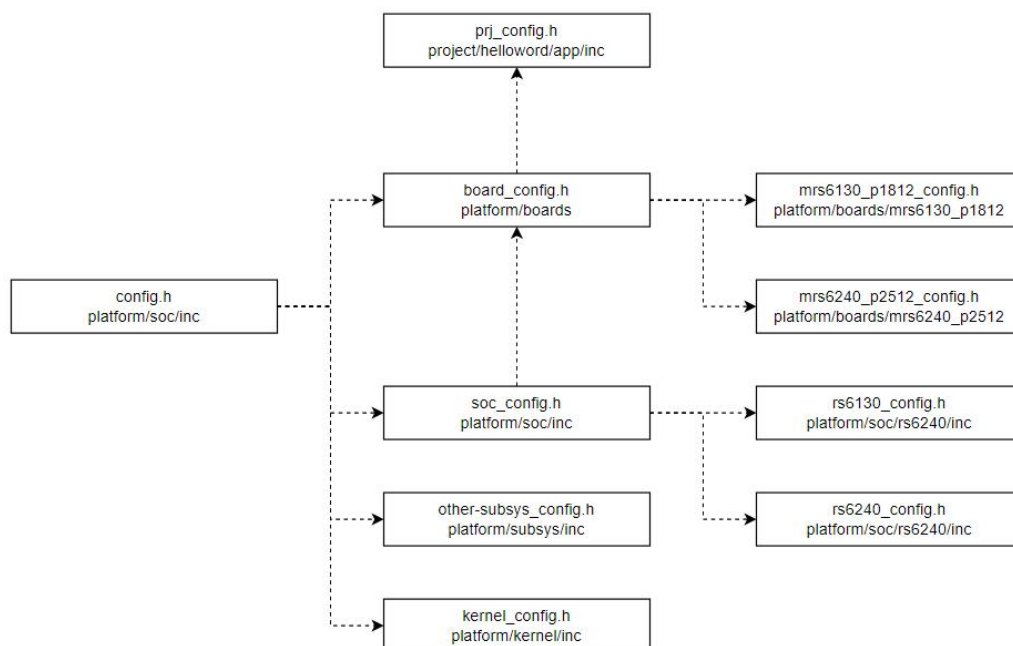


Figure 4-11 Configuration Relationships

4.6 Clock

Clock configuration is initialized in each board file. For example, the mrs6240_p2512 board implements it in BOARD_InitPowerClock in [platform/boards/mrs6240_p2512/mrs6240_p2512.c](#).

The exact configuration depends on the SoC power and clock architecture. For ease of use, three preset configurations are provided and selected by CONFIG_BOARD_CLOCK_CFG. Do not adjust individual parameters unless you understand the power and clock details of the current SoC.

```
#if (CONFIG_BOARD_CLOCK_CFG == 32)
#include "rs624x_config_cpuf_32mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 96)
#include "rs624x_config_cpuf_96mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 112)
#include "rs624x_config_cpuf_112mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 160)
#include "rs624x_config_cpuf_160mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 192)
#include "rs624x_config_cpuf_192mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 224)
#include "rs624x_config_cpuf_224mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 240)
#include "rs624x_config_cpuf_240mhz.h"
#elif (CONFIG_BOARD_CLOCK_CFG == 256)
#include "rs624x_config_cpuf_256mhz.h"
#endif
```

The clock frequencies corresponding to these configurations are listed below.

Table 4-7 RS6130 Clock Configuration

CPUF MHz	Msi MHz	PLL MHz	SYS_CLK MHz	AHB_CLK MHz	APB_CLK MHz	XIP MHz	SPI MHz
32	32	OFF	32	32	32	16	8

CPUF MHz	Msi MHz	PLL MHz	SYS_CLK MHz	AHB_CLK MHz	APB_CLK MHz	XIP MHz	SPI MHz
96	32	192	96	96	48	48	24
112	32	224	112	112	56	56	28
128	32	256	128	128	64	64	32

Table 4-8 RS6240 Clock Configuration

CPUF MHz	Msi MHz	PLL MHz	SYS_CLK MHz	AHB_CLK MHz	APB_CLK MHz	XIP MHz	SPI MHz
32	32	OFF	32	32	32	16	8
96	32	192	96	96	48	48	24
112	32	224	112	112	56	56	28
160	32	320	160	80	40	80	40
192	32	384	192	96	48	96	48
224	32	224	224	112	56	56	56
240	32	240	240	120	60	60	60
256	32	256	256	128	64	64	64

4.7 Pinmux

Pin-multiplexing configurations are defined in the board initialization implementation, for example in [platform/boards/mrs6240_p2512/mrs6240_p2512.c](#). Use DEFINE_PINMUX_WEAK in the specified format.

```
#if CONFIG_DRIVER_UART
DEFINE_PINMUX_WEAK(uart0, 2, // Name: uart0; the pinmux list contains two pins, which are initialized below.
{ GPIO_PORT_B, GPIO_PIN_0, {GPIOB_P0_F2_UART0_TX, GPIO_DRIVING_LEVEL_1, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* TX */
// GPIO port number, pin number, pin-function name, output drive capability, output type, and pull-up/pull-down resistors
{ GPIO_PORT_B, GPIO_PIN_1, {GPIOB_P1_F2_UART0_RX, GPIO_DRIVING_LEVEL_1, GPIO_OPEN_DRAIN, GPIO_PULL_UP } }, /* RX */
);

DEFINE_PINMUX_WEAK(uart0_deinit, 2,
{ GPIO_PORT_B, GPIO_PIN_0, {GPIOx_Pn_F15_DIS, GPIO_DRIVING_LEVEL_1, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* TX */
{ GPIO_PORT_B, GPIO_PIN_1, {GPIOx_Pn_F15_DIS, GPIO_DRIVING_LEVEL_1, GPIO_OPEN_DRAIN, GPIO_PULL_UP } }, /* RX */
);
#endif
```

When adding pinmux settings for another purpose, modify board_GetPinMux.

```

HAL_Status_t board_GetPinMux(
    HAL_DevType_t type,
    uint8_t idx,
    HAL_PinMuxType_t cfgType,
    HAL_Pinmux_t * pPinMux)
{
    HAL_Status_t status = HAL_STATUS_INVALID_STATE;

    switch (type) {
    case HAL_DEV_TYPE_UART:    // Driver type
        if (idx == UART0_ID) { // Driver ID
            if (cfgType == HAL_PINMUX_TYPE_INIT) { // Similar configuration for initialization
                pPinMux->pinConfig = (HAL_PinmuxConfig_t *)pinmux_uart0.pinConfig;
                pPinMux->pinCnt = pinmux_uart0.pinCnt;
                status = HAL_STATUS_OK;
            } else if (cfgType == HAL_PINMUX_TYPE_DEINIT) { // Similar configuration for deinitialization
                pPinMux->pinConfig = (HAL_PinmuxConfig_t *)pinmux_uart0_deinit.pinConfig;
                pPinMux->pinCnt = pinmux_uart0_deinit.pinCnt;
                status = HAL_STATUS_OK;
            }
        } else {
            break;
        }
        break;

    default:
        break;
    }

    return status;
}

```

4.8 LOG

The Log subsystem consists of printing and I/O functions.

To enable the Log subsystem, add the source files and header paths under [subsyst/log](#) to the build, and then enable the corresponding macros defined in [subsyst/log/inc/log_config.h](#).

```

/* Whether to enable printing: 1 = enabled; 0 = disabled */
#ifndef CONFIG_PRINTF
#define CONFIG_PRINTF 0
#endif
/* Whether to print through UART; the default is 1 */
#ifndef CONFIG_PRINTF_UART
#define CONFIG_PRINTF_UART 1
#endif
/* Printing UART number: 0 = UART0; 1 = UART1 */
#ifndef CONFIG_PRINTF_UART_NUM
#define CONFIG_PRINTF_UART_NUM 0
#endif

```

```
/* UART printing baud rate */
#ifndef CONFIG_PRINTF_UART_BAUDRATE
#define CONFIG_PRINTF_UART_BAUDRATE          115200
#endif
```

In each board configuration file, such as [mrs6130_p1812_config.h](#), CONFIG_PRINTF_UART_NUM enables the corresponding UART.

```
/** Printf **/
#if CONFIG_PRINTF
#ifndef CONFIG_PRINTF_UART_NUM
#define CONFIG_PRINTF_UART_NUM          0
#endif

#ifndef CONFIG_PRINTF_UART_BAUDRATE
#define CONFIG_PRINTF_UART_BAUDRATE      115200
#endif

/* depend on UART_DRIVER */
#if ((CONFIG_PRINTF >= 1) && (CONFIG_PRINTF_UART_NUM == 0))
#undef CONFIG_DRIVER_UART_0
#define CONFIG_DRIVER_UART_0            1

#elif ((CONFIG_PRINTF >= 1) && (CONFIG_PRINTF_UART_NUM == 1))
#undef CONFIG_DRIVER_UART_1
#define CONFIG_DRIVER_UART_1            1

#elif ((CONFIG_PRINTF >= 1) && (CONFIG_PRINTF_UART_NUM == 2))
#undef CONFIG_DRIVER_UART_2
#define CONFIG_DRIVER_UART_2            1

#else
#error "CONFIG_PRINTF enable, but CONFIG_PRINTF_UART_NUM is error value!"
#endif
#endif /* CONFIG_PRINTF */
```

Note: The UART pinmux is defined in each [board.c](#) file, such as [mrs6130_p1812.c](#).

Include files as follows:

```
/* Include common.h or hal_types.h before log.h. */
#include "common.h"

#define LOG_MODULE          "name"      /* Module prefix in log output */
#define LOG_LEVEL           LEVEL_ERR   /* Log level */
#include "log.h"
```

The available logging macros are defined in [log.h](#). They include level-specific macros such as LOG_ERR and LOG_ERR_HEX, and macros such as LOG_PRINT and LOG_HEX that are not restricted by LOG_LEVEL. See the sample project [project/basic/shell log](#).

```
LOG_PRINT("Current 'LOG_LEVEL' is '%s'\n", strLogLevel[logLevel]);
LOG_PRINT("Current 'LOG_MODULE' is '%s'\n\n", LOG_MODULE);

LOG_PRINT("1> This is 'LOG_PRINT' print\n");
```

```
LOG_HEX(sampleData, SAMPLE_DATA_SIZE, "2> This is 'LOG_HEX' print");
LOG_DBG("3> This is 'LOG_DBG' print\n");
LOG_DBG_HEX(sampleData, SAMPLE_DATA_SIZE, "2> This is 'LOG_DBG' print");
LOG_INF("1> This is 'LOG_INF' print\n");
LOG_INF_HEX(sampleData, SAMPLE_DATA_SIZE, "2> This is 'LOG_INF_HEX' print");
LOG_WRN("1> This is 'LOG_WRN' print\n");
LOG_WRN_HEX(sampleData, SAMPLE_DATA_SIZE, "2> This is 'LOG_WRN_HEX' print");
LOG_ERR("1> This is 'LOG_ERR' print\n");
LOG_ERR_HEX(sampleData, SAMPLE_DATA_SIZE, "2> This is 'LOG_ERR_HEX' print");
```

LOG_IO is a debugging method that toggles an I/O pin high and low. Configure CONFIG_LOG_IO_TAG_MASK and CONFIG_LOG_IO_MASK in [prj_config.h](#). See the sample project [project/basic/run time measure](#).

```
/* Define a TAG. It must not overlap an existing system TAG defined in log.h. */
#define CONFIG_LOG_IO_TAG_MASK 0x00040000
/* Define the I/O mask. 0x3 indicates that PA0 and PA1 are controlled. */
#define CONFIG_LOG_IO_MASK 0x3

LOG_IO(0x00040000, 1, 1, 0); /* Drive PA1 high */
OSI_MSleep(500);
LOG_IO(0x00040000, 1, 0, 0); /* Drive PA1 low */

LOG_IO(0x00040000, 1, 0, 3); /* Output three consecutive positive pulses on PA1 */
```

4.9 Console Shell

To use the shell, add the source files and path under [platform/subsys/shell](#) to the build and enable CONFIG_SHELL.

```
/* Enable the shell */
#ifndef CONFIG_SHELL
#define CONFIG_SHELL 0
#endif

/* Configure the shell UART number */
#if CONFIG_SHELL
#ifndef CONFIG_SHELL_UART_NUM
#define CONFIG_SHELL_UART_NUM 0
#endif
/* Configure the shell UART baud rate */
#ifndef CONFIG_SHELL_UART_BAUDRATE
#define CONFIG_SHELL_UART_BAUDRATE 115200
#endif
```

Several default shell commands are defined in [platform/cmd](#) for reference.

```
/* Shell implementation function */
int set_config_cmd(Shell *shell, int argc, char *argv[])
{
    /* ..... */
}
```

```

return 0;
}

/* shell subcommand */
ShellCommand gpio_cmds[] = {
/* set_config: subcommand prompt
 * set_config_cmd: command implementation function
 * "<port>": command-help prompt
 */
SHELL_CMD_ITEM(set_config,      set_config_cmd, "<port> <pull>"),
SHELL_CMD_ITEM_END()
};

/* shell command group */
SHELL_CMD_GROUP(gpio, gpio_cmds, "gpio command");

```

4.10 Power Management

4.10.1 Introduction

Power management is implemented through the OS idle thread. The system enters the idle thread when no other application thread is running. The idle thread obtains the time until the next task is unblocked and compares it with the preset sleep-entry latency (`enter_latency`) and wake-up latency (`exit_latency`) to select light sleep or deep sleep. If deep sleep is possible, the system configures the low-power timer wake-up time and calls each device's sleep interface in registration order. System sleep processing then saves the system thread context and configures the hardware to power down. The low-power timer or another wake-up source wakes the system. After the hardware powers up, the system resume function restores the system context and handles the wake-up source. It then calls the device resume interfaces in reverse sleep order, enables interrupts, and performs a context switch.

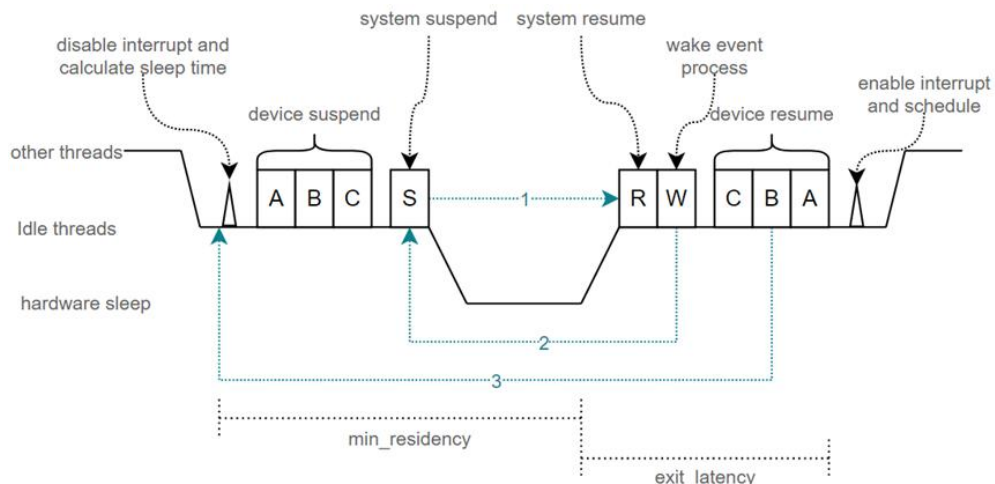


Figure 4-12 Power Management Flow

Before using the power-management APIs, enable the following macros in the project configuration file:

```

#define CONFIG_PM 1
#define CONFIG_PM_DEVICE 1

/* To use device runtime power management, also configure this macro. */
#define CONFIG_PM_DEVICE_RUNTIME 1

```

4.10.2 Interface

4.10.2.1 pm_device_register

Table 4-9 pm_device_register

Interface	Description
Function	<code>int pm_device_register(struct pm_device *dev, pm_device_action_cb_t action);</code>
Description	Registers a device for sleep and wake-up handling.
Parameters	struct pm_device *dev Points to the device object to register. Define an object of type struct pm_device; its members do not need to be assigned. pm_device_action_cb_t action Device sleep/resume callback. It is executed whenever the system enters or exits sleep. pm_device_action_cb_t is declared as follows: <pre>typedef int (*pm_device_action_cb_t)(void *dev, enum pm_device_action action), pm_device_action is declared as follows: PM_DEVICE_ACTION_SUSPEND, /* Suspend the device */ PM_DEVICE_ACTION_RESUME, /* Resume the device */ PM_DEVICE_ACTION_TURN_OFF, /* Currently unused */ PM_DEVICE_ACTION_TURN_ON, /* Currently used */ };</pre>
Considerations	None.
Return Value	Returns 0 if registration succeeds, 1 if the device is already registered, or another value if registration fails.

4.10.2.2 pm_device_unregister

Table 4-10 pm_device_unregister

Interface	Description
Function	<code>int pm_device_unregister(struct pm_device *dev)</code>
Description	Unregisters a device from sleep and wake-up handling.
Parameters	struct pm_device *dev Points to a registered device object.
Considerations	None.
Return Value	Returns 0 if unregistration succeeds; any other value indicates failure.

4.10.2.3 pm_notifier_register

Table 4-11 pm_notifier_register

Interface	Description
Function	<code>void pm_notifier_register(struct pm_notifier *notifier);</code>
Description	Register a device sleep notification.
Parameters	struct pm_notifier *notifier

Interface	Description
	Points to the notification object to register. struct pm_notifier is declared as follows: <pre>struct pm_notifier { sys_snode_t _node; void (*state_entry)(enum pm_state state); void (*state_exit)(enum pm_state state); };</pre> After registration succeeds, the functions pointed to by state_entry and state_exit are executed when the system enters and exits sleep, respectively.
Considerations	None.
Return Value	None.

4.10.2.4 pm_notifier_unregister

Table 4-12 pm_notifier_unregister

Interface	Description
Function	int pm_notifier_unregister(struct pm_notifier *notifier);
Description	Unregister a device sleep notification.
Parameters	struct pm_notifier *notifier Points to a registered notification object.
Considerations	None.
Return Value	Returns 0 if unregistration succeeds; any other value indicates failure.

4.10.2.5 pm_policy_latency_update

Table 4-13 pm_policy_latency_update

Interface	Description
Function	void pm_policy_latency_update(uint32_t entry_latency, uint32_t exit_latency);
Description	Update the sleep-entry and wake-up latencies.
Parameters	uint32_t entry_latency Time required for the system to enter sleep. uint32_t exit_latency Time required for the system to exit sleep. Update these values based on the actual execution time of the application.
Considerations	None.
Return Value	None.

4.10.2.6 pm_policy_wake_lock_is_active

Table 4-14 pm_policy_wake_lock_is_active

Interface	Description
Function	bool pm_policy_wake_lock_is_active(void);
Description	Check whether a wake lock exists.

Interface	Description
Parameters	None.
Considerations	None.
Return Value	Returns true if a wake lock exists, or false if no wake lock exists.

4.10.2.7 pm_policy_wake_lock

Table 4-15 pm_policy_wake_lock

Interface	Description
Function	void pm_policy_wake_lock(struct wakelock *lock);
Description	Acquire a wake lock. The system cannot enter sleep while it holds a wake lock. Use together with pm_policy_wake_unlock.
Parameters	struct wakelock *lock Wake lock to acquire. Its type is declared as follows: <pre>struct wakelock { sys_snode_t _node; /* Wake-lock node */ char *name; /* Wake-lock name; configurable by the user */ uint16_t ref; /* Wake-lock count; no user configuration required */ uint32_t expires; /* Wake-lock timeout; no user configuration required */ };</pre>
Considerations	None.
Return Value	None.

4.10.2.8 pm_policy_wake_unlock

Table 4-16 pm_policy_wake_unlock

Interface	Description
Function	void pm_policy_wake_unlock(struct wakelock *lock);
Description	Release a wake lock. Use together with pm_policy_wake_lock.
Parameters	struct wakelock *lock Wake lock to release.
Considerations	None.
Return Value	None.

4.10.2.9 pm_wake_lock_timeout

Table 4-17 pm_wake_lock_timeout

Interface	Description
Function	void pm_wake_lock_timeout(struct wakelock *lock, uint32_t timeout);
Description	Acquire a timed wake lock.
Parameters	struct wakelock *lock Wake lock to acquire. uint32_t timeout

Interface	Description
	Duration for which the wake lock is held, in ms. After the lock is acquired, the system releases it automatically after timeout. To release it early, set timeout to 0.
Considerations	None.
Return Value	None.

4.10.2.10 pm_device_runtime_enable

Table 4-18 pm_device_runtime_enable

Interface	Description
Function	<code>int pm_device_runtime_enable(struct pm_device *dev);</code>
Description	Enable device runtime power management, which powers on the device only when the application needs it.
Parameters	struct pm_device *dev Device for which runtime power management is to be enabled. When this function is enabled, an already-open device is closed, and its sleep/resume behavior is no longer controlled by system power management.
Considerations	None.
Return Value	Returns 0 if enabling succeeds; any other value indicates failure.

4.10.2.11 pm_device_runtime_disable

Table 4-19 pm_device_runtime_disable

Interface	Description
Function	<code>int pm_device_runtime_disable(struct pm_device *dev);</code>
Description	Disables device runtime power management, which powers on the device only when the application needs it.
Parameters	struct pm_device *dev Device for which runtime power management is to be disabled.
Considerations	None.
Return Value	Returns 0 if disabling succeeds; any other value indicates failure.

4.10.2.12 pm_device_runtime_get

Table 4-20 pm_device_runtime_get

Interface	Description
Function	<code>int pm_device_runtime_get(struct pm_device *dev);</code>
Description	Open a device when the application needs to use it.
Parameters	struct pm_device *dev Device to open.
Considerations	None.

Interface	Description
Return Value	Returns 0 if opening succeeds; any other value indicates failure.

4.10.2.13 pm_device_runtime_put

Table 4-21 pm_device_runtime_put

Interface	Description
Function	int pm_device_runtime_put(struct pm_device *dev);
Description	Close a device when the application no longer needs it.
Parameters	struct pm_device *dev Device to close.
Considerations	None.
Return Value	Returns 0 if disabling succeeds; any other value indicates failure.

4.11 KVR & KVF

4.11.1 Introduction

The SDK provides two key-value storage systems: KVR in SRAM and KVF in flash.

KVR occupies 1 KB at the low end of SRAM and is divided into system and application storage. It is lost after power-off. System storage is cleared and initialized at first power-on, while application storage is cleared and reinitialized on every system reset. It is implemented in [subsys/kv/kvr](#).

KVF occupies the final 8 KB of flash, with 4 KB of usable storage (8 KB total for ping-pong operation). It is retained after power-off and cleared by a full-chip erase. It is implemented in [subsys/kv/kvf](#).

It is initialized by default in [Board.c](#) during system initialization.

4.11.2 Interface

4.11.2.1 KVR_Save

Table 4-22 KVR_Save

Interface	Description
Function	int KVR_Save(uint16_t id, void *pdata, uint32_t size);
Description	Create and save data in SRAM.
Parameters	uint16_t id Storage ID. If the ID already exists, its data is overwritten; otherwise, a new storage ID is created. For system storage, set the most significant bit to 1. void *pdata Data to store uint32_t size Length of the data to store. For the same ID, if the new data is longer than the previous data, the storage allocated to that ID is expanded.
Considerations	None.

Interface	Description
Return Value	A value less than or equal to 0 indicates storage failure; a value greater than 0 is the stored-data length.

4.11.2.2 KVR_Load

Table 4-23 KVR_Load

Interface	Description
Function	int KVR_Load(uint16_t id, void *pdata, uint32_t size);
Description	Read stored data from the specified ID.
Parameters	uint16_t id Storage ID. For system storage, set the most significant bit to 1. void *pdata Data to store uint32_t size Length of the data to read. If the ID has not been created, an error code is returned. If the requested length is less than or equal to the stored length, the requested length is returned; otherwise, only the stored length is returned.
Considerations	None.
Return Value	A value less than or equal to 0 indicates a read failure; a value greater than 0 is the length of the data read.

4.11.2.3 KVR_Delete

Table 4-24 KVR_Delete

Interface	Description
Function	int KVR_Delete(uint16_t id);
Description	Delete data with the specified ID from the storage system.
Parameters	uint16_t id Storage ID. For system storage, set the most significant bit to 1.
Considerations	None.
Return Value	A value less than or equal to 0 indicates a read failure; a value greater than 0 is the length of the data read.

4.11.2.4 KVF_Save

Table 4-25 KVF_Save

Interface	Description
Function	int KVF_Save(uint16_t id, void *pdata, uint32_t size);
Description	Create and save data in flash.
Parameters	uint16_t id

Interface	Description
	Storage ID. If the ID already exists, its data is overwritten; otherwise, a new storage ID is created. For system storage, set the most significant bit to 1. void *pdata Data to store uint32_t size Length of the data to store. For the same ID, if the new data is longer than the previous data, the storage allocated to that ID is expanded.
Considerations	None.
Return Value	A value less than or equal to 0 indicates storage failure; a value greater than 0 is the stored-data length.

4.11.2.5 KVF_Load

Table 4-26 KVF_Load

Interface	Description
Function	<code>int KVF_Load(uint16_t id, void *pdata, uint32_t size);</code>
Description	Read stored data from the specified ID.
Parameters	uint16_t id Storage ID. For system storage, set the most significant bit to 1. void *pdata Data to store uint32_t size Length of the data to read. If the ID has not been created, an error code is returned. If the requested length is less than or equal to the stored length, the requested length is returned; otherwise, only the stored length is returned.
Considerations	None.
Return Value	A value less than or equal to 0 indicates a read failure; a value greater than 0 is the length of the data read.

4.11.2.6 KVF_Delete

Table 4-27 KVF_Delete

Interface	Description
Function	<code>int KVF_Delete(uint16_t id);</code>
Description	Delete data with the specified ID from the storage system.
Parameters	uint16_t id Storage ID. For system storage, set the most significant bit to 1.
Considerations	None.

Interface	Description
Return Value	A value less than or equal to 0 indicates a read failure; a value greater than 0 is the length of the data read.

4.12 Version

4.12.1 Introduction

A set of APIs is provided to obtain the versions of system images. The image IDs are as follows.

```
typedef enum {
    VER_TYPE_SYS                = 0, /* Automatically identifies sys0 or sys1 based on the current system */
    VER_TYPE_SYS0               = 1,
    VER_TYPE_SYS1               = 2,
    VER_TYPE_MMW                = 3,
    VER_TYPE_MMW_ALG            = 4,
    VER_TYPE_MPW                = 5,
    VER_TYPE_CUSTOM_CFG         = 6,
    VER_TYPE_FLASH_LIST         = 7,
    VER_TYPE_MAX
} VER_SubType_t;
```

Implemented in [platform/utils/version.h](#).

A sub-image version is typically:

[Major.minor.revision.build_num](#)

The Sys image version is determined by the following macros. The application version can be defined in each [prj_config.h](#).

```
#ifndef CONFIG_PROJECT_VERSION_MAJOR
#define CONFIG_PROJECT_VERSION_MAJOR 1
#endif

#ifndef CONFIG_PROJECT_VERSION_MINOR
#define CONFIG_PROJECT_VERSION_MINOR 0
#endif

#ifndef CONFIG_PROJECT_VERSION_REVISION
#define CONFIG_PROJECT_VERSION_REVISION 0
#endif

#ifndef CONFIG_PROJECT_VERSION_BUILD
#define
```

The merged-image version is:

[R minor.type - M minor.project.number-V major.minor.revision](#)

This version is typically provided by Possumic technology.

4.12.2 Interface

4.12.2.1 VER_GetSdkVersion

Table 4-28 VER_GetSubimgVersion

Interface	Description
Function	int VER_GetSubimgVersion(VER_SubType_t verType, VER_SubVersion_t * pver);
Description	Obtain a sub-image version number.
Parameters	VER_SubType_t verType Sub-image version type. <pre>typedef enum { VER_TYPE_SYS = 0, VER_TYPE_SYS0 = 1, VER_TYPE_SYS1 = 2, VER_TYPE_MMW = 3, VER_TYPE_MMW_ALG = 4, VER_TYPE_MPW = 5, VER_TYPE_CUSTOM_CFG = 6, VER_TYPE_FLASH_LIST = 7, VER_TYPE_MAX } VER_SubType_t;</pre> VER_SubVersion_t * pver Sub-image version number. <pre>typedef struct { uint8_t iv_major; uint8_t iv_minor; uint16_t iv_revision; uint32_t iv_build_num; } VER_SubVersion_t;</pre>
Considerations	None.
Return Value	Returns 0 on success; a nonzero value indicates failure.

4.12.2.2 VER_GetSubimgVersion

Table 4-29 VER_GetSdkVersion

Interface	Description
Function	int VER_GetSdkVersion(VER_SdkVersion_t * pver);
Description	Obtains the SDK version.
Parameters	VER_SdkVersion_t * pver SDK version number. Defined as follows: <pre>typedef struct { uint16_t ir_minor; uint8_t ir_type;</pre>

Interface	Description
	<pre>uint8_t im_project; uint16_t im_number; uint16_t im_minor; uint8_t iv_major; uint8_t iv_minor; uint16_t iv_revision; } VER_SdkVersion_t;</pre>
Considerations	None.
Return Value	Returns 0 on success; a nonzero value indicates failure.

4.12.2.3 VER_GetBootVersion

Table 4-30 VER_GetBootVersion

Interface	Description
Function	<code>int VER_GetBootVersion(VER_BootVersion_t * pver);</code>
Description	Obtains the bootloader version number.
Parameters	VER_BootVersion_t * pver Bootloader version number. Defined as follows: <pre>typedef struct { uint8_t iv_major; uint8_t iv_minor; uint16_t iv_revision; } VER_BootVersion_t;</pre>
Considerations	None.
Return Value	Returns 0 on success; a nonzero value indicates failure.

4.12.2.4 VER_GetBuildTime

Table 4-31 VER_GetBuildTime

Interface	Description
Function	<code>void VER_GetBuildTime(char *pdate, char *ptime);</code>
Description	Obtains the application build time.
Parameters	char *pdate Build date, such as Aug 25 2025. char *ptime Build time, such as 09:22:53.
Considerations	None.
Return Value	None.

4.13 HIF

4.13.1 Function Overview

The SDK provides the Host Interface (HIF) submodule for communication with other devices. It implements the HIF protocol format and communication flow in three main layers: the application interface layer, HIF layer, and physical interface layer. UART and SPI hardware interfaces are supported and selected through macros. For timing requirements and data-frame definitions, read this manual together with '[RS6x 7x HIF Communication Format Description Manual.pdf](#)'. For interface usage, see the [project/subsys/hif/hif_uart](#) project.

4.13.2 Macro Switches

Before using the HIF module, enable the HIF master switch in [prj_config.h](#), then select the communication interface and wake-up method.

Table 4-32 CONFIG_HIF

Macro	Description
CONFIG_HIF	HIF master switch
CONFIG_HIF_DEVICE_COM_UART	Enable UART as the HIF communication interface.
CONFIG_HIF_UART_PORT	Select the HIF UART; if undefined, uart0 is used by default.
CONFIG_HIF_DEVICE_WAKEUP_UART	Enables HIF UART wake-up.
CONFIG_HIF_DEVICE_COM_SPI	Enable SPI as the HIF communication interface.
CONFIG_HIF_DEVICE_WAKEUP_SPI	Enables HIF SPI wake-up.
CONFIG_HIF_SEND_DMA	Enable DMA (supported only for UART/SPI). If this macro is not enabled, interrupt mode is used by default. Interrupt mode is not currently supported.

4.13.3 API Description

4.13.3.1 HIF_Init

Table 4-33 HIF_Init

Interface	Description
Function	<code>int HIF_Init(HIF_InitParam_t *pInitParam);</code>
Description	Initialize the HIF module and its associated resources.
Parameters	<pre>typedef enum { HIF_COM_TYPE_DIS = 0, HIF_COM_TYPE_UART = 1, HIF_COM_TYPE_IIC = 2, //Not currently supported HIF_COM_TYPE_SPI = 3, } HIF_ComType_t;</pre> <pre>typedef enum {</pre>

Interface	Description
	<pre> HIF_NOTIFY_TYPE_DIS = 0, HIF_NOTIFY_TYPE_IO = 1, //Drive the I/O high to notify the host to read data HIF_NOTIFY_TYPE_DLY = 2, //Send data to the host after a delay; not recommended for communication types other than UART } HIF_NotifyType_t; typedef struct { HIF_ComType_t comType; //Communication interface uint32_t comRate; //Communication rate: SPI requires no modification and supports host access below 60 MHz; UART supports common baud rates below 2 Mbps uint32_t comParam; //Communication parameter HIF_NotifyType_t notifyType; //Notification type HIF_IOLevel_t notifyIoLevel; //Notification I/O level uint32_t notifyIoNum; //Notification I/O pin HIF_WakeType_t wakeType; //Wake-up type HIF_IOLevel_t wakeIoLevel; //Wake-up I/O level uint32_t wakeIoNum; //Wake-up I/O pin uint32_t wakeComParam; //Wake-up threshold uint32_t rcvFrameTo; //Receive timeout uint32_t sendFrameTo; //Transmit timeout uint32_t sendDelay; //Delayed-send time uint32_t sleepTo; //Sleep timeout uint32_t splitSize; //Packet-splitting size; not currently supported uint32_t splitEna; //Enable packet splitting; not currently supported uint32_t simplex; //Enable simplex mode; not currently supported uint32_t retry_enable; //Enable HIF-layer retransmission; not currently supported } HIF_InitParam_t; </pre>
Considerations	1、 Before calling this interface, refer to HIF_DefaultInitParam to initialize plnitParam. 2、 This interface must not be called more than once.
Return Value	Returns 0 if initialization succeeds; any other value indicates failure.

4.13.3.2 HIF_DeInit

Table 4-34 HIF_DeInit

Interface	Description
Function	int HIF_DeInit(void);
Description	Deinitialize the HIF module and release HIF-layer resources.

Interface	Description
Parameters	None
Considerations	This interface must not be called more than once.
Return Value	Returns 0 if deinitialization succeeds; any other value indicates failure.

4.13.3.3 HIF_DefaultInitParam

Table 4-35 HIF_DefaultInitParam

Interface	Description
Function	<code>int HIF_DefaultInitParam(HIF_InitParam_t * cfgParam, HIF_ComType_t comType, uint32_t comRate);</code>
Description	Initialize an HIF_InitParam_t object with default values for use by HIF_Init.
Parameters	HIF_InitParam_t * cfgParam Address of the cfgParam parameter HIF_ComType_t comType Communication type typedef enum { HIF_COM_TYPE_DIS = 0, HIF_COM_TYPE_UART = 1, HIF_COM_TYPE_IIC = 2, //Not currently supported HIF_COM_TYPE_SPI = 3, } HIF_ComType_t; uint32_t comRate Baud Rate
Considerations	None
Return Value	Returns 0 if the default values are set successfully; any other value indicates failure.

4.13.3.4 HIF_ExtControl

Table 4-36 HIF_ExtControl

Interface	Description
Function	<code>int HIF_ExtControl(HIF_Attr_t attr, void * arg, uint32_t argSize);</code>
Description	Set an individual HIF_InitParam_t member by type.
Parameters	HIF_Attr_t attr Type of parameter to set. typedef enum { HIF_SET_COM_TYPE = 0, HIF_SET_COM_RATE = 1, HIF_SET_COM_PARAM = 2, HIF_SET_NOTIFY_TYPE = 3, HIF_SET_NOTIFY_IO_LEVEL = 4, }

Interface	Description
	<pre> HIF_SET_NOTIFY_IO_NUM = 5, HIF_SET_WAKE_TYPE = 6, HIF_SET_WAKE_IO_LEVEL = 7, HIF_SET_WAKE_IO_NUM = 8, HIF_SET_WAKE_COM_PARAM = 9, HIF_SET_RECV_FRAME_TO = 10, HIF_SET_SEND_FRAME_TO = 11, HIF_SET_SEND_DELAY = 12, HIF_SET_SLEEP_TO = 13, HIF_SET_SPLIT_SIZE = 14, //Not currently supported HIF_SET_SPLIT_ENA = 15, //Not currently supported HIF_SET_SIMPLEX_MODE = 16, //Not currently supported HIF_SET_RETRY = 17, //Not currently supported } HIF_Attr_t; void * arg Parameter pointer uint32_t argSize Parameter length </pre>
Considerations	Call after HIF_DefaultInitParam and before HIF_Init.
Return Value	Returns 0 on success; any other value indicates failure.

4.13.3.5 HIF_ParamGet

Table 4-37 HIF_ParamGet

Interface	Description
Function	<code>int HIF_ParamGet(HIF_Attr_t attr, void * arg, uint32_t argSize);</code>
Description	Get an individual HIF_InitParam_t member by type.
Parameters	HIF_Attr_t attr Type of parameter to get. <pre> typedef enum { HIF_SET_COM_TYPE = 0, HIF_SET_COM_RATE = 1, HIF_SET_COM_PARAM = 2, HIF_SET_NOTIFY_TYPE = 3, HIF_SET_NOTIFY_IO_LEVEL = 4, HIF_SET_NOTIFY_IO_NUM = 5, HIF_SET_WAKE_TYPE = 6, HIF_SET_WAKE_IO_LEVEL = 7, HIF_SET_WAKE_IO_NUM = 8, HIF_SET_WAKE_COM_PARAM = 9, HIF_SET_RECV_FRAME_TO = 10, HIF_SET_SEND_FRAME_TO = 11, </pre>

Interface	Description
	<pre> HIF_SET_SEND_DELAY = 12, HIF_SET_SLEEP_TO = 13, HIF_SET_SPLIT_SIZE = 14, //Not currently supported HIF_SET_SPLIT_ENA = 15, //Not currently supported HIF_SET_SIMPLEX_MODE = 16, //Not currently supported HIF_SET_RETRY = 17, //Not currently supported } HIF_Attr_t; void * arg Parameter pointer uint32_t argSize Parameter length </pre>
Considerations	None
Return Value	Returns 0 on success; any other value indicates failure.

4.13.3.6 HIF_MsgHdl_Regist

Table 4-38 HIF_MsgHdl_Regist

Interface	Description
Function	<code>int HIF_MsgHdl_Regist(uint8_t msgId, HIF_MsgHdl_t msgHdl);</code>
Description	Register a callback by msgId. After successful registration, the callback is invoked when the device receives a valid HIF data frame with this msgId.
Parameters	uint8_t msgId Msg ID HIF_MsgHdl_t msgHdl Callback function. NULL is equivalent to unregistering the callback. typedef int (* HIF_MsgHdl_t)(HIF_MsgHdr_t *msg);
Considerations	None.
Return Value	Returns 0 if registration succeeds; any other value indicates failure.

4.13.3.7 HIF_MsgResp

Table 4-39 HIF_MsgResp

Interface	Description
Function	<code>int HIF_MsgResp(HIF_MsgHdr_t *msg, uint16_t dataLen, uint8_t status);</code>
Description	Reply after receiving a host command. When dataLen is 0, the payload length is 1 and the payload contains status. When dataLen is nonzero, obtain the reply-data address through HIF_Msg_AckDataPtr and copy the reply data to that address.
Parameters	HIF_MsgHdr_t *msg HIF frame header. For details, see 'Host Driver Development Manual.pdf'. typedef struct hif_message_header_t {

Interface	Description
	<pre> uint32_t type : 2; uint32_t flag : 6; uint32_t msg_id : 8; uint32_t length : 12; uint32_t seq : 3; uint32_t frag : 1; } HIF_MsgHdr_t uint16_t dataLen Data length uint8_t status Reply status. </pre>
Considerations	Pass the callback's msg argument as this interface's msg argument. Example: <pre> int ft_voltage_handle(HIF_MsgHdr_t *msg) { uint8_t *ack_data = (uint8_t *)HIF_Msg_AckDataPtr(msg); uint32_t adc_val = mp_ctrl_adc_read_mv(); memcpy(ack_data, &adc_val, sizeof(adc_val)); return HIF_MsgResp(msg, sizeof(ack_data), status); } </pre>
Return Value	Returns 0 if registration succeeds; any other value indicates failure.

4.13.3.8 HIF_MsgReport

Table 4-40 HIF_MsgReport

Interface	Description
Function	<code>int HIF_MsgReport(HIF_MsgHdr_t *msg, void *data, HIF_MsgTrans_Callback sentCb);</code>
Description	Add report data to the transmit queue.
Parameters	HIF_MsgHdr_t *msg HIF frame header. For details, see 'Host Driver Development Manual.pdf'. <pre> typedef struct hif_message_header_t { uint32_t type : 2; uint32_t flag : 6; uint32_t msg_id : 8; uint32_t length : 12; uint32_t seq : 3; uint32_t frag : 1; } HIF_MsgHdr_t </pre> void *data Report Data HIF_MsgTrans_Callback sentCb

Interface	Description
	Data-report callback. Transmission is asynchronous when the callback is non-NULL and blocking when it is NULL. typedef void (* HIF_MsgTrans_Callback)(hif_queue_item_t *hif_queue_item, int8_t status);
Considerations	1、The registered callback is invoked when transmission completes or fails. Determine success from the callback status.
Return Value	Returns 0 if the data is added to the transmit queue; any other value indicates failure.

4.13.3.9 HIF_MsgReport_ListAddData

Table 4-41 HIF_MsgReport_ListAddData

Interface	Description
Function	payload_node_t *HIF_MsgReport_ListAddData(payload_node_t * payload_head, void *data, uint16_t len);
Description	Adds payload report data in blocks.
Parameters	payload_node_t * payload_head Pointer to the payload head node. Pass NULL for the first payload; the return value is the successfully allocated head node. void *data Address of the data to send. uint16_t len Size of the data to send.
Considerations	None.
Return Value	Returns the payload head pointer.

4.13.3.10 HIF_MsgReport_ListStart

Table 4-42 HIF_MsgReport_ListStart

Interface	Description
Function	int HIF_MsgReport_ListStart(HIF_MsgHdr_t *msg, payload_node_t * payload_head, HIF_MsgTrans_Callback sentCb);
Description	Start transmission after adding the payload in blocks.
Parameters	HIF_MsgHdr_t *msg HIF frame header. For details, see 'Host Driver Development Manual.pdf'. payload_node_t * payload_head Address of the head node allocated when the payload was added. HIF_MsgTrans_Callback sentCb Data-report callback. Transmission is asynchronous when the callback is non-NULL and

Interface	Description
	blocking when it is NULL.
Considerations	1. The registered callback is invoked when transmission completes or fails. Determine success from the callback status.
Return Value	Returns 0 if the data is added to the transmit queue; any other value indicates failure.

4.14 Other

The SDK also provides general-purpose interfaces, such as obtaining the system clock and system frequency.

4.14.1 Interface

4.14.1.1 HAL_BOARD_GetFreq

Table 4-43 HAL_BOARD_GetFreq

Interface	Description
Function	int32_t HAL_BOARD_GetFreq(CLOCK_Type_t clkType);
Description	Obtains the system frequency.
Parameters	CLOCK_Type_t clkType Frequency type. <pre>typedef enum { CLOCK_LSI = 0, CLOCK_MSI = 1, CLOCK_LSE = 2, CLOCK_DCXO = 3, CLOCK_LPCLK = 4, CLOCK_LPCLK_INTE_DIV = 5, CLOCK_LPCLK_FRAC_DIV = 6, CLOCK_PLL = 7, CLOCK_PLL_SOC = 8, CLOCK_PLL_DEV = 9, CLOCK_PLL_DEV1 = 10, CLOCK_PLL_CDAA = 11, CLOCK_SYS = 12, CLOCK_AHB = 13, CLOCK_APB0 = 14, CLOCK_APB1 = 15, CLOCK_CURR_CPU = 16, CLOCK_CPUS = 17, CLOCK_CPUF = 18, CLOCK_MAX = 19 } CLOCK_Type_t;</pre>

Interface	Description
Considerations	None.
Return Value	A value greater than 0 is the system frequency; a value less than 0 indicates failure; 0 indicates that the clock of the current type is disabled.

4.14.1.2 HAL_BOARD_GetTime

Table 4-44 HAL_BOARD_GetTime

Interface	Description
Function	uint32_t HAL_BOARD_GetTime(HAL_TimeType_t timeType);
Description	Obtains the system clock.
Parameters	HAL_TimeType_t timeType Clock type. <pre>typedef enum { HAL_TIME_CYCLE = 0, // AHB clock cycles HAL_TIME_US = 1, // Microseconds HAL_TIME_MS = 2, // Milliseconds HAL_TIME_S = 3, // Seconds } HAL_TimeType_t;</pre>
Considerations	Can be used to measure system software execution time.
Return Value	A value greater than or equal to 0 is the system frequency; a value less than 0 indicates failure.

4.14.1.3 HAL_BOARD_BlDelay

Table 4-45 HAL_BOARD_BlDelay

Interface	Description
Function	void HAL_BOARD_BlDelay(uint32_t delayVal, HAL_TimeType_t timeType);
Description	Blocking delay.
Parameters	uint32_t delayVal Delay value. HAL_TimeType_t timeType Delay unit.
Considerations	The delay does not yield the CPU and may prevent lower-priority tasks from being scheduled when used at high priority.
Return Value	None

4.14.1.4 HAL_BOARD_GetRand

Table 4-46 HAL_BOARD_GetRand

Interface	Description
Function	uint32_t HAL_BOARD_GetRand(void);
Description	Obtains a random number.
Parameters	None
Considerations	None
Return Value	Random number.

4.14.1.5 HAL_BOARD_Reset

Table 4-47 HAL_BOARD_Reset

Interface	Description
Function	void HAL_BOARD_Reset(HAL_ResetType_t resetType);
Description	Reset the system.
Parameters	HAL_ResetType_t resetType Reset type. typedef enum { HAL_RESET_CPU = 0, // reset the CPU only HAL_RESET_SYS = 1, // reset the CPU and other peripherals } HAL_ResetType_t;
Considerations	None
Return Value	Random number.

5 Peripheral Drivers

This chapter introduces the on-chip peripheral drivers to help users quickly develop with the supported on-chip resources.

5.1 GPIO

GPIO (General-Purpose Input/Output) is the basic I/O module of RS-series SoCs. Each pin is an independent control unit and can provide input, output, or multiplexed functions. Multiple pins form a port such as Port A or Port B. See the applicable SoC data sheet for the number of pins and supported functions.

Examples:

For the RS6240 SoC, see Section 4.3, GPIO Multiplexing, in 'SDK/01 Document/RS6240 Datasheet EN.pdf'.

The RS6240 SoC supports two GPIO ports: Port A with 16 pins and Port B with 8 pins.

Each RS6240 SoC pin supports up to 16 selectable functions, one of which can be active at a time.

5.1.1 Function Overview

The basic structure of a GPIO pin is shown below.

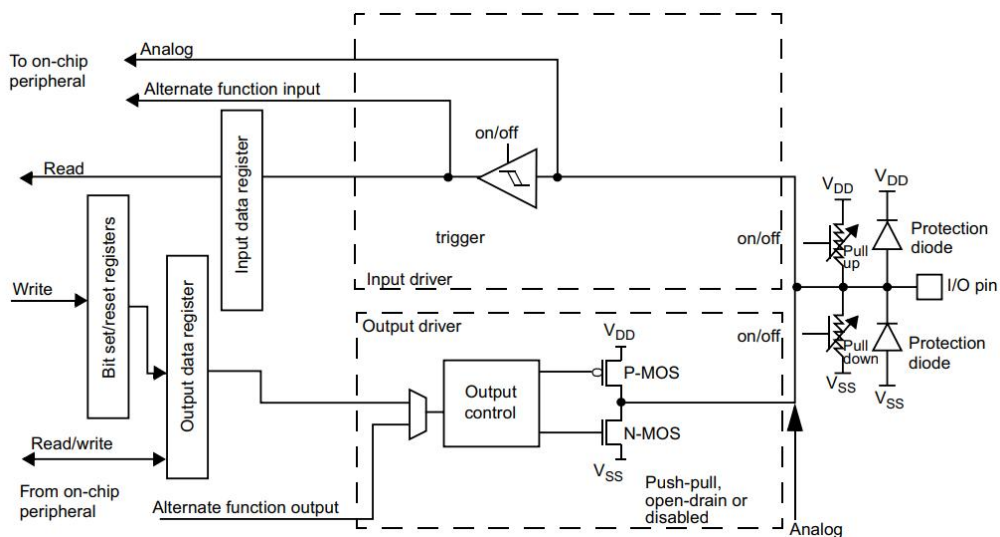


Figure 5-1 GPIO Framework Diagram

5.1.1.1 Pull-up/Pull-down Resistors

The SoC contains software-controlled pull-up and pull-down resistors, typically about 50-100 kΩ. See Digital I/O Characteristics in the data sheet for exact values.

Internal pull resistors are normally used in input mode and input-type multiplexed modes to maintain a defined level. With both resistors disabled, the input floats and may be susceptible to interference; UART RX, for example, should be configured with a pull-up. Pull resistors are also used in some output applications, such as the pull-up that implements wired-AND behavior for an I2C open-drain output.

The API parameters for selecting pull-up and pull-down resistors are as follows:

```
typedef enum {
    GPIO_PULL_FLOATING = 0U,    /* Disable both internal pull-up and pull-down resistors */
    GPIO_PULL_UP       = 1U,    /* Enable the internal pull-up resistor */
    GPIO_PULL_DOWN     = 2U,    /* Enable the internal pull-down resistor */
} GPIO_Pull_t;
```

5.1.1.2 Output Drive Capability

The output is driven by P-MOS and N-MOS transistors. Push-pull mode uses both, while open-drain mode uses only the N-MOS transistor. The SoC provides four push-pull drive levels, corresponding to 20-80 mA at 3.3 V or 6-24 mA at 1.8 V. See the GPIO chapter of the data sheet for exact values; the drive-current step differs with voltage. Different communication rates have different signal-settling requirements. More drive current shortens settling time, but the drive level should match the application. Open-drain drive capability is governed mainly by the pull-up resistor.

The API parameters for selecting the output mode are as follows:

```
typedef enum {
    GPIO_PUSH_PULL    = 0U,    /* push-pull output */
    GPIO_OPEN_DRAIN   = 1U     /* open-drain output */
} GPIO_Push_t;
```

The API parameters for push-pull drive strength are as follows:

```
typedef enum {
    GPIO_DRIVING_LEVEL_0 = 0U,    /* Approximately 20 mA at 3.3 V / 6 mA at 1.8 V */
    GPIO_DRIVING_LEVEL_1 = 1U,    /* Approximately 40 mA at 3.3 V / 12 mA at 1.8 V */
    GPIO_DRIVING_LEVEL_2 = 2U,    /* Approximately 60 mA at 3.3 V / 18 mA at 1.8 V */
    GPIO_DRIVING_LEVEL_3 = 3U     /* Approximately 80 mA at 3.3 V / 24 mA at 1.8 V */
} GPIO_Driving_t;
```

5.1.1.3 Functional Modes

As shown above, each I/O pin can operate in general-purpose output, general-purpose input, general-purpose input interrupt, multiplexed-function, or disabled mode. Multiplexed functions include UART, SPI, PWM, and others. Normally only the input or output path operates at a time; for example, input is disabled in general-purpose output mode. In communication functions such as I2C, the input and output paths operate simultaneously.

The API parameters for selecting the mode are as follows:

```
typedef enum {
    HAL_WKIO_MODE_GPIO      = 0U,    /* gpio */
    HAL_WKIO_MODE_HOLD      = 1U,    /* output */
    HAL_WKIO_MODE_WAKE      = 2U,    /* input wake-up */
    HAL_WKIO_MODE_DISA      = 3U     /* disable */
} WKIO_PinMode_t;
```

5.1.1.4 GPIO and Power-Management WKIO

To reduce power, SoCs typically disable power and clocks to some functions during sleep. On some RS-series SoCs, GPIO power is disabled during sleep; WKIO provides a reduced feature set while GPIO is powered down.

WKIO only retains an output level and supports wake-up through an I/O input. Its parameters therefore include pull-resistor and wake-up level/edge settings. Output mode is push-pull at drive level LEVEL0.

Because WKIO precedes GPIO in the signal path, GPIO functions are valid only when WKIO is configured in GPIO mode.

Not every GPIO has a corresponding WKIO. On the RS6240, for example, only Port A has WKIO. See the data sheet for details.

```
typedef enum {
    GPIOx_Pn_F0_INPUT      = 0U,    /* General-purpose I/O input mode */
    GPIOx_Pn_F1_OUTPUT      = 1U,    /* General-purpose I/O output mode */
}
```

```

GPIOx_Pn_F14_EINT          = 14U,    /* General-purpose I/O input-interrupt mode */
GPIOx_Pn_F15_DIS          = 15U,    /* Disabled mode; both input and output are disabled */
/

/* Other multiplexed-function modes */

} GPIO_Mode_t;

```

Because WKIO uses a low-speed clock, configuration changes take effect after a delay determined by the functional clock. With LPCLK_FDIV, the delay is normally up to approximately 30 μ s.

5.1.1.5 Debounce

In GPIO interrupt mode and WKIO wake-up mode (but not in other modes), I/O bounce may cause repeated interrupts or wake-ups. Software debouncing is therefore provided. Each GPIO port and each WKIO port has independent debouncing.

Debouncing samples the input N consecutive times using the functional clock. If all N samples match, the value is passed to the next circuit stage. If the value changes, the new value is recorded and counting restarts. The recorded value is output when the count reaches N. Examples follow.

1. With debounce set to N and low-level triggering selected, an event is triggered after N consecutive low samples. Any high sample restarts the count when the input returns low.
2. With debounce set to N and falling-edge triggering selected, an interrupt or wake-up occurs after N consecutive high samples followed immediately by N consecutive low samples.

5.1.2 Interface

5.1.2.1 HAL_GPIO_Init

Table 5-1 HAL_GPIO_Init

Interface	Description
Function	HAL_Dev_t * HAL_GPIO_Init(GPIO_Port_t id);
Description	Initializes a GPIO port, allocates system resources, and enables its clock.
Parameters	GPIO_Port_t id GPIO peripheral ID. Supported ports depend on the SoC. <pre> typedef enum { #if (CONFIG_SOC_GPIOA == 1) GPIO_PORT_A = 0U, #endif #if (CONFIG_SOC_GPIOB == 1) GPIO_PORT_B = 1U, #endif #if (CONFIG_SOC_GPIOC == 1) GPIO_PORT_C = 2U, #endif GPIO_PORT_NUM, } GPIO_Port_t; </pre>

Interface	Description
Considerations	1. Initialization is idempotent; repeated initialization does not reset GPIO and only returns the control handle. 2. Each call initializes one GPIO port. If the SoC has Port A and Port B, initialize them separately. 3. By default, the system initializes all GPIO ports in Board.c.
Return Value	GPIO control handle, or NULL if initialization fails.

5.1.2.2 HAL_GPIO_DeInit

Table 5-2 HAL_GPIO_DeInit

Interface	Description
Function	<code>void HAL_GPIO_DeInit(HAL_Dev_t *gpio);</code>
Description	Deinitializes GPIO.
Parameters	HAL_Dev_t *gpio GPIO handle.
Considerations	1. Before deinitialization, ensure that no module uses this GPIO port. This API is normally not used.
Return Value	None.

5.1.2.3 HAL_GPIO_SetPinParam

Table 5-3 HAL_GPIO_SetPinParam

Interface	Description
Function	<code>HAL_Status_t HAL_GPIO_SetPinParam(HAL_Dev_t *gpio, GPIO_Pin_t pin, GPIO_PinParam_t *param);</code>
Description	Sets the configuration parameters for a GPIO pin.
Parameters	HAL_Dev_t *gpio GPIO control handle returned by HAL_GPIO_Init or HAL_DEV_Find. GPIO_Pin_t pin GPIO pin value. GPIO_PinParam_t *param Configuration parameters to set <pre>typedef struct { GPIO_Mode_t mode; // Function mode; same as HAL_GPIO_SetPinMode GPIO_Pull_t pull; // Pull-up/pull-down configuration; same as HAL_GPIO_SetPinPull GPIO_Push_t push; // Output-mode configuration; same as HAL_GPIO_SetPinPush GPIO_Driving_t driving; // Output drive-strength configuration; same as HAL_GPIO_SetPinDriving } GPIO_PinParam_t;</pre>

Interface	Description
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.4 HAL_GPIO_GetPinParam

Table 5-4 HAL_GPIO_GetPinParam

Interface	Description
Function	HAL_Status_t HAL_GPIO_GetPinParam(HAL_Dev_t *gpio, GPIO_Pin_t pin, GPIO_PinParam_t *param);
Description	Obtains the configuration parameters for a GPIO pin.
Parameters	HAL_Dev_t *gpio GPIO control handle returned by HAL_GPIO_Init or HAL_DEV_Find. GPIO_Pin_t pin GPIO pin value. GPIO_PinParam_t *param Configuration parameters to set <pre>typedef struct { GPIO_Mode_t mode; // Function mode; same as HAL_GPIO_SetPinMode GPIO_Pull_t pull; // Pull-up/pull-down configuration; same as HAL_GPIO_SetPinPull GPIO_Push_t push; // Output-mode configuration; same as HAL_GPIO_SetPinPush GPIO_Driving_t driving; // Output drive-strength configuration; same as HAL_GPIO_SetPinDriving } GPIO_PinParam_t;</pre>
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.5 HAL_GPIO_DisablePin

Table 5-5 HAL_GPIO_DisablePin

Interface	Description
Function	HAL_Status_t HAL_GPIO_DisablePin(HAL_Dev_t *gpio, GPIO_Pin_t pin);
Description	Disables the specified GPIO pin.
Parameters	HAL_Dev_t *gpio GPIO control handle returned by HAL_GPIO_Init or HAL_DEV_Find.

Interface	Description
	GPIO_Pin_t pin GPIO pin value.
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.6 HAL_GPIO_SetPinMode

Table 5-6 HAL_GPIO_SetPinMode

Interface	Description
Function	HAL_Status_t HAL_GPIO_SetPinMode(HAL_Dev_t *gpio, GPIO_Pin_t pin, GPIO_Mode_t mode);
Description	Sets the functional mode of a GPIO pin.
Parameters	HAL_Dev_t *gpio GPIO control handle returned by HAL_GPIO_Init or HAL_DEV_Find. GPIO_Pin_t pin GPIO pin value. GPIO_Mode_t mode <pre>typedef enum { GPIOx_Pn_F0_INPUT = 0U, // General-purpose input mode GPIOx_Pn_F1_OUTPUT = 1U, // General-purpose output mode GPIOx_Pn_F14_EINT = 14U, // interrupt mode GPIOx_Pn_F15_DIS = 15U, // disabled mode /* Other multiplexed functions */ } GPIO_Mode_t;</pre>
Considerations	1. In disabled mode, the input path, output path, pull-up, and pull-down are all disabled. 2. The pin's GPIO mode is valid only when WKIO is in HAL_WKIO_MODE_GPIO. See HAL_WKIO_SetWakeParam.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.7 HAL_GPIO_SetPinPull

Table 5-7 HAL_GPIO_SetPinPull

Interface	Description
Function	HAL_Status_t HAL_GPIO_SetPinPull(HAL_Dev_t *gpio, GPIO_Pin_t pin, GPIO_Pull_t pull);
Description	Configure the pull-up or pull-down resistor for a GPIO pin.

Interface	Description
Parameters	HAL_Dev_t *gpio GPIO control handle returned by HAL_GPIO_Init or HAL_DEV_Find. GPIO_Pin_t pin GPIO pin value. GPIO_Pull_t pull typedef enum { GPIO_PULL_FLOATING = 0U, // Both pull-up and pull-down disabled /* 35Kohm ~ 75Kohm (3.3V) / 70K ohm ~150K ohm (1.8V) */ GPIO_PULL_UP = 1U, // Enable the pull-up resistor and disable the pull-down resistor /* 30Kohm ~ 75Kohm (3.3V) / 60K ohm ~150K ohm (1.8V) */ GPIO_PULL_DOWN = 2U // Enable the pull-down resistor and disable the pull-up resistor } GPIO_Pull_t;
Considerations	1. Pull resistors are normally configured for inputs. A floating input may be susceptible to interference because its state is undefined. 2. A pull-up can also be configured for open-drain output. If the internal pull-up resistance does not meet communication requirements, add an external pull-up resistor.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.8 HAL_GPIO_SetPinPush

Table 5-8 HAL_GPIO_SetPinPush

Interface	Description
Function	HAL_Status_t HAL_GPIO_SetPinPush(HAL_Dev_t *gpio, GPIO_Pin_t pin, GPIO_Push_t push);
Description	Sets the output mode of a GPIO pin.
Parameters	HAL_Dev_t *gpio GPIO control handle returned by HAL_GPIO_Init or HAL_DEV_Find. GPIO_Pin_t pin GPIO pin value. GPIO_Push_t push typedef enum { GPIO_PUSH_PULL = 0U, // push-pull output GPIO_OPEN_DRAIN = 1U // open-drain output } GPIO_Push_t;
Considerations	1. Open-drain output is commonly used for the I2C wired-AND function.

Interface	Description
	2. Open-drain output can also be used for high-current drive.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.9 HAL_GPIO_SetPinDriving

Table 5-9 HAL_GPIO_SetPinDriving

Interface	Description
Function	HAL_Status_t HAL_GPIO_SetPinDriving(HAL_Dev_t *gpio, GPIO_Pin_t pin, GPIO_Driving_t driving);
Description	Sets the output drive capability of a GPIO pin.
Parameters	HAL_Dev_t *gpio GPIO control handle returned by HAL_GPIO_Init or HAL_DEV_Find. GPIO_Pin_t pin GPIO pin value. GPIO_Driving_t driving Drive-capability configuration. <pre>typedef enum { GPIO_DRIVING_LEVEL_0 = 0U, /* 20mA(3.3V) / 6mA(1.8V) */ GPIO_DRIVING_LEVEL_1 = 1U, /* 40mA(3.3V) / 12mA(1.8V) */ GPIO_DRIVING_LEVEL_2 = 2U, /* 60mA(3.3V) / 18mA(1.8V) */ GPIO_DRIVING_LEVEL_3 = 3U, /* 80mA(3.3V) / 24mA(1.8V) */ } GPIO_Driving_t;</pre>
Considerations	1. Drive-strength settings apply in push-pull output mode. 2. Output drive strength affects I/O signal settling time. Increase the drive strength at high speeds; for example, LEVEL_1 can be used when SPI exceeds 64 MHz.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.10 HAL_GPIO_ReadPort

Table 5-10 HAL_GPIO_ReadPort

Interface	Description
Function	int32_t HAL_GPIO_ReadPort(HAL_Dev_t *gpio);
Description	Obtains the input levels of all GPIO pins.
Parameters	HAL_Dev_t *gpio GPIO control handle returned by HAL_GPIO_Init or HAL_DEV_Find.

Interface	Description
Considerations	If a pin is not configured in input mode, its corresponding bit is 0; this does not mean the pin is low.
Return Value	GPIO input level or state. A return value of 0 indicates that the corresponding I/O is low. A return value of 1 indicates that the corresponding I/O is high. A negative return value indicates retrieval failure. See HAL_STATUS_* for details.

5.1.2.11 HAL_GPIO_WritePort

Table 5-11 HAL_GPIO_WritePort

Interface	Description
Function	HAL_Status_t HAL_GPIO_WritePort(HAL_Dev_t *gpio, uint32_t pinMask);
Description	Write output levels to all GPIO pins.
Parameters	HAL_Dev_t *gpio GPIO control handle returned by HAL_GPIO_Init or HAL_DEV_Find. uint32_t pinMask GPIO output-level value. A bit value of 1 outputs high; 0 outputs low.
Considerations	Effective only for pins configured in output mode. In other modes, neither 0 nor 1 has any effect.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.12 HAL_GPIO_ReadPin

Table 5-12 HAL_GPIO_ReadPin

Interface	Description
Function	int32_t HAL_GPIO_ReadPin(HAL_Dev_t *gpio, GPIO_Pin_t pin);
Description	Obtains the input level of the specified pin.
Parameters	HAL_Dev_t *gpio GPIO control handle returned by HAL_GPIO_Init or HAL_DEV_Find. GPIO_Pin_t pin GPIO pin value.
Considerations	Use together with the pull-resistor API HAL_GPIO_SetPinPull.
Return Value	A negative return value indicates that the function failed. See HAL_STATUS_* for details. A return value of 0 indicates that the corresponding pin is low. A return value of 1 indicates that the corresponding pin is high.

5.1.2.13 HAL_GPIO_WritePin

Table 5-13 HAL_GPIO_WritePin

Interface	Description
Function	HAL_Status_t HAL_GPIO_WritePin(HAL_Dev_t *gpio, GPIO_Pin_t pin, GPIO_PinState_t PinState);
Description	Outputs the specified level on the corresponding pin.
Parameters	HAL_Dev_t *gpio GPIO control handle returned by HAL_GPIO_Init or HAL_DEV_Find. GPIO_Pin_t pin GPIO pin value. GPIO_PinState_t PinState typedef enum { GPIO_PIN_RESET = 0U, // Output low GPIO_PIN_SET // Output high } GPIO_PinState_t;
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.14 HAL_GPIO_TogglePin

Table 5-14 HAL_GPIO_TogglePin

Interface	Description
Function	HAL_Status_t HAL_GPIO_TogglePin(HAL_Dev_t *gpio, GPIO_Pin_t pin);
Description	Toggle the output level of the corresponding pin.
Parameters	HAL_Dev_t *gpio GPIO control handle returned by HAL_GPIO_Init or HAL_DEV_Find. GPIO_Pin_t pin GPIO pin value.
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.15 HAL_GPIO_SetIRQParam

Table 5-15 HAL_GPIO_SetIRQParam

Interface	Description
Function	HAL_Status_t HAL_GPIO_SetIRQParam(HAL_Dev_t *gpio, GPIO_Pin_t pin, GPIO_IrqParam_t *param);

Interface	Description
Description	Sets the interrupt-mode parameters for the specified pin.
Parameters	HAL_Dev_t *gpio GPIO control handle returned by HAL_GPIO_Init or HAL_DEV_Find. GPIO_Pin_t pin GPIO pin value. GPIO_IrqParam_t *param Interrupt parameter configuration <pre>typedef struct { GPIO_IrqEvent_t event; // Interrupt trigger mode GPIO_IrqCallback_t callback; //Interrupt callback void *arg; //Interrupt argument passed to the callback } GPIO_IrqParam_t;</pre> <pre>typedef enum { GPIO_IRQ_EVT_RISING_EDGE = 0U, // Rising-edge trigger GPIO_IRQ_EVT_FALLING_EDGE = 1U, // Falling-edge trigger GPIO_IRQ_EVT_BOTH_EDGE = 2U, // Both-edge trigger GPIO_IRQ_EVT_HIGH_LEVEL = 3U, // High-level trigger GPIO_IRQ_EVT_LOW_LEVEL = 4U, // Low-level trigger } GPIO_IrqEvent_t;</pre> HAL_Dev_t * HAL_GPIO_Init(GPIO_Port_t id); // Callback function type
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.16 HAL_GPIO_SetIRQDebounce

Table 5-16 HAL_GPIO_SetIRQDebounce

Interface	Description
Function	HAL_Status_t HAL_GPIO_SetIRQDebounce(HAL_Dev_t *gpio, uint32_t debounce);
Description	Set the interrupt-mode debounce value for all pins in the port.
Parameters	uint32_t debounce Debounce count; the sampling clock is the GPIO functional clock. The default GPIO functional clock is AHB and is configured in HAL_GPIO_Init.
Considerations	Debounce time is determined by the functional-clock frequency and Debounce value. For example, 32 MHz and 32000000 produce a debounce time of 1 second.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.17 HAL_GPIO_EnableIRQ

Table 5-17 HAL_GPIO_EnableIRQ

Interface	Description
Function	HAL_Status_t HAL_GPIO_EnableIRQ(HAL_Dev_t *gpio, GPIO_Pin_t pin);
Description	Enable interrupts for the corresponding pin. The trigger is configured by HAL_GPIO_SetIRQParam.
Parameters	HAL_Dev_t *gpio GPIO control handle returned by HAL_GPIO_Init or HAL_DEV_Find. GPIO_Pin_t pin GPIO pin value.
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.18 HAL_GPIO_DisableIRQ

Table 5-18 HAL_GPIO_DisableIRQ

Interface	Description
Function	HAL_Status_t HAL_GPIO_DisableIRQ(HAL_Dev_t *gpio, GPIO_Pin_t pin, int8_t ClearPending);
Description	Disables interrupt mode for the specified GPIO pin.
Parameters	HAL_Dev_t *gpio GPIO control handle returned by HAL_GPIO_Init or HAL_DEV_Find. GPIO_Pin_t pin GPIO pin value. Int8_t ClearPending Whether to clear the pending interrupt flag: 0 = do not clear; 1 = clear.
Considerations	This API only disables the pin interrupt; interrupt-mode configuration is retained.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.19 HAL_WKIO_Init

Table 5-19 HAL_WKIO_Init

Interface	Description
Function	HAL_Dev_t *HAL_WKIO_Init(WKIO_Port_t id);
Description	Initialize the WKIO driver.
Parameters	WKIO_Port_t id Device ID, as shown below. For macros such as CONFIG_SOC_WKIOA, see rsxxx.h , for

Interface	Description
	example rs6240.h . <pre> typedef enum { #if (CONFIG_SOC_WKIOA == 1) WKIO_PORT_A = 0U, #endif #if (CONFIG_SOC_WKIOB == 1) WKIO_PORT_B = 1U, #endif WKIO_PORT_NUM } WKIO_Port_t; </pre>
Considerations	1. Initialization is idempotent; repeated initialization does not reset WKIO and only returns the control handle. 2. Each call initializes one WKIO port. If the SoC has Port A and Port B, initialize them separately. 3. Unlike GPIO, WKIO is not initialized by default.
Return Value	NULL indicates initialization failure. A non-NULL value is the WKIO control handle.

5.1.2.20 HAL_WKIO_DeInit

Table 5-20 HAL_WKIO_DeInit

Interface	Description
Function	<code>void HAL_WKIO_DeInit(HAL_Dev_t *wkio);</code>
Description	Deinitialize the WKIO driver.
Parameters	HAL_Dev_t *wkio Wake-up I/O control handle returned by HAL_WKIO_Init or HAL_DEV_Find.
Considerations	Deinitialization is idempotent; repeated calls have no effect.
Return Value	None

5.1.2.21 HAL_WKIO_SetPinMode

Table 5-21 HAL_WKIO_SetPinMode

Interface	Description
Function	<code>HAL_Status_t HAL_WKIO_SetPinMode(HAL_Dev_t *wkio, WKIO_Pin_t pin, WKIO_PinMode_t mode);</code>
Description	Sets the operating mode of the specified WKIO pin.
Parameters	HAL_Dev_t *wkio WKIO control handle returned by HAL_WKIO_Init or HAL_DEV_Find. WKIO_Pin_t pin WKIO pin value. WKIO_PinMode_t mode

Interface	Description
	Operating mode to set: <pre>typedef enum { HAL_WKIO_MODE_GPIO = 0U, // General-purpose I/O mode (GPIO) HAL_WKIO_MODE_HOLD = 1U, //Output-hold mode; retains the level during SoC sleep HAL_WKIO_MODE_WAKE = 2U, //Input wake-up mode; wakes the SoC from sleep HAL_WKIO_MODE_DISA = 3U //I/O fully disabled } WKIO_PinMode_t;</pre>
Considerations	The pin's GPIO configuration is valid only when mode is HAL_WKIO_MODE_GPIO.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.22 HAL_WKIO_GetPinMode

Table 5-22 HAL_WKIO_GetPinMode

Interface	Description
Function	HAL_Status_t HAL_WKIO_GetPinMode(HAL_Dev_t *wkio, WKIO_Pin_t pin, WKIO_PinMode_t *mode);
Description	Obtains the operating mode of the specified WKIO pin.
Parameters	HAL_Dev_t *wkio WKIO control handle returned by HAL_WKIO_Init or HAL_DEV_Find. WKIO_Pin_t pin WKIO pin value. WKIO_PinMode_t mode Pin operating mode to obtain: <pre>typedef enum { HAL_WKIO_MODE_GPIO = 0U, // General-purpose I/O mode (GPIO) HAL_WKIO_MODE_HOLD = 1U, //Output-hold mode; retains the level during SoC sleep HAL_WKIO_MODE_WAKE = 2U, //Input wake-up mode; wakes the SoC from sleep HAL_WKIO_MODE_DISA = 3U //I/O fully disabled } WKIO_PinMode_t;</pre>
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.23 HAL_WKIO_SetPinPull

Table 5-23 HAL_WKIO_SetPinPull

Interface	Description
Function	HAL_Status_t HAL_WKIO_SetPinPull(HAL_Dev_t *wkio, WKIO_Pin_t pin, WKIO_PinPull_t pull);
Description	Obtains the pull-up or pull-down resistor of the specified WKIO pin.
Parameters	HAL_Dev_t *wkio WKIO control handle returned by HAL_WKIO_Init or HAL_DEV_Find. WKIO_Pin_t pin WKIO pin value. WKIO_PinPull_t pull Configure the pin pull resistors: typedef enum { HAL_WKIO_PULL_FLOATING = 0U, // Both pull-up and pull-down disabled /* 35Kohm ~ 75Kohm (3.3V) / 70K ohm ~150K ohm (1.8V) */ HAL_WKIO_PULL_UP = 1U, // Enable the pull-up resistor and disable the pull-down resistor /* 30Kohm ~ 75Kohm (3.3V) / 60K ohm ~150K ohm (1.8V) */ HAL_WKIO_PULL_DOWN = 2U // Disable the pull-down resistor and enable the pull-up resistor } WKIO_PinPull_t;
Considerations	Select pull resistors according to the function. For push-pull output, disable both pull resistors.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.24 HAL_WKIO_GetPinPull

Table 5-24 HAL_WKIO_GetPinPull

Interface	Description
Function	HAL_Status_t HAL_WKIO_GetPinPull(HAL_Dev_t *wkio, WKIO_Pin_t pin, WKIO_PinPull_t *pull);
Description	Obtains the pull-up/pull-down configuration of the specified WKIO pin.
Parameters	HAL_Dev_t *wkio WKIO control handle returned by HAL_WKIO_Init or HAL_DEV_Find. WKIO_Pin_t pin WKIO pin value. WKIO_PinPull_t pull Pin pull-up/pull-down configuration to obtain:

Interface	Description
	<pre>typedef enum { HAL_WKIO_PULL_FLOATING = 0U, // Both pull-up and pull-down disabled /* 35Kohm ~ 75Kohm (3.3V) / 70K ohm ~150K ohm (1.8V) */ HAL_WKIO_PULL_UP = 1U, // Enable the pull-up resistor and disable the pull-down resistor /* 30Kohm ~ 75Kohm (3.3V) / 60K ohm ~150K ohm (1.8V) */ HAL_WKIO_PULL_DOWN = 2U // Disable the pull-down resistor and enable the pull-up resistor } WKIO_PinPull_t;</pre>
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.25 HAL_WKIO_WritePin

Table 5-25 HAL_WKIO_WritePin

Interface	Description
Function	HAL_Status_t HAL_WKIO_WritePin(HAL_Dev_t *wkio, WKIO_Pin_t pin, WKIO_PinState_t PinState);
Description	Write the output level of a WKIO pin.
Parameters	HAL_Dev_t *wkio WKIO control handle returned by HAL_WKIO_Init or HAL_DEV_Find. WKIO_Pin_t pin WKIO pin value. WKIO_PinState_t PinState Set the level: <pre>typedef enum { WKIO_PIN_RESET = 0U, // Output Logic Low WKIO_PIN_SET // Output high } WKIO_PinState_t;</pre>
Considerations	This interface may have a delay of up to one functional-clock cycle. At a 1 MHz WKIO functional clock, for example, the delay is 1 μ s.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.26 HAL_WKIO_WritePort

Table 5-26 HAL_WKIO_WritePort

Interface	Description
-----------	-------------

Interface	Description
Function	HAL_Status_t HAL_WKIO_WritePort(HAL_Dev_t *wkio, uint32_t pin_mask);
Description	Write output levels to all WKIO pins.
Parameters	HAL_Dev_t *wkio WKIO control handle returned by HAL_WKIO_Init or HAL_DEV_Find. uint32_t pin_mask WKIO output-level value. A bit value of 1 outputs high; 0 outputs low.
Considerations	This interface may have a delay of up to one functional-clock cycle. At a 1 MHz WKIO functional clock, for example, the delay is 1 μ s.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.27 HAL_WKIO_SetWakeParam

Table 5-27 HAL_WKIO_SetWakeParam

Interface	Description
Function	HAL_Status_t HAL_WKIO_SetWakeParam(HAL_Dev_t *wkio, WKIO_Pin_t pin, WKIO_WakeParam_t *param);
Description	Sets the wake-up parameters of the specified WKIO pin.
Parameters	HAL_Dev_t *wkio WKIO control handle returned by HAL_WKIO_Init or HAL_DEV_Find. WKIO_Pin_t pin WKIO pin value. WKIO_PinState_t PinState Set the level: <pre>typedef struct { WKIO_WakeEvent_t event; // Wake-up event type WKIO_PinPull_t pull; // Pull-up/pull-down configuration WKIO_WakeCallback_t callback; // wake-up callback void *arg; // Wake-up callback argument } WKIO_WakeParam_t;</pre>
Considerations	For the event parameter, see HAL_WKIO_SetWakeEvent. For the pull parameter, see HAL_WKIO_SetPinPull.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.28 HAL_WKIO_EnableWake

Table 5-28 HAL_WKIO_EnableWake

Interface	Description
-----------	-------------

Interface	Description
Function	HAL_Status_t HAL_WKIO_EnableWake(HAL_Dev_t *wkio);
Description	Enables WKIO wake-up mode.
Parameters	HAL_Dev_t *wkio Wake-up I/O control handle returned by HAL_WKIO_Init or HAL_DEV_Find.
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.29 HAL_WKIO_DisableWake

Table 5-29 HAL_WKIO_DisableWake

Interface	Description
Function	HAL_Status_t HAL_WKIO_DisableWake(HAL_Dev_t *wkio);
Description	Disables WKIO wake-up mode.
Parameters	HAL_Dev_t *wkio Wake-up I/O control handle returned by HAL_WKIO_Init or HAL_DEV_Find.
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.30 HAL_WKIO_SetWakeEvent

Table 5-30 HAL_WKIO_SetWakeEvent

Interface	Description
Function	HAL_Status_t HAL_WKIO_SetWakeEvent(HAL_Dev_t *wkio, WKIO_Pin_t pin, WKIO_WakeEvent_t waketype);
Description	Set the wake-up event type for a WKIO pin.
Parameters	HAL_Dev_t *wkio Wake-up I/O control handle returned by HAL_WKIO_Init or HAL_DEV_Find. WKIO_Pin_t pin Wake-up I/O pin value. WKIO_WakeEvent_t waketype Wake-up event type, as follows: typedef enum { WKIO_WAKE_EVT_RISING_EDGE = 0U, //Rising-edge trigger WKIO_WAKE_EVT_FALLING_EDGE = 1U, //Falling-edge trigger WKIO_WAKE_EVT_BOTH_EDGE = 2U, //Both-edge trigger WKIO_WAKE_EVT_HIGH_LEVEL = 3U, //High-level trigger WKIO_WAKE_EVT_LOW_LEVEL = 4U, //Low-level trigger

Interface	Description
	} WKIO_WakeEvent_t;
Considerations	Use together with the pull-resistor interface HAL_WKIO_SetPinPull.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.31 HAL_WKIO_GetWakeEvent

Table 5-31 HAL_WKIO_GetWakeEvent

Interface	Description
Function	HAL_Status_t HAL_WKIO_GetWakeEvent(HAL_Dev_t *wkio, WKIO_Pin_t pin, WKIO_WakeEvent_t *waketype);
Description	Get the wake-up event type for a WKIO pin.
Parameters	HAL_Dev_t *wkio Wake-up I/O control handle returned by HAL_WKIO_Init or HAL_DEV_Find. WKIO_Pin_t pin Wake-up I/O pin value. WKIO_WakeEvent_t waketype Obtained wake-up event type, as follows: <pre>typedef enum { WKIO_WAKE_EVT_RISING_EDGE = 0U, //Rising-edge trigger WKIO_WAKE_EVT_FALLING_EDGE = 1U, //Falling-edge trigger WKIO_WAKE_EVT_BOTH_EDGE = 2U, //Both-edge trigger WKIO_WAKE_EVT_HIGH_LEVEL = 3U, //High-level trigger WKIO_WAKE_EVT_LOW_LEVEL = 4U, //Low-level trigger } WKIO_WakeEvent_t;</pre>
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.1.2.32 HAL_WKIO_ReadWakePin

Table 5-32 HAL_WKIO_ReadWakePin

Interface	Description
Function	int32_t HAL_WKIO_ReadWakePin(HAL_Dev_t *wkio, WKIO_Pin_t pin);
Description	Get the pin level in WKIO wake-up mode.
Parameters	HAL_Dev_t *wkio Wake-up I/O control handle returned by HAL_WKIO_Init or HAL_DEV_Find. WKIO_Pin_t pin Wake-up I/O pin value.

Interface	Description
Considerations	None
Return Value	Wake-up I/O level or state. A return value of 0 indicates that the corresponding I/O is low. A return value of 1 indicates that the corresponding I/O is high. A negative return value indicates retrieval failure. See HAL_STATUS_* for details.

5.1.2.33 HAL_WKIO_SetWakeDebounce

Table 5-33 HAL_WKIO_SetWakeDebounce

Interface	Description
Function	HAL_Status_t HAL_WKIO_SetWakeDebounce(HAL_Dev_t *wkio, uint32_t debounce);
Description	Set the wake-up-mode debounce value for all pins in the port.
Parameters	HAL_Dev_t *wkio Wake-up I/O control handle returned by HAL_WKIO_Init or HAL_DEV_Find. uint32_t debounce Debounce count; the sampling clock is the WKIO functional clock. The default WKIO functional clock is LPCLK_FRAC and is configured in HAL_WKIO_Init by CONFIG_WKIO_SRC.
Considerations	Debounce time is determined by the functional-clock frequency and Debounce value. For example, 1 MHz and 1000000 produce a debounce time of 1 second.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.2 UART

5.2.1 Function Overview

5.2.1.1 Features

1. Supports 5-, 6-, 7-, and 8-bit data widths
2. Supports 1 or 2 stop bits
3. Supports no, odd, or even parity
4. Supports baud rates up to 6 MHz
5. Partial support for hardware flow control
6. Supports wake-up by a special sequence
7. TX/RX FIFO: up to 32 bits x 64
8. Supports polling, interrupt, and DMA transfers

5.2.1.2 Usage

The UART HAL APIs provide two ways to implement these features:

1. Integrated APIs: use APIs such as Init, Open, Receive, and Transmit without handling implementation details.

2. Individual APIs: use single-function APIs for more complex or customized behavior, with more attention to implementation details.

The integrated APIs are recommended.

Sample:

[project/driver/uart/uart_poll](#)

[project/driver/uart/uart_dma](#)

[project/driver/uart/uart_interrupt](#)

[project/driver/uart/callback](#)

The integrated-API workflow is as follows:

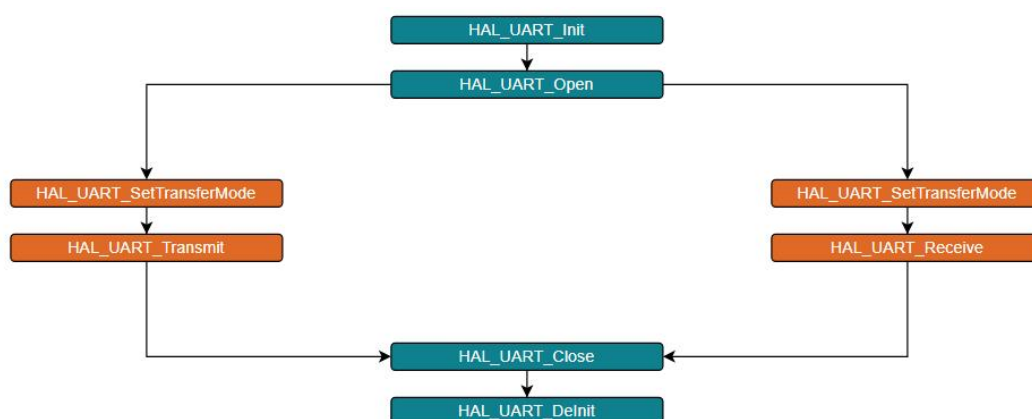


Figure 5-2 UART Integrated-API Workflow

HAL_UART_Init configures the basic UART features as follows.

```

typedef struct {
uint32_t      baudRate;      /* Baud Rate */
UART_DataWidth_t  dataBits;  /* data width */
  UART_StopBit_t    stopBits;  /* Number of stop bits */
  UART_Parity_t     parity;     /* Parity type */
  UART_FlowCtrl_t   autoFlowCtrl; /* Hardware flow control */
} UART_InitParam_t;
  
```

HAL_UART_SetTransferMode selects polling, interrupt, or DMA transfer mode for TX or RX.

5.2.2 Interface

5.2.2.1 HAL_UART_Init

Table 5-34 HAL_UART_Init

Interface	Description
Function	HAL_Dev_t * HAL_UART_Init(UART_ID_t id, const UART_InitParam_t *param)
Description	Initializes the UART device, allocates system resources, and obtains a device control handle.
Parameters	UART_ID_t id UART peripheral ID. typedef enum { UART0_ID = 0U, UART1_ID, }

Interface	Description
	<pre> UART2_ID, UART_NUM, UART_INVALID_ID = 0xFFU } UART_ID_t; const UART_InitParam_t * param Address of the UART initialization-parameter structure. Baud rate; data width; stop-bit width; parity; flow control typedef struct { uint32_t baudRate; UART_DataWidth_t dataBits; UART_StopBit_t stopBits; UART_Parity_t parity; UART_FlowCtrl_t autoFlowCtrl; } UART_InitParam_t; </pre>
Considerations	None
Return Value	UART device instance handle (HAL_Dev_t *).

5.2.2.2 HAL_UART_DeInit

Table 5-35 HAL_UART_DeInit

Interface	Description
Function	HAL_Status_t HAL_UART_DeInit(HAL_Dev_t *uart);
Description	Release the specified UART driver resources.
Parameters	HAL_Dev_t *uart UART device control handle.
Considerations	Call Close before releasing resources.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.2.2.3 HAL_UART_ExtControl

Table 5-36 HAL_UART_ExtControl

Interface	Description
Function	HAL_Status_t HAL_UART_ExtControl(const HAL_Dev_t *uart, UART_ParamType_t ptype, void *param);
Description	Extended control for getting or setting infrequently used parameters.
Parameters	HAL_Dev_t *uart UART device control handle. UART_ParamType_t ptype

Interface	Description
	Parameter type <pre>typedef enum { UART_PARAM_WAKEUP_THRESHOLD = 0, // Wake-up threshold UART_PARAM_WAKEUP_CTRL = 1, // Wake-up enable status UART_PARAM_WAKEUP_CALLBACK = 2, // Wake-up callback UART_PARAM_WAKEUP_MODE = 3, // Wake-up mode: 0 = automatic; 1 = application-initiated } UART_ParamType_t;</pre> void *param Buffer for the parameter to read or set
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.2.2.4 HAL_UART_Open

Table 5-37 HAL_UART_Open

Interface	Description
Function	HAL_Status_t HAL_UART_Open(HAL_Dev_t *uart);
Description	Enables the UART hardware.
Parameters	HAL_Dev_t *uart UART device control handle.
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.2.2.5 HAL_UART_Close

Table 5-38 HAL_UART_Close

Interface	Description
Function	HAL_Status_t HAL_UART_Close(HAL_Dev_t *uart);
Description	Disables the UART hardware.
Parameters	HAL_Dev_t *uart UART device control handle.
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.2.2.6 HAL_UART_SetTransferMode

Table 5-39 HAL_UART_SetTransferMode

Interface	Description
Function	HAL_Status_t HAL_UART_SetTransferMode(const HAL_Dev_t *uart, UART_TransDir_t transDir, UART_TransMode_t Mode);
Description	Sets the transfer mode.
Parameters	HAL_Dev_t *uart UART device control handle. UART_TransDir_t transDir Transfer direction typedef enum { UART_DIR_TX = 0, UART_DIR_RX = 1, } UART_TransDir_t; UART_TransMode_t Mode Transfer mode typedef enum { UART_TRANS_MODE_NOMA = 0, // Polling-mode transfer UART_TRANS_MODE_INTERRUPT = 1, // Interrupt-mode transfer UART_TRANS_MODE_DMA = 2, // DMA transfer } UART_TransMode_t;
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.2.2.7 HAL_UART_Receive

Table 5-40 HAL_UART_Receive

Interface	Description
Function	int32_t HAL_UART_Receive(const HAL_Dev_t *uart, uint8_t *buff, int32_t size, uint32_t msec);
Description	receive data
Parameters	HAL_Dev_t *uart UART device control handle. uint8_t *buff receive data buffer int32_t size expected receive length

Interface	Description
	uint32_t msec data receive timeout
Considerations	None
Return Value	A negative return value is an error code and indicates execution failure. A non-negative return value indicates receive data length.

5.2.2.8 HAL_UART_Transmit

Table 5-41 HAL_UART_Transmit

Interface	Description
Function	<code>int32_t HAL_UART_Transmit(const HAL_Dev_t *uart, const uint8_t *buff, int32_t size, uint32_t msec);</code>
Description	
Parameters	HAL_Dev_t *uart UART device control handle. uint8_t *buff transmit data buffer int32_t size expected transmit length uint32_t msec data transmit timeout
Considerations	None
Return Value	A negative return value is an error code and indicates execution failure. A non-negative return value indicates transmit data length.

5.2.2.9 HAL_UART_GetRxData

Table 5-42 HAL_UART_GetRxData

Interface	Description
Function	<code>uint8_t HAL_UART_GetRxData(const HAL_Dev_t *uart);</code>
Description	Read data from the UART RX FIFO.
Parameters	HAL_Dev_t *uart UART device control handle.
Considerations	None
Return Value	Data read.

5.2.2.10 HAL_UART_PutTxData

Table 5-43 HAL_UART_PutTxData

Interface	Description
Function	void HAL_UART_PutTxData(const HAL_Dev_t *uart, uint8_t data);
Description	Write data to the UART TX FIFO.
Parameters	HAL_Dev_t *uart UART device control handle.
Considerations	None
Return Value	None

5.2.2.11 HAL_UART_IsTxReady

Table 5-44 HAL_UART_IsTxReady

Interface	Description
Function	bool HAL_UART_IsTxReady(const HAL_Dev_t *uart);
Description	Check whether the UART TX FIFO is not full and can accept more data.
Parameters	HAL_Dev_t *uart UART device control handle.
Considerations	None
Return Value	True: the TX FIFO is not full and can accept more data. False: the TX FIFO is full and cannot accept more data.

5.2.2.12 HAL_UART_IsTxEmpty

Table 5-45 HAL_UART_IsTxEmpty

Interface	Description
Function	bool HAL_UART_IsTxEmpty(const HAL_Dev_t *uart);
Description	Checks whether the UART TX FIFO is empty.
Parameters	HAL_Dev_t *uart UART device control handle.
Considerations	None
Return Value	True: the TX FIFO is empty and transmission can end. False: the TX FIFO is not empty and still has data to send.

5.2.2.13 HAL_UART_IsRxReady

Table 5-46 HAL_UART_IsRxReady

Interface	Description
Function	bool HAL_UART_IsRxReady(const HAL_Dev_t *uart);
Description	Check whether the UART RX FIFO contains data.
Parameters	HAL_Dev_t *uart UART device control handle.

Interface	Description
Considerations	None
Return Value	True: the RX FIFO has received data. False: the RX FIFO contains no data.

5.2.2.14 HAL_UART_RegisterIRQ

Table 5-47 HAL_UART_RegisterIRQ

Interface	Description
Function	HAL_Status_t HAL_UART_RegisterIRQ(HAL_Dev_t *uart, UART_TransDir_t transDir, const HAL_Callback_t *irqCb);
Description	Registers a transfer interrupt callback.
Parameters	HAL_Dev_t *uart UART device control handle. UART_TransDir_t transDir Transfer direction typedef enum { UART_DIR_TX = 0, UART_DIR_RX = 1, } UART_TransDir_t; HAL_Callback_t *irqCb Callback Function typedef struct { HAL_CbFunc_t cb; // callback function pointer void * arg; // Callback input argument } HAL_Callback_t;
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.2.2.15 HAL_UART_UnregisterIRQ

Table 5-48 HAL_UART_UnregisterIRQ

Interface	Description
Function	HAL_Status_t HAL_UART_UnregisterIRQ(const HAL_Dev_t *uart, UART_TransDir_t transDir);
Description	Unregister the interrupt callback for the specified direction.
Parameters	HAL_Dev_t *uart UART device control handle. UART_TransDir_t transDir

Interface	Description
	Transfer direction <pre>typedef enum { UART_DIR_TX = 0, UART_DIR_RX = 1, } UART_TransDir_t;</pre>
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.2.2.16 HAL_UART_EnableTxReadyIRQ

Table 5-49 HAL_UART_EnableTxReadyIRQ

Interface	Description
Function	HAL_Status_t HAL_UART_EnableTxReadyIRQ(const HAL_Dev_t *uart);
Description	Enable the UART TX FIFO-below-threshold interrupt.
Parameters	HAL_Dev_t *uart UART device control handle.
Considerations	An interrupt is triggered when the TX FIFO level falls below the configured threshold.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.2.2.17 HAL_UART_EnableRxReadyIRQ

Table 5-50 HAL_UART_EnableRxReadyIRQ

Interface	Description
Function	HAL_Status_t HAL_UART_EnableRxReadyIRQ(const HAL_Dev_t *uart);
Description	Enable the UART RX FIFO-above-threshold interrupt.
Parameters	HAL_Dev_t *uart UART device control handle.
Considerations	An interrupt is triggered when the RX FIFO level rises above the configured threshold.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.2.2.18 HAL_UART_DisableTxReadyIRQ

Table 5-51 HAL_UART_DisableTxReadyIRQ

Interface	Description
Function	HAL_Status_t HAL_UART_DisableTxReadyIRQ(const HAL_Dev_t *uart);
Description	Disable the UART TX FIFO-below-threshold interrupt.

Interface	Description
Parameters	HAL_Dev_t *uart UART device control handle.
Considerations	An interrupt is triggered when the TX FIFO level falls below the configured threshold.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.2.2.19 HAL_UART_DisableRxReadyIRQ

Table 5-52 HAL_UART_DisableRxReadyIRQ

Interface	Description
Function	HAL_Status_t HAL_UART_DisableRxReadyIRQ(const HAL_Dev_t *uart);
Description	Disable the UART RX FIFO-above-threshold interrupt.
Parameters	HAL_Dev_t *uart UART device control handle.
Considerations	An interrupt is triggered when the RX FIFO level rises above the configured threshold.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.2.2.20 HAL_UART_RegisterDMA

Table 5-53 HAL_UART_RegisterDMA

Interface	Description
Function	HAL_Status_t HAL_UART_RegisterDMA(HAL_Dev_t *uart, UART_TransDir_t transDir, HAL_Callback_t *dmaCb);
Description	Register the DMA transfer-complete callback.
Parameters	HAL_Dev_t *uart UART device control handle. UART_TransDir_t transDir Transfer direction typedef enum { UART_DIR_TX = 0, UART_DIR_RX = 1, } UART_TransDir_t; HAL_Callback_t *irqCb Callback Function typedef struct { HAL_CbFunc_t cb; // callback function pointer void * arg; // Callback input argument } HAL_Callback_t;

Interface	Description
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.2.2.21 HAL_UART_UnregisterDMA

Table 5-54 HAL_UART_UnregisterDMA

Interface	Description
Function	HAL_Status_t HAL_UART_UnregisterDMA(const HAL_Dev_t *uart, UART_TransDir_t transDir);
Description	Unregister the DMA transfer-complete interrupt callback.
Parameters	HAL_Dev_t *uart UART device control handle. UART_TransDir_t transDir Transfer direction typedef enum { UART_DIR_TX = 0, UART_DIR_RX = 1, } UART_TransDir_t;
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.2.2.22 HAL_UART_StartTransmitDMA

Table 5-55 HAL_UART_StartTransmitDMA

Interface	Description
Function	HAL_Status_t HAL_UART_StartTransmitDMA(const HAL_Dev_t *uart, const uint8_t *buf, int32_t size);
Description	Starts DMA transmission.
Parameters	HAL_Dev_t *uart UART device control handle. const uint8_t *buf transmit data buffer int32_t size transmit data length
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the

Interface	Description
	failure type.

5.2.2.23 HAL_UART_StopTransmitDMA

Table 5-56 HAL_UART_StopTransmitDMA

Interface	Description
Function	int32_t HAL_UART_StopTransmitDMA(const HAL_Dev_t *uart);
Description	Stops DMA transmission.
Parameters	HAL_Dev_t *uart UART device control handle.
Considerations	None
Return Value	A negative value is an error code; a positive value is the number of bytes transmitted.

5.2.2.24 HAL_UART_StartReceiveDMA

Table 5-57 HAL_UART_StartReceiveDMA

Interface	Description
Function	HAL_Status_t HAL_UART_StartReceiveDMA(const HAL_Dev_t *uart, uint8_t *buf, int32_t size);
Description	Starts receiving data through DMA.
Parameters	HAL_Dev_t *uart UART device control handle. const uint8_t *buf receive data buffer int32_t size receive data length
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.2.2.25 HAL_UART_StopReceiveDMA

Table 5-58 HAL_UART_StopReceiveDMA

Interface	Description
Function	int32_t HAL_UART_StopReceiveDMA(const HAL_Dev_t *uart);
Description	Stops DMA data reception.
Parameters	HAL_Dev_t *uart UART device control handle.
Considerations	None
Return Value	A negative value is an error code; a positive value is the number of bytes received.

5.3 SPI

5.3.1 Function Overview

5.3.1.1 Features

1. Supports standard, dual, and quad transfers
2. Supports half-duplex and full-duplex transfers
3. Supports master and slave modes
4. Supports multiple masters and multiple slaves
5. Supports 8-, 16-, and 32-bit transfers
6. Supports communication frequencies up to 100 MHz
7. Supports MSB-first transmission and reception
8. Supports software-controlled or automatic hardware CS
9. TX/RX FIFO: up to 32 bits x 32
10. Supports polling, interrupt, and DMA transfers

SPI communication uses the following pins. See the applicable SoC data sheet for each PinNumber.

Table 5-59 SPI Pin Functions

Pin Name	Description
CLK	Clock signal, which determines the SPI operating frequency. 1. Master output, slave input 2. Provided by the master; the slave transmits and receives according to this clock. 3. Data can be sampled on the rising or falling edge.
CS(NSS)	Slave-select signal, active low. 1. Master output, slave input 2. Can be controlled automatically by hardware or by software. For software control, the master uses HAL_SPI_EnableCS/HAL_SPI_DisableCS and the slave uses HAL_SPI_IsCsi nterrupt. 3. A master can output multiple CS signals to control multiple slaves.
MOSI (Q0)	Master Output Slave Input: master data output and slave data input. 1. Can operate simultaneously with MISO for full-duplex transfer. 2. In standard SPI, this pin is an output on the master and an input on the slave. In dual or quad SPI, it is bidirectional. 3. In dual SPI, this pin and MISO form a 2-bit data bus; this pin is the low-order bit. 4. In quad SPI, this pin forms a 4-bit data bus with MISO, WP, and HOLD; this pin is the least-significant bit.
MISO (Q1)	Like MOSI: in standard SPI, it is the slave data output; in dual or quad SPI, it is the second data bit.
WP (Q2)	Same as MOSI; valid in quad SPI mode.
HOLD (Q3)	Same as MOSI; valid in quad SPI mode.

Both SPI endpoints must select the same clock phase and polarity, corresponding to Mode 0 through Mode 3:

1. Mode 0: CPOL = 0, CPHA = 0. The receiver samples data on the first rising CLK edge after CS goes low.
2. Mode 1: CPOL = 0, CPHA = 1. The receiver samples data on the second falling CLK edge after CS goes low.

3. Mode 2: CPOL = 1, CPHA = 0. The receiver samples data on the first falling CLK edge after CS goes low.
4. Mode 3: CPOL = 1, CPHA = 1. The receiver samples data on the second rising CLK edge after CS goes low.

To keep the data signal stable at the receiver's sampling edge, the transmitter must change the data a half a cycle earlier.

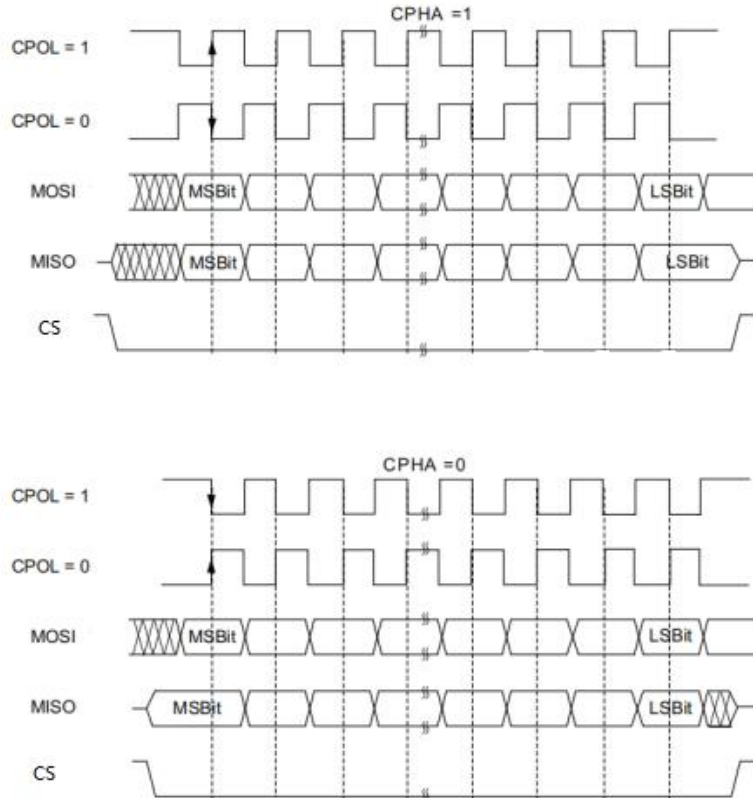


Figure 5-3 SPI Mode Diagram

5.3.1.2 Usage

The SPI HAL APIs provide two ways to implement these features:

1. Integrated APIs: use APIs such as Init, Open, Receive, and Transmit for SP transfers without handling implementation details.
2. Individual APIs: use single-function APIs for more complex or customized behavior, with more attention to implementation details.

The integrated APIs are recommended.

Sample:

[project/driver/spi/spi_poll_master](#)
[project/driver/spi/spi_poll_slave](#)
[project/driver/spi/spi_interrupt_master](#)
[project/driver/spi/spi_interrupt_slave](#)
[project/driver/spi/spi_dma_master](#)
[project/driver/spi/spi_dma_slave](#)
[project/driver/spi/spi_dual_master](#)

[project/driver/spi/spi_dual_slave](#)

[project/driver/spi/spi_quad_master](#)

[project/driver/spi/spi_quad_slave](#)

The integrated-API workflow is as follows:

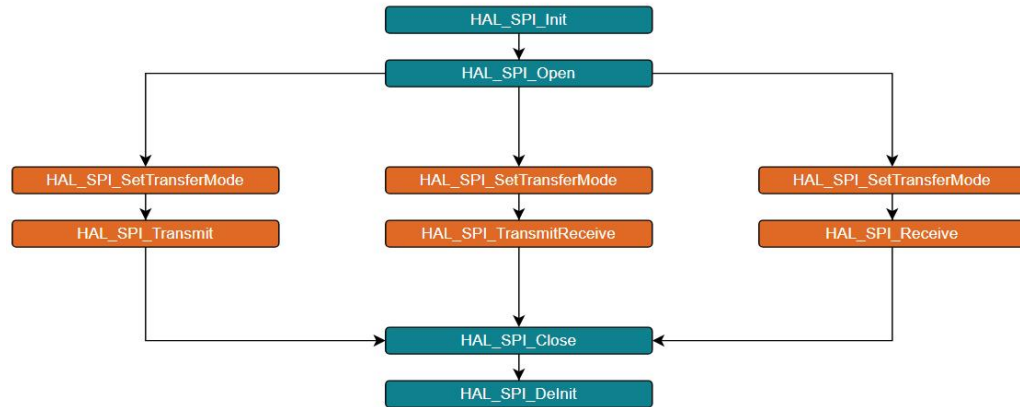


Figure 5-4 SPI Integrated-API Workflow

HAL_SPI_Init configures the SPI features using the following parameters.

```

typedef struct {
    SPI_Mode_t          mode;          /* Master / Slave */
    SPI_SclkMode_t      sclkMode;      /* CPOL and CPHA */
    SPI_IOMode_t        ioMode;        /* Standard, dual, quad */
    SPI_Cs_t            csMode;        /* Software CS or hardware CS */
    SPI_DataWidth_t     dataWidth;     /* Data width: 8, 16, or 32 bits */
    uint32_t            sclk;          /* Operating Frequency */
} SPI_InitParam_t;
  
```

HAL_SPI_Init also configures Pinmux internally. By default, Pinmux settings are centrally defined in the board configuration file (for example, as weak definitions in [mrs6240_p2512.c](#)), as shown below. Definitions in project files can override those in the board configuration file.

```

/* Weak definition of SPI active-mode pin configuration. Configure I/O drive strength according to SPI speed; see the GPIO chapter.*/
DEFINE_PINMUX_WEAK(spi0, 4,
    { GPIO_PORT_A, GPIO_PIN_0, {GPIOA_P0_F2_SPI_CS,    GPIO_DRIVING_LEVEL_0, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* CS */
    { GPIO_PORT_A, GPIO_PIN_1, {GPIOA_P1_F2_SPI_CLK,    GPIO_DRIVING_LEVEL_0, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* CLK */
    { GPIO_PORT_A, GPIO_PIN_2, {GPIOA_P2_F2_SPI_MOSI,    GPIO_DRIVING_LEVEL_0, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* MOSI */
    { GPIO_PORT_A, GPIO_PIN_3, {GPIOA_P3_F2_SPI_MISO,    GPIO_DRIVING_LEVEL_0, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* MISO */
);
/* Weak definition of SPI sleep-mode pin configuration.*/
DEFINE_PINMUX_WEAK(spi0_deinit, 4,
    { GPIO_PORT_A, GPIO_PIN_0, {GPIOx_Pn_F15_DIS,    GPIO_DRIVING_LEVEL_0, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* CS */
    { GPIO_PORT_A, GPIO_PIN_1, {GPIOx_Pn_F15_DIS,    GPIO_DRIVING_LEVEL_0, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* CLK */
    { GPIO_PORT_A, GPIO_PIN_2, {GPIOx_Pn_F15_DIS,    GPIO_DRIVING_LEVEL_0, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* MOSI */
    { GPIO_PORT_A, GPIO_PIN_3, {GPIOx_Pn_F15_DIS,    GPIO_DRIVING_LEVEL_0, GPIO_PUSH_PULL, GPIO_PULL_UP } }, /* MISO */
);
  
```

```
);
```

```
/* A variable with the same name can be defined in a project file to override this configuration. */
const HAL_PinmuxConfig_t pinmux_spi0[] = {
    { GPIO_PORT_A, GPIO_PIN_0, {GPIOA_P0_F2_SPI_CS,    GPIO_DRIVING_LEVEL_3, GPIO_PUSH_PULL, GPIO
_PULL_UP } }, /* CS */
    { GPIO_PORT_A, GPIO_PIN_1, {GPIOA_P1_F2_SPI_CLK,    GPIO_DRIVING_LEVEL_2, GPIO_PUSH_PULL, GPI
O_PULL_UP } }, /* CLK */
    { GPIO_PORT_A, GPIO_PIN_2, {GPIOA_P2_F2_SPI_MOSI,    GPIO_DRIVING_LEVEL_2, GPIO_PUSH_PULL, G
PIO_PULL_UP } }, /* MOSI */
    { GPIO_PORT_A, GPIO_PIN_3, {GPIOA_P3_F2_SPI_MISO,    GPIO_DRIVING_LEVEL_2, GPIO_PUSH_PULL, G
PIO_PULL_UP } }, /* MISO */
    { GPIO_PORT_A, GPIO_PIN_4, {GPIOA_P4_F2_SPI_HOLD,    GPIO_DRIVING_LEVEL_2, GPIO_PUSH_PULL, G
PIO_PULL_UP } }, /* HOLD */
    { GPIO_PORT_A, GPIO_PIN_5, {GPIOA_P5_F2_SPI_WP,     GPIO_DRIVING_LEVEL_2, GPIO_PUSH_PULL, GPI
O_PULL_UP } }, /* WP */
};
```

HAL_SPI_Open enables the SPI hardware. If the SPI master is connected to multiple slaves, the HAL_SPI_Init parameters include the configuration of each slave (param_num specifies the number of slave parameter sets). The slave_addrIndex parameter of HAL_SPI_Open selects the slave with which to communicate.

HAL_SPI_SetTransferMode selects polling, interrupt, or DMA mode for transmission or reception.

HAL_SPI_Receive and HAL_SPI_Transmit perform half-duplex reception and transmission, respectively. HAL_SPI_TransmitReceive performs full-duplex transfers.

When using individual APIs, understand the FIFO configuration. The TX and RX FIFOs have a trigger level that determines when a flag, interrupt, or DMA transfer is triggered.

For interrupt-mode reception, the RX FIFO trigger defaults to 1 so that every received item generates an interrupt and the final data in a frame is read.

For interrupt-mode transmission, the TX FIFO trigger defaults to half of the total depth to prevent overflow.

In DMA mode, the FIFO level must correspond to the DMA FIFO settings. See the DMA chapter.

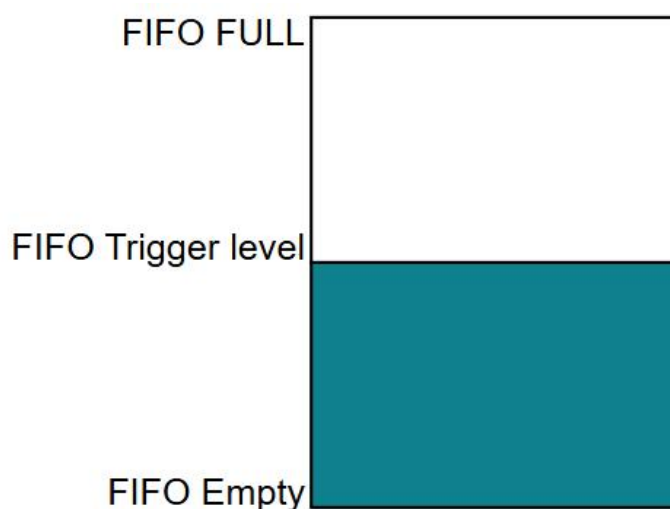


Figure 5-5 SPI FIFO Usage

5.3.2 Mode Description

SPI provides two modes to support different transfer frequencies.

By default, the SDK uses MODE1 for SPI configuration at transfer frequencies up to 30 MHz. Above 30 MHz, MODE1 requires adjustment of master delay, while MODE2 requires DUMMY transfers (sending invalid data). For details on the two modes, see Sections 5.3.2.1 and 5.3.2.2.

The mode is selected automatically based on the user-configured frequency. It can also be configured through `LL_SPI_SlvClkSelect`: select `LL_QSPI_SLVCLK_SSICLK` for MODE1 or `LL_QSPI_SLVCLK_FUNCLK` for MODE2.

The mode-selection procedure is shown below.

```

/* mode 1 or mode 2 set */
tmp_div = LL_SPI_GetPrescaler(spi_instance[temp_handle->spi_id]);
if ((tmp_param->sclk <= 30000000) && (tmp_div > SPI_CLOCK_DEV_4)) {
    /* mode 1 */
    LL_SPI_SlvClkSelect(spi_instance[temp_handle->spi_id], LL_QSPI_SLVCLK_SSICLK);
} else {
    /* mode 2 */
    LL_SPI_SetMode(spi_instance[temp_handle->spi_id], LL_QSPI_MST_MODE);
    LL_SPI_EnableSlvClkIn(spi_instance[temp_handle->spi_id]);
    LL_SPI_SetMode(spi_instance[temp_handle->spi_id], LL_QSPI_SLV_MODE);
    LL_SPI_SlvClkSelect(spi_instance[temp_handle->spi_id], LL_QSPI_SLVCLK_FUNCLK);
    LL_SPI_DisableSlvClkIn(spi_instance[temp_handle->spi_id]);
}

```

Figure 5-6 SPI Mode Selection

5.3.2.1 SPI_MODE1

SPI_MODE1 must operate below 30 MHz, with a clock-divider factor PSC greater than 4. When the system clock is selected as the SPI peripheral clock source, which is the default, $PSC = \text{system clock} / \text{SPI transfer rate}$. To use SPI_MODE1 at high speed, the divider factor must be greater than 5; a larger value is preferable.

The reason for adjusting master delay when SPI_MODE1 operates above 30 MHz is as follows:

When $CPOL = 0$ and $CPHA = 0$, the SPI hardware changes the data signal on the falling edge of CLK and samples it on the rising edge. The data transition must therefore complete before the next rising edge of CLK. The available timing margin is only half a CLK cycle. At 60 MHz, this margin is approximately 8.3 ns, while the measured port delay at the chip's slave end alone is approximately 6-10 ns. Reliable sampling is therefore almost impossible if the master samples using its original transmitted CLK.

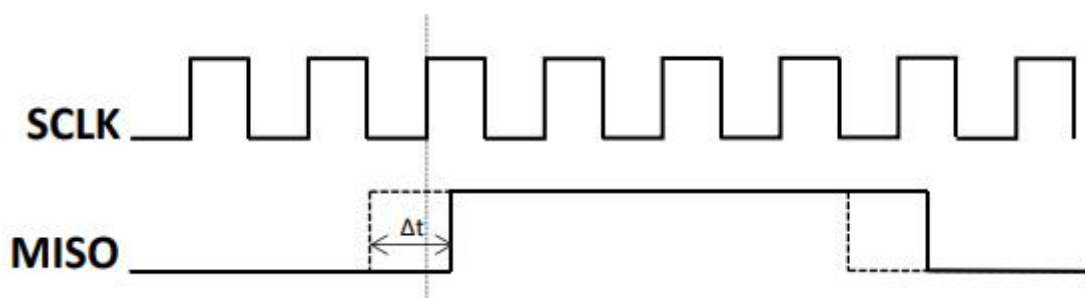


Figure 5-7 SPI Rising-Edge Sampling Error Caused by Delay

The SPI master normally solves this problem through delayed sampling. Instead of the original CLK, it samples using `CLK_0`, which is delayed relative to CLK by adjusting master delay. The data and clock delays then compensate for each other, allowing SPI to sample the data correctly.

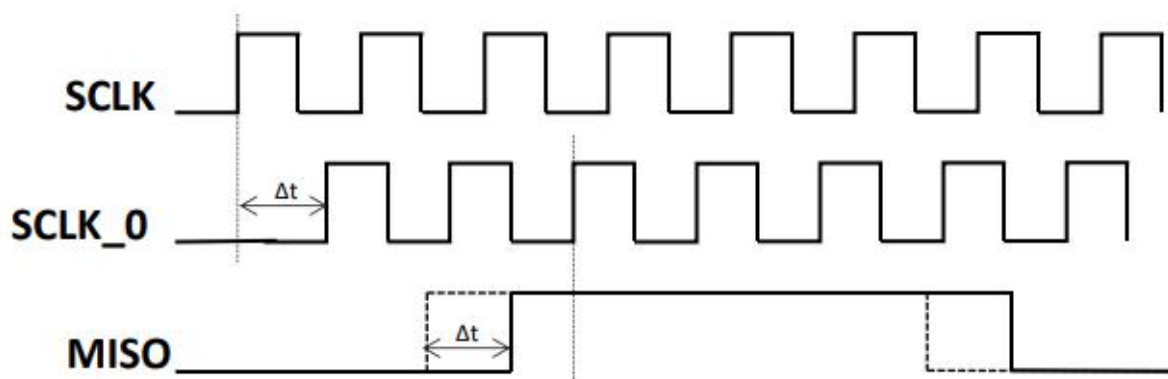


Figure 5-8 SPI Data Sampling Using CLK_0

5.3.2.2 SPI_MODE2

SPI_MODE2 has no clock-divider requirement, but the hardware design requires DUMMY transfers. The reason SPI_MODE2 requires DUMMY transfers (sending invalid data) is as follows:

In the MODE2 hardware design, data is popped (read) from the FIFO in two cases. The first is on the falling edge of CS, to prepare data before it is read on the first rising edge. The second is at the end of each transfer, to update the FIFO automatically and ensure that the next transfer proceeds without interruption.

When a data set is split across multiple CS assertions, duplicate pop operations occur: one at the end of each transfer and another when CS is driven low again. Subsequent data is popped before it is transmitted, resulting in data loss. DUMMY transfers (sending invalid data) are therefore required to complete the SPI transfer.

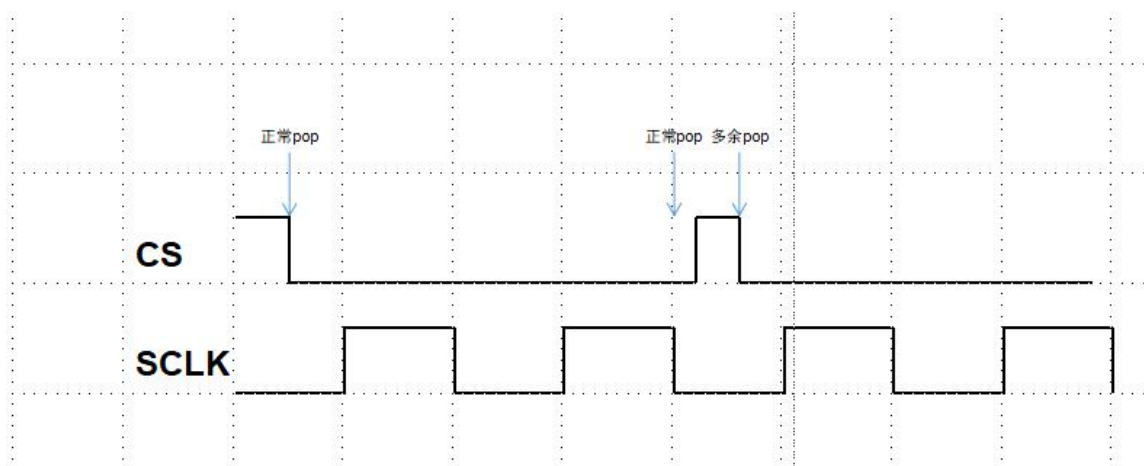


Figure 5-9 FIFO Pop Operations During SPI Transfers

5.3.3 Interface

5.3.3.1 HAL_SPI_Init

Table 5-60 HAL_SPI_Init

Interface	Description
Function	HAL_Dev_t * HAL_SPI_Init(SPI_ID_t id, const SPI_InitParam_t *param, const uint8_t param_num);
Description	Initialize an SPI device, allocate system resources, and obtain a device control handle.
Parameters	SPI_ID_t id SPI peripheral ID. const SPI_InitParam_t * param Address of the SPI initialization-parameter structure. <pre>typedef struct { SPI_Mode_t mode; // MASTER or SLAVE SPI_SclkMode_t sclkMode; // Clock mode 0-3 SPI_IOMode_t ioMode; // SPI mode: standard/dual/quad SPI_Cs_t csMode; // CS control mode: hardware or software SPI_DataWidth_t dataWidth; // data width uint32_t sclk; // clock frequency } SPI_InitParam_t;</pre>
Considerations	None
Return Value	UART device control handle (HAL_Dev_t *).

5.3.3.2 HAL_SPI_DeInit

Table 5-61 HAL_SPI_DeInit

Interface	Description
Function	HAL_Status_t HAL_SPI_DeInit(HAL_Dev_t *spi);
Description	Release the specified SPI driver resources.
Parameters	HAL_Dev_t *spi SPI device control handle.
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the

Interface	Description
	failure type.

5.3.3.3 HAL_SPI_Open

Table 5-62 HAL_SPI_Open

Interface	Description
Function	HAL_Status_t HAL_SPI_Open(HAL_Dev_t *spi, uint8_t slave_addrIndex);
Description	Enables the SPI hardware.
Parameters	HAL_Dev_t *spi SPI device control handle. uint8_t slave_addrIndex Slave-selection index.
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.4 HAL_SPI_Close

Table 5-63 HAL_SPI_Close

Interface	Description
Function	HAL_Status_t HAL_SPI_Close(HAL_Dev_t *spi);
Description	Disables the SPI hardware.
Parameters	HAL_Dev_t *spi SPI device control handle.
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.5 HAL_SPI_ExtControl

Table 5-64 HAL_SPI_ExtControl

Interface	Description
Function	HAL_Status_t HAL_SPI_ExtControl(const HAL_Dev_t *spi, SPI_ParamType_t ptype, void *param);
Description	Extended controls for infrequently used SPI parameters.
Parameters	HAL_Dev_t *spi SPI device control handle. SPI_ParamType_t ptype

Interface	Description
	Configuration parameter type <pre>typedef enum { SPI_PARAM_TX_START_LEVEL = 0, // TX FIFO transmission-start threshold SPI_PARAM_TX_EMPTY_LEVEL = 1, // TX FIFO empty threshold SPI_PARAM_RX_FULL_LEVEL = 2, // RX FIFO full threshold SPI_PARAM_TX_DMA_LEVEL = 3, // TX FIFO DMA trigger threshold SPI_PARAM_RX_DMA_LEVEL = 4, //RX FIFO DMA trigger threshold SPI_PARAM_FRAME_CTRL = 5, //Command format and cmd/addr/data segment settings SPI_PARAM_WAKEUP_THRESHOLD = 6, // Wake-up CLK threshold SPI_PARAM_WAKEUP_CTRL = 7, // Wake-up control SPI_PARAM_WAKEUP_CALLBACK = 8, // wake-up callback SPI_PARAM_WAKEUP_MODE = 9, // wake-up mode } SPI_ParamType_t;</pre> void *param Configuration parameters for reading or writing.
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.6 HAL_SPI_SetTransferMode

Table 5-65 HAL_SPI_SetTransferMode

Interface	Description
Function	<pre>HAL_Status_t HAL_SPI_SetTransferMode(const HAL_Dev_t *spi, SPI_TransDir_t transDir, SPI_TransMode_t Mode);</pre>
Description	Sets the transfer mode.
Parameters	HAL_Dev_t *spi SPI device control handle. SPI_TransDir_t transDir Transfer direction <pre>typedef enum { SPI_DIR_TX = 0, // Transmit only SPI_DIR_RX = 1, // Receive only SPI_DIR_FULL = 2, // Simultaneous transmit and receive } SPI_TransDir_t;</pre> SPI_TransMode_t Mode Transfer mode <pre>typedef enum {</pre>

Interface	Description
	<pre> SPI_TRANS_MODE_NOMA = 0, // polling mode SPI_TRANS_MODE_INTERRUPT = 1, // interrupt mode SPI_TRANS_MODE_DMA = 2, // DMA mode } SPI_TransMode_t; </pre>
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.7 HAL_SPI_Receive

Table 5-66 HAL_SPI_Receive

Interface	Description
Function	<pre> int32_t HAL_SPI_Receive(const HAL_Dev_t *spi, uint8_t *buff, int32_t size, uint32_t msec); </pre>
Description	Receives data through SPI.
Parameters	HAL_Dev_t *spi SPI device control handle. uint8_t *buff Receive-data buffer int32_t size Expected receive-data length. uint32_t msec Timeout in ms
Considerations	None
Return Value	A negative value is an error code; a positive value is the number of data items received successfully.

5.3.3.8 HAL_SPI_Transmit

Table 5-67 HAL_SPI_Transmit

Interface	Description
Function	<pre> int32_t HAL_SPI_Transmit(const HAL_Dev_t *spi, const uint8_t *buff, int32_t size, uint32_t msec); </pre>
Description	transmit data
Parameters	HAL_Dev_t *spi

Interface	Description
	SPI device control handle. uint8_t *buff Transmit-data buffer int32_t size Expected transmit-data length. uint32_t msec Timeout in ms
Considerations	None
Return Value	A negative value is an error code; a positive value is the number of data items transmitted successfully.

5.3.3.9 HAL_SPI_TransmitReceive

Table 5-68 HAL_SPI_TransmitReceive

Interface	Description
Function	int32_t HAL_SPI_TransmitReceive(const HAL_Dev_t *spi, uint8_t *tx_data, uint8_t *rx_data, int32_t size, uint32_t msec);
Description	Transfer mode for simultaneous transmission and reception.
Parameters	HAL_Dev_t *spi SPI device control handle. uint8_t *tx_data Transmit-data buffer uint8_t *rx_data Receive-data buffer int32_t size Expected transfer-data length. uint32_t msec Timeout in ms
Considerations	None
Return Value	A negative value is an error code; a positive value is the number of data items transferred successfully.

5.3.3.10 HAL_SPI_GetRxData

Table 5-69 HAL_SPI_GetRxData

Interface	Description
Function	int32_t HAL_SPI_GetRxData(const HAL_Dev_t *spi);
Description	Get received data from the FIFO.
Parameters	HAL_Dev_t *spi SPI device control handle.
Considerations	Use together with HAL_SPI_IsRxReady.
Return Value	Data read from the FIFO.

5.3.3.11 HAL_SPI_PutTxData

Table 5-70 HAL_SPI_PutTxData

Interface	Description
Function	void HAL_SPI_PutTxData(const HAL_Dev_t *spi, uint32_t data);
Description	Write data to the TX FIFO.
Parameters	HAL_Dev_t *spi SPI device control handle. uint32_t data Data to write.
Considerations	None
Return Value	None

5.3.3.12 HAL_SPI_StartTrans

Table 5-71 HAL_SPI_StartTrans

Interface	Description
Function	HAL_Status_t HAL_SPI_StartTrans(const HAL_Dev_t *spi , SPI_TransDir_t dir);
Description	Starts the transfer.
Parameters	HAL_Dev_t *spi SPI device control handle. SPI_TransDir_t dir Transfer direction
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.13 HAL_SPI_StopTrans

Table 5-72 HAL_SPI_StopTrans

Interface	Description
Function	HAL_Status_t HAL_SPI_StopTrans(const HAL_Dev_t *spi);
Description	Stops the transfer.
Parameters	HAL_Dev_t *spi SPI device control handle.
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.14 HAL_SPI_IsTxReady

Table 5-73 HAL_SPI_IsTxReady

Interface	Description
Function	int HAL_SPI_IsTxReady(HAL_Dev_t *spi);
Description	Check whether the TX FIFO exceeds the threshold.
Parameters	HAL_Dev_t *spi SPI device control handle.
Considerations	None
Return Value	0 = exceeded; 1 = not exceeded.

5.3.3.15 HAL_SPI_IsTxEmpty

Table 5-74 HAL_SPI_IsTxEmpty

Interface	Description
Function	int HAL_SPI_IsTxEmpty(HAL_Dev_t *spi);
Description	Checks whether the transmit FIFO is empty.
Parameters	HAL_Dev_t *spi SPI device control handle.
Considerations	None
Return Value	1 = empty; 0 = not empty.

5.3.3.16 HAL_SPI_IsRxReady

Table 5-75 HAL_SPI_IsRxReady

Interface	Description
Function	int HAL_SPI_IsRxReady(HAL_Dev_t *spi);
Description	Check whether the RX FIFO is nonempty.
Parameters	HAL_Dev_t *spi SPI device control handle.

Interface	Description
Considerations	None
Return Value	1 = nonempty; 0 = empty.

5.3.3.17 HAL_SPI_IsBusy

Table 5-76 HAL_SPI_IsBusy

Interface	Description
Function	int HAL_SPI_IsBusy(HAL_Dev_t *spi);
Description	Check whether SPI is transferring data.
Parameters	HAL_Dev_t *spi SPI device control handle.
Considerations	None
Return Value	1 = transferring; 0 = idle.

5.3.3.18 HAL_SPI_GetFifoLevel

Table 5-77 HAL_SPI_GetFifoLevel

Interface	Description
Function	int HAL_SPI_GetFifoLevel(HAL_Dev_t *spi, SPI_FifoType_t type);
Description	Get the FIFO threshold.
Parameters	HAL_Dev_t *spi SPI device control handle. SPI_FifoType_t type FIFO type typedef enum { SPI_FIFO_TX = 0, SPI_FIFO_RX = 1, } SPI_FifoType_t;
Considerations	None
Return Value	Configured FIFO threshold.

5.3.3.19 HAL_SPI_IsCsInterrupt

Table 5-78 HAL_SPI_IsCsInterrupt

Interface	Description
Function	int HAL_SPI_IsCsInterrupt(HAL_Dev_t *spi);
Description	Check whether the slave is selected.
Parameters	HAL_Dev_t *spi SPI device control handle.
Considerations	None

Interface	Description
Return Value	1 = selected; 0 = not selected.

5.3.3.20 HAL_SPI_EnableDMA

Table 5-79 HAL_SPI_EnableDMA

Interface	Description
Function	HAL_Status_t HAL_SPI_EnableDMA(const HAL_Dev_t *spi , SPI_TransDir_t dir);
Description	Enables DMA transfers.
Parameters	HAL_Dev_t *spi SPI device control handle. SPI_TransDir_t dir Transfer direction
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.21 HAL_SPI_DisableDMA

Table 5-80 HAL_SPI_DisableDMA

Interface	Description
Function	HAL_Status_t HAL_SPI_DisableDMA(const HAL_Dev_t *spi , SPI_TransDir_t dir);
Description	Disables DMA transfers.
Parameters	HAL_Dev_t *spi SPI device control handle. SPI_TransDir_t dir Transfer direction
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.22 HAL_SPI_EnableCS

Table 5-81 HAL_SPI_EnableCS

Interface	Description
Function	HAL_Status_t HAL_SPI_EnableCS(const HAL_Dev_t *spi, uint8_t csId);
Description	Assert software-controlled CS.
Parameters	HAL_Dev_t *spi SPI device control handle. uint8_t csId Slave-selection index.

Interface	Description
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.23 HAL_SPI_DisableCS

Table 5-82 HAL_SPI_DisableCS

Interface	Description
Function	HAL_Status_t HAL_SPI_DisableCS(const HAL_Dev_t *spi, uint8_t csId);
Description	Deassert software-controlled CS.
Parameters	HAL_Dev_t *spi SPI device control handle. uint8_t csId Slave-selection index.
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.24 HAL_SPI_RegisterIRQ

Table 5-83 HAL_SPI_RegisterIRQ

Interface	Description
Function	HAL_Status_t HAL_SPI_RegisterIRQ(HAL_Dev_t *spi, SPI_IRQ_t irq_type, const HAL_Callback_t *irqCb);
Description	Register an interrupt service routine.
Parameters	HAL_Dev_t *spi SPI device control handle. SPI_IRQ_t irq_type Interrupt type typedef enum { SPI_IRQ_TX = 0, SPI_IRQ_RX = 1, SPI_IRQ_CS = 2, } SPI_IRQ_t; HAL_Callback_t *irqCb Callback Function
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully.

Interface	Description
	A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.25 HAL_SPI_UnregisterIRQ

Table 5-84 HAL_SPI_UnregisterIRQ

Interface	Description
Function	HAL_Status_t HAL_SPI_UnregisterIRQ(const HAL_Dev_t *spi, SPI_IRQ_t irq_type);
Description	Unregister interrupt handling.
Parameters	HAL_Dev_t *spi SPI device control handle. SPI_IRQ_t irq_type Interrupt type typedef enum { SPI_IRQ_TX = 0, SPI_IRQ_RX = 1, SPI_IRQ_CS = 2, } SPI_IRQ_t;
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.26 HAL_SPI_EnableIRQ

Table 5-85 HAL_SPI_EnableIRQ

Interface	Description
Function	HAL_Status_t HAL_SPI_EnableIRQ(const HAL_Dev_t *spi, SPI_IRQ_t irq_type);
Description	Enables interrupts.
Parameters	HAL_Dev_t *spi SPI device control handle. SPI_IRQ_t irq_type Interrupt type typedef enum { SPI_IRQ_TX = 0, SPI_IRQ_RX = 1, SPI_IRQ_CS = 2, } SPI_IRQ_t;
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.27 HAL_SPI_DisableIRQ

Table 5-86 HAL_SPI_DisableIRQ

Interface	Description
Function	HAL_Status_t HAL_SPI_DisableIRQ(const HAL_Dev_t *spi, SPI_IRQ_t irq_type);
Description	Disables interrupts.
Parameters	HAL_Dev_t *spi SPI device control handle. SPI_IRQ_t irq_type Interrupt type typedef enum { SPI_IRQ_TX = 0, SPI_IRQ_RX = 1, SPI_IRQ_CS = 2, } SPI_IRQ_t;
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.28 HAL_SPI_RegisterDMA

Table 5-87 HAL_SPI_RegisterDMA

Interface	Description
Function	HAL_Status_t HAL_SPI_RegisterDMA(HAL_Dev_t *spi, SPI_TransDir_t transDir, HAL_Callback_t *dmaCb);
Description	Registers a DMA transfer callback.
Parameters	HAL_Dev_t *spi SPI device control handle. SPI_TransDir_t transDir Transfer direction HAL_Callback_t *irqCb Callback Function
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.29 HAL_SPI_UnregisterDMA

Table 5-88 HAL_SPI_UnregisterDMA

Interface	Description
Function	HAL_Status_t HAL_SPI_UnregisterDMA(const HAL_Dev_t *spi, SPI_TransDir_t transDir);
Description	Unregister the interrupt service routine.
Parameters	HAL_Dev_t *spi SPI device control handle. SPI_TransDir_t transDir Transfer direction
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.30 HAL_SPI_StartDMA

Table 5-89 HAL_SPI_StartDMA

Interface	Description
Function	HAL_Status_t HAL_SPI_StartDMA(const HAL_Dev_t *spi, SPI_TransDir_t transDir, const uint8_t *buf, int32_t size);
Description	Starts SPI transfers through DMA.
Parameters	HAL_Dev_t *spi SPI device control handle. SPI_TransDir_t transDir Transfer direction uint8_t *buf Transfer buffer int32_t size Transfer length
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.31 HAL_SPI_StopDMA

Table 5-90 HAL_SPI_StopDMA

Interface	Description
-----------	-------------

Interface	Description
Function	int32_t HAL_SPI_StopDMA(const HAL_Dev_t *spi, SPI_TransDir_t transDir);
Description	Stops SPI transfers through DMA.
Parameters	HAL_Dev_t *spi SPI device control handle. SPI_TransDir_t transDir Transfer direction
Considerations	None
Return Value	A negative value is an error code. A positive value is the number of data items transferred successfully.

5.3.3.32 HAL_SPI_PM_Suspend

Table 5-91 HAL_SPI_PM_Suspend

Interface	Description
Function	HAL_Status_t HAL_SPI_PM_Suspend(HAL_Dev_t *spi);
Description	Suspends SPI.
Parameters	HAL_Dev_t *spi SPI device control handle.
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.3.3.33 HAL_SPI_PM_Resume

Table 5-92 HAL_SPI_PM_Resume

Interface	Description
Function	HAL_Status_t HAL_SPI_PM_Resume(HAL_Dev_t *spi);
Description	Wake and resume SPI.
Parameters	HAL_Dev_t *spi SPI device control handle.
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.4 I2C

5.4.1 Function Overview

5.4.1.1 Features

1. Supports master and slave modes.
2. Supports 100 kHz, 400 kHz, and 1 MHz rates.
3. Supports configurable slave addresses.

5.4.1.2 Usage

The I2C HAL APIs provide two ways to implement these features:

1. Integrated APIs: use APIs such as Init, Open, Receive, and Transmit for SP transfers without handling implementation details.
2. Individual APIs: use single-function APIs for more complex or customized behavior, with more attention to implementation details.

The integrated APIs are recommended.

Sample:

[project/driver/i2c/i2c_poll_master](#)

[project/driver/i2c/i2c_poll_slave](#)

[project/driver/i2c/i2c_interrupt_master](#)

[project/driver/i2c/i2c_interrupt_slave](#)

The integrated-API workflow is as follows:

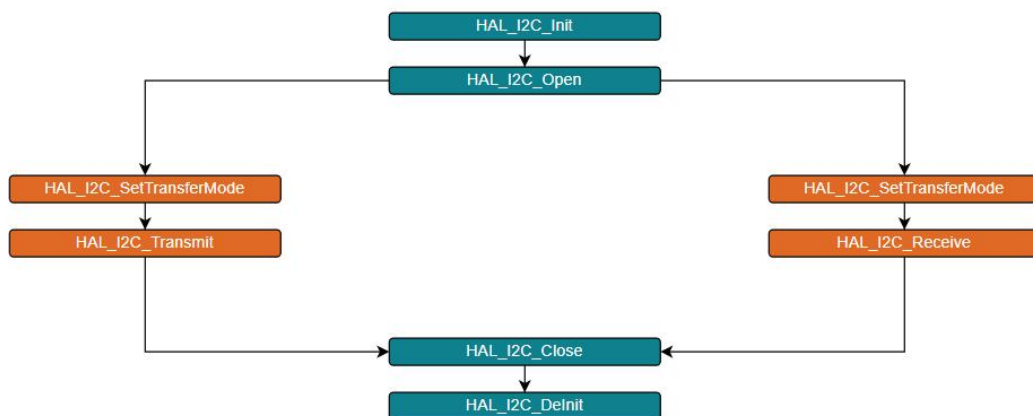


Figure 5-10 I2C Integrated-API Workflow

HAL_I2C_Init configures the I2C features using the following parameters.

```

typedef struct {
    I2C_Mode_t      mode;    /* Master or slave */
    I2C_Speed_t     speed;   /* Select 100k, 400k, or 1M */
    I2C_Callback_t  busErrCb; /* Error callback */
} I2C_InitParam_t;
  
```

The I2C bus can become stuck. busErrCb provides a bus-recovery interface. Normally, HAL_I2C_RecoverBus is called according to the error type to recover the bus.

Feature configuration macros

```

/* Enable I2C slave mode: 0 = disabled; 1 = enabled. */
Note: I2C_SLAVE mode is selected by default.
#define CONFIG_I2C_SLAVE 1

/* Enable I2C master mode: 0 = disabled; 1 = enabled. */
#define CONFIG_I2C_MASTER 0

/* Enable I2C interrupt transfer mode: 0 = disabled; 1 = enabled. */
#define CONFIG_I2C_TRANS_IT_ENABLE 0

/* Enable I2C parameter checking: 0 = disabled; 1 = enabled. */
#define CONFIG_I2C_ASSERT 0

/* Enable wake-up by addressing in I2C sleep mode: 0 = disabled; 1 = enabled. */
#define CONFIG_I2C_WAKEUP 0

```

Prj_config.h

```

/* Enable board MRS6130_p1812 */
#define CONFIG_BOARD_MRS6130_P1812 1

/* Enable I2C master mode: 0 = disabled; 1 = enabled. */
#define CONFIG_I2C_MASTER 1

/* Enable I2C interrupt transfer mode: 0 = disabled; 1 = enabled. */
#define CONFIG_I2C_TRANS_IT_ENABLE 0

```

5.4.2 Interface

5.4.2.1 HAL_I2C_Init

Table 5-93 HAL_I2C_init

Interface	Description
Function	HAL_Dev_t *HAL_I2C_Init(I2C_ID_t id, const I2C_InitParam_t *pinitParam);
Description	Initialize an I2C device, allocate system resources, and obtain a device control handle.
Parameters	I2C_ID_t id I2C peripheral ID. <pre>typedef enum { I2C0_ID = 0U, I2C1_ID = 1U, I2C_NUM, } I2C_ID_t;</pre> const I2C_InitParam_t *pinitParam Address of the I2C initialization-parameter structure. I2C speed; I2C mode; bus-error callback. <pre>typedef struct { I2C_Mode_t mode; I2C_Speed_t speed; I2C_Callback_t busErrCb; } I2C_InitParam_t;</pre>

Interface	Description
Considerations	None.
Return Value	I2C device instance handle (HAL_Dev_t *).

5.4.2.2 HAL_I2C_DeInit

Table 5-94 HAL_I2C_Deinit

Interface	Description
Function	HAL_Status_t HAL_I2C_DeInit(HAL_Dev_t *i2c);
Description	Deinitializes the I2C device and releases system resources.
Parameters	HAL_Dev_t *i2c I2C device instance handle.
Considerations	None.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.4.2.3 HAL_I2C_Open

Table 5-95 HAL_I2C_Open

Interface	Description
Function	HAL_Status_t HAL_I2C_Open(HAL_Dev_t *i2c, I2C_AddrWidth_t addrWidth, const uint16_t addr);
Description	Open the bus device at the configured address and start the peripheral.
Parameters	HAL_Dev_t *i2c I2C device instance handle. I2C_AddrWidth_t addrWidth Address width configured at startup. typedef enum { I2C_ADDR_WIDTH_7BIT = 0U, I2C_ADDR_WIDTH_10BIT = 1U, } I2C_AddrWidth_t; const uint16_t addr Address configured at startup.
Considerations	1. When initialized as I2C_MODE_MASTER, the configured address is the target address. 2. When initialized as I2C_MODE_SLAVE, the configured address is the slave address. 3. Do not call this interface repeatedly. 4. To change the address after Open, call Close and then Open again.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the

Interface	Description
	failure type.

5.4.2.4 HAL_I2C_Open

Table 5-96 HAL_I2C_Close

Interface	Description
Function	HAL_Status_t HAL_I2C_Open(HAL_Dev_t *i2c)
Description	Close the peripheral.
Parameters	HAL_Dev_t *i2c I2C device instance handle.
Considerations	The peripheral must have been opened before it can be closed.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.4.2.5 HAL_I2C_RecoverBus

Table 5-97 HAL_I2C_RecoverBus

Interface	Description
Function	HAL_Status_t HAL_I2C_RecoverBus(HAL_Dev_t *i2c)
Description	Recover the bus.
Parameters	HAL_Dev_t *i2c I2C device instance handle.
Considerations	None.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.4.2.6 HAL_I2C_SetTransferMode

Table 5-98 HAL_I2C_SetTransferMode

Interface	Description
Function	HAL_Status_t HAL_I2C_SetTransferMode(HAL_Dev_t *i2c, I2C_TransfDir_t tranfDir, I2C_TransfMode_t tranfMode);
Description	Configure the transfer mode for both Transmit and Receive.
Parameters	HAL_Dev_t *i2c I2C device instance handle. I2C_TransfDir_t tranfDir Transfer direction. typedef enum {

Interface	Description
	<pre> I2C_TRANSFDIR_TX = 0U, I2C_TRANSFDIR_RX = 1U, } I2C_TransfDir_t; I2C_TransfMode_t tranfMode Transfer type: polling (NORMAL), interrupt, or DMA. typedef enum { I2C_TRANSFMODE_NORMAL = 0U, I2C_TRANSFMODE_INTERRUPT = 1U, I2C_TRANSFMODE_DMA = 2U, } I2C_TransfMode_t; </pre>
Considerations	The default is NORMAL mode.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.4.2.7 HAL_I2C_Transmit

Table 5-99 HAL_I2C_Transmit

Interface	Description
Function	<pre>int32_t HAL_I2C_Transmit(HAL_Dev_t *i2c, const uint8_t *buff, uint32_t size, uint32_t msec);</pre>
Description	Transfers a fixed length of data using the transfer type configured by HAL_I2C_SetTransferMode .
Parameters	HAL_Dev_t *i2c I2C device instance handle. const uint8_t *buff Address of the data to transfer. uint32_t size Size of the data to transfer. uint32_t msec Transfer timeout (ms).
Considerations	◆ If not configured, data is transferred in NORMAL mode by default.
Return Value	A nonnegative return value is the actual length transmitted. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.4.2.8 HAL_I2C_Receive

Table 5-100 HAL_I2C_Receive

Interface	Description
-----------	-------------

Interface	Description
Function	<code>int32_t HAL_I2C_Receive(HAL_Dev_t *i2c, const uint8_t *buff, uint32_t size, uint32_t msec);</code>
Description	Receives a fixed length of data using the receive type configured by <code>HAL_I2C_SetTransferMode</code> .
Parameters	HAL_Dev_t *i2c I2C device instance handle. const uint8_t *buff Address of the receive-data buffer. uint32_t size Size of the data to receive. uint32_t msec Receive timeout (ms).
Considerations	If not configured, data is received in NORMAL mode by default.
Return Value	A nonnegative return value is the actual length received. A negative return value indicates that the function failed; <code>HAL_STATUS_*</code> identifies the failure type.

5.4.2.9 HAL_I2C_GetRxData

Table 5-101 HAL_I2C_GetRxData

Interface	Description
Function	uint8_t HAL_I2C_GetRxData(HAL_Dev_t *i2c);
Description	Obtains data from the RX FIFO.
Parameters	HAL_Dev_t *i2c I2C device instance handle.
Considerations	None
Return Value	The return value is the data obtained from the RX FIFO.

5.4.2.10 HAL_I2C_PutTxData

Table 5-102 HAL_I2C_PutTxData

Interface	Description
Function	void HAL_I2C_PutTxData(HAL_Dev_t *i2c, uint8_t txData, I2C_Cmd_t cmd);
Description	Put data into the TX FIFO.
Parameters	HAL_Dev_t *i2c I2C device instance handle. uint8_t txData One byte of data to transfer I2C_Cmd_t cmd Bus signal associated with the data. typedef enum { I2C_CMD_WRITE = 0x00U, I2C_CMD_READ = CMD_Msk, I2C_CMD_WRITE_STOP = STOP_Msk, I2C_CMD_WRITE_RESTART = RESTART_Msk, I2C_CMD_WRITE_RESTART_STOP = STOP_Msk RESTART_Msk, I2C_CMD_READ_STOP = CMD_Msk STOP_Msk, I2C_CMD_READ_RESTART = CMD_Msk RESTART_Msk, I2C_CMD_READ_RESTART_STOP = CMD_Msk STOP_Msk RESTART_Msk, } I2C_Cmd_t;
Considerations	1. In I2C_MODE_SLAVE mode, set this parameter to I2C_CMD_WRITE. 2. Check that the TX FIFO is not full before adding data.
Return Value	void

5.4.2.11 HAL_I2C_IsTxReady

Table 5-103 HAL_I2C_IsTxReady

Interface	Description
Function	bool HAL_I2C_IsTxReady(HAL_Dev_t *i2c);
Description	Check whether the TX FIFO is ready.
Parameters	HAL_Dev_t *i2c I2C device instance handle.
Considerations	None
Return Value	True: the TX FIFO is not empty and is ready. False: the TX FIFO is full and is not ready.

5.4.2.12 HAL_I2C_IsRxReady

Table 5-104 HAL_I2C_IsRxReady

Interface	Description
Function	bool HAL_I2C_IsRxReady(HAL_Dev_t *i2c);
Description	Check whether the RX FIFO is ready.
Parameters	HAL_Dev_t *i2c I2C device instance handle.
Considerations	None
Return Value	True: the RX FIFO is full and ready; False: the RX FIFO is not full and is not ready.

5.4.2.13 HAL_I2C_IsTxEEmpty

Table 5-105 HAL_I2C_IsTxEEmpty

Interface	Description
Function	bool HAL_I2C_IsTxEEmpty(HAL_Dev_t *i2c);
Description	Check whether the TX FIFO is empty.
Parameters	HAL_Dev_t *i2c I2C device instance handle.
Considerations	None
Return Value	True: the TX FIFO is empty. False: the TX FIFO is not empty.

5.4.2.14 HAL_I2C_RegisterIRQ

Table 5-106 HAL_I2C_RegisterIRQ

Interface	Description
Function	HAL_Status_t HAL_I2C_RegisterIRQ(HAL_Dev_t *i2c, I2C_Callback_t *irqCb);
Description	Register the callback invoked by the interrupt service routine.

Interface	Description
Parameters	HAL_Dev_t *i2c I2C device instance handle. I2C_Callback_t *irqCb Address of the interrupt-callback registration structure.
Considerations	Note that using this function disables the interrupt-mode Transmit and Receive functions.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.4.2.15 HAL_I2C_UnregisterIRQ

Table 5-107 HAL_I2C_UnregisterIRQ

Interface	Description
Function	HAL_Status_t HAL_I2C_UnregisterIRQ(HAL_Dev_t *i2c);
Description	Unregister the callback invoked by the interrupt service routine.
Parameters	HAL_Dev_t *i2c I2C device instance handle.
Considerations	None.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.4.2.16 HAL_I2C_EnableIRQ

Table 5-108 HAL_I2C_EnableIRQ

Interface	Description
Function	HAL_Status_t HAL_I2C_EnableIRQ(HAL_Dev_t *i2c, uint32_t irqStatus);
Description	Enables the specified interrupt type.
Parameters	HAL_Dev_t *i2c I2C device instance handle. uint32_t irqStatus Interrupt type to enable. typedef enum { I2C_IRQSTATUS_START = HAL_BIT(0), I2C_IRQSTATUS_STOP = HAL_BIT(1), I2C_IRQSTATUS_READ_REQ = HAL_BIT(2), I2C_IRQSTATUS_READ_DONE = HAL_BIT(3), I2C_IRQSTATUS_ERROR = HAL_BIT(4), I2C_IRQSTATUS_TX_FIFO_EMPTY = HAL_BIT(5),

Interface	Description
	<pre> I2C_IRQSTATUS_RX_FIFO_FULL = HAL_BIT(6), I2C_IRQSTATUS_ALL = HAL_BIT(0) HAL_BIT(1) HAL_BIT(2) HAL_BIT(3) HAL_BIT(4) HAL_BIT(5) HAL_BIT(6), } I2C_IRQStatus_t; </pre>
Considerations	After enabling the relevant flags, check and clear the corresponding interrupt status in the interrupt service routine.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.4.2.17 HAL_I2C_DisableIRQ

Table 5-109 HAL_I2C_DisableIRQ

Interface	Description
Function	HAL_Status_t HAL_I2C_DisableIRQ(HAL_Dev_t *i2c, uint32_t irqStatus);
Description	Disables the specified interrupt type.
Parameters	HAL_Dev_t *i2c I2C device instance handle. uint32_t irqStatus Interrupt type to disable. <pre> typedef enum { I2C_IRQSTATUS_START = HAL_BIT(0), I2C_IRQSTATUS_STOP = HAL_BIT(1), I2C_IRQSTATUS_READ_REQ = HAL_BIT(2), I2C_IRQSTATUS_READ_DONE = HAL_BIT(3), I2C_IRQSTATUS_ERROR = HAL_BIT(4), I2C_IRQSTATUS_TX_FIFO_EMPTY = HAL_BIT(5), I2C_IRQSTATUS_RX_FIFO_FULL = HAL_BIT(6), I2C_IRQSTATUS_ALL = HAL_BIT(0) HAL_BIT(1) HAL_BIT(2) HAL_BIT(3) HAL_BIT(4) HAL_BIT(5) HAL_BIT(6), } I2C_IRQStatus_t; </pre>
Considerations	None.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.4.2.18 HAL_I2C_ClearIRQ

Table 5-110 HAL_I2C_ClearIRQ

Interface	Description
Function	HAL_Status_t HAL_I2C_ClearIRQ(HAL_Dev_t *i2c, uint32_t irqStatus);

Interface	Description
Description	Disables the specified interrupt type.
Parameters	HAL_Dev_t *i2c I2C device instance handle. uint32_t irqStatus Interrupt type to clear. <pre>typedef enum { I2C_IRQSTATUS_START = HAL_BIT(0), I2C_IRQSTATUS_STOP = HAL_BIT(1), I2C_IRQSTATUS_READ_REQ = HAL_BIT(2), I2C_IRQSTATUS_READ_DONE = HAL_BIT(3), I2C_IRQSTATUS_ERROR = HAL_BIT(4), I2C_IRQSTATUS_TX_FIFO_EMPTY = HAL_BIT(5), I2C_IRQSTATUS_RX_FIFO_FULL = HAL_BIT(6), I2C_IRQSTATUS_ALL = HAL_BIT(0) HAL_BIT(1) HAL_BIT(2) HAL_BIT(3) HAL_BIT(4) HAL_BIT(5) HAL_BIT(6), } I2C_IRQStatus_t;</pre>
Considerations	None.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.4.2.19 HAL_I2C_GetIRQStatus

Table 5-111 HAL_I2C_GetIRQStatus

Interface	Description
Function	uint32_t HAL_I2C_GetIRQStatus(HAL_Dev_t *i2c);
Description	Get the triggered interrupt type.
Parameters	HAL_Dev_t *i2c I2C device instance handle. uint32_t irqStatus Interrupt type to clear. <pre>typedef enum { I2C_IRQSTATUS_START = HAL_BIT(0), I2C_IRQSTATUS_STOP = HAL_BIT(1), I2C_IRQSTATUS_READ_REQ = HAL_BIT(2), I2C_IRQSTATUS_READ_DONE = HAL_BIT(3), I2C_IRQSTATUS_ERROR = HAL_BIT(4), I2C_IRQSTATUS_TX_FIFO_EMPTY = HAL_BIT(5), I2C_IRQSTATUS_RX_FIFO_FULL = HAL_BIT(6), I2C_IRQSTATUS_ALL = HAL_BIT(0) HAL_BIT(1) HAL_BIT(2) </pre>

Interface	Description
	<pre> HAL_BIT(3) HAL_BIT(4) HAL_BIT(5) HAL_BIT(6), } I2C_IRQStatus_t; </pre>
Considerations	1. In the registered interrupt callback, first obtain the trigger type, then execute the corresponding function. 2. GetRxData requires I2C_IRQSTATUS_RX_FIFO_FULL to be triggered before receiving data. 3. PutTxData requires I2C_IRQSTATUS_TX_FIFO_EMPTY to be triggered before adding data.
Return Value	The return value is the obtained trigger type. The type is described by the following enumeration: <pre> typedef enum { I2C_IRQSTATUS_START = HAL_BIT(0), I2C_IRQSTATUS_STOP = HAL_BIT(1), I2C_IRQSTATUS_READ_REQ = HAL_BIT(2), I2C_IRQSTATUS_READ_DONE = HAL_BIT(3), I2C_IRQSTATUS_ERROR = HAL_BIT(4), I2C_IRQSTATUS_TX_FIFO_EMPTY = HAL_BIT(5), I2C_IRQSTATUS_RX_FIFO_FULL = HAL_BIT(6), I2C_IRQSTATUS_ALL = HAL_BIT(0) HAL_BIT(1) HAL_BIT(2) HAL_BIT(3) HAL_BIT(4) HAL_BIT(5) HAL_BIT(6), } I2C_IRQStatus_t; </pre>

5.5 DMA

5.5.1 Function Overview

DMA supports the following features:

1. Memory-to-memory, memory-to-peripheral, and peripheral-to-memory transfers.
2. Eight channels operating in parallel.
3. Data widths up to 32 bits.
4. Block and linked-list transfers.
5. A 32-bit x 8 FIFO for each channel.

Sample:

[project/driver/dma/mem2mem_block](#)

[project/driver/dma/mem2mem_list](#)

5.5.1.1 Block Transfer

The block-transfer workflow is shown below.

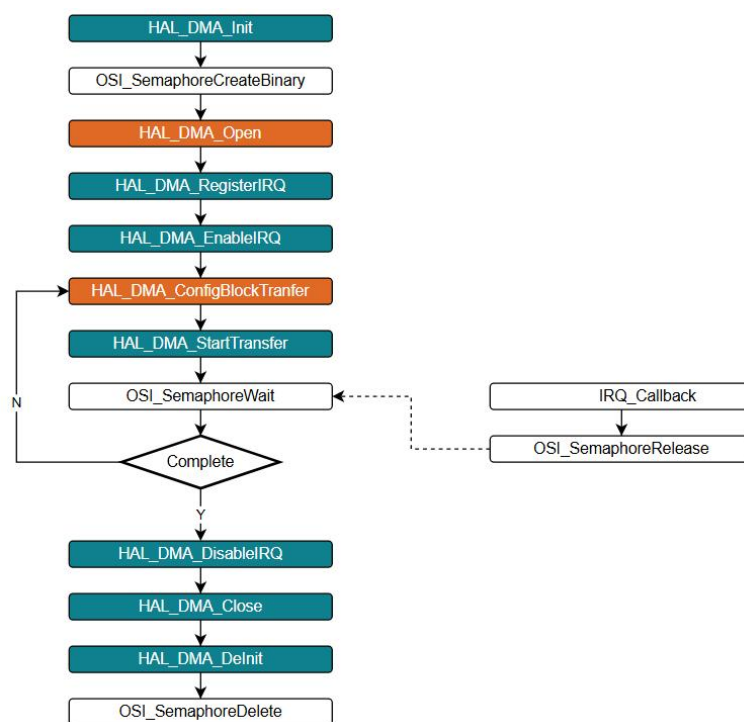


Figure 5-11 DMA Block-Transfer Workflow

The key interfaces are `HAL_DMA_Open` and `HAL_DMA_ConfigBlockTransfer`. `HAL_DMA_Open` allocates an unused channel and configures its common properties; call it once. `HAL_DMA_ConfigBlockTransfer` configures per-transfer data parameters and can be called repeatedly together with `HAL_DMA_StartTransfer`.

`HAL_DMA_Open` channel parameters are as follows.

```

typedef struct {
    uint32_t direction      : 4;
    /* Channel priority, 0-7. A larger number has higher priority when multiple channels operate simultaneously. */
    uint32_t priority       : 4;
    /* Source-address increment mode: increment, decrement, or fixed. Use fixed for peripherals and increment/decrement for memory. */
    uint32_t srcIncrMode    : 4;
    /* Destination-address increment mode: increment, decrement, or fixed. Use fixed for peripherals and increment/decrement for memory. */
    uint32_t dstIncrMode    : 4;
    /* Block-transfer count. Not used for memory-to-memory transfers; for peripheral/memory transfers it depends on the peripheral and DMA FIFO depths. */
    uint32_t srcBurstNum    : 4;
    uint32_t dstBurstNum    : 4;
    /* Source data width. Memory can use 8/16/32 bits; a peripheral must match its transfer width, usually 8 bits. */
    uint32_t srcWidth       : 4;
    /* Destination data width. Memory can use 8/16/32 bits; a peripheral must match its transfer width, usually 8 bits. */
    uint32_t dstWidth       : 4;
    /* Source interface ID, such as UART0_TX or UART_RX. */
    uint16_t srcHandShake    : 4;
    /* Destination interface ID, such as UART0_TX or UART_RX. */
    uint16_t dstHandShake    : 4;
    /* Whether to reload the source start address; see DMA_BlockConf_t for HAL_DMA_ConfigBlockTransfer. */
    uint16_t srcEnReload     : 1;
    /* Whether to reload the destination start address; see DMA_BlockConf_t for HAL_DMA_ConfigBlockTransfer. */
}
  
```

```

*/
uint16_t  dstEnReload    : 1;
uint16_t  srcEnStatus   : 1;
uint16_t  dstEnStatus   : 1;
uint16_t  pad           : 4;
} DMA_ChannelConf_t;

```

Memory to memory

Two configurations are common. DMA_BITWIDTH_32 is efficient but requires the source or destination address to be 32-bit aligned. DMA_BITWIDTH_8 is less efficient but requires no address alignment. Select as needed. In this scenario, set IncrMode to DMA_ADDR_INCREMENT and blockSize to the transfer length in bytes; for a 64-byte transfer, set it to 64 regardless of Width.

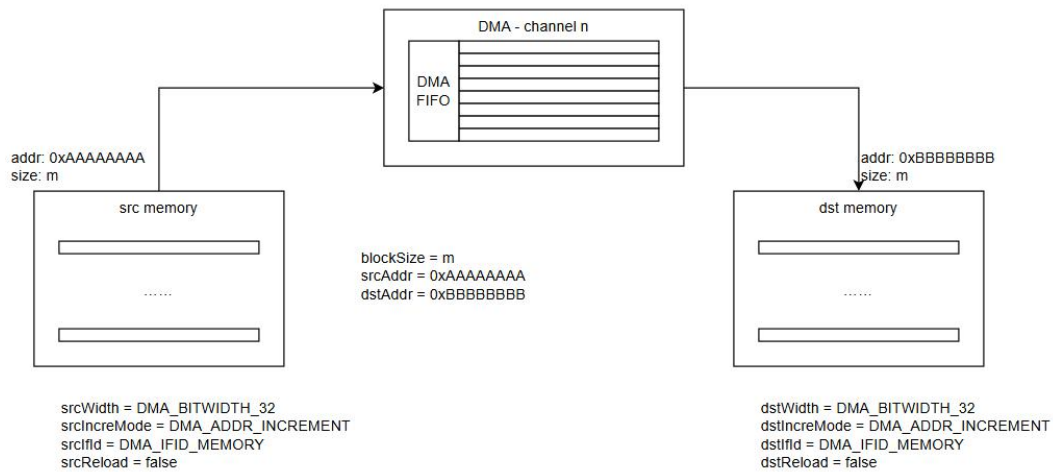


Figure 5-12 DMA Memory-to-Memory Transfer with 32-bit Width

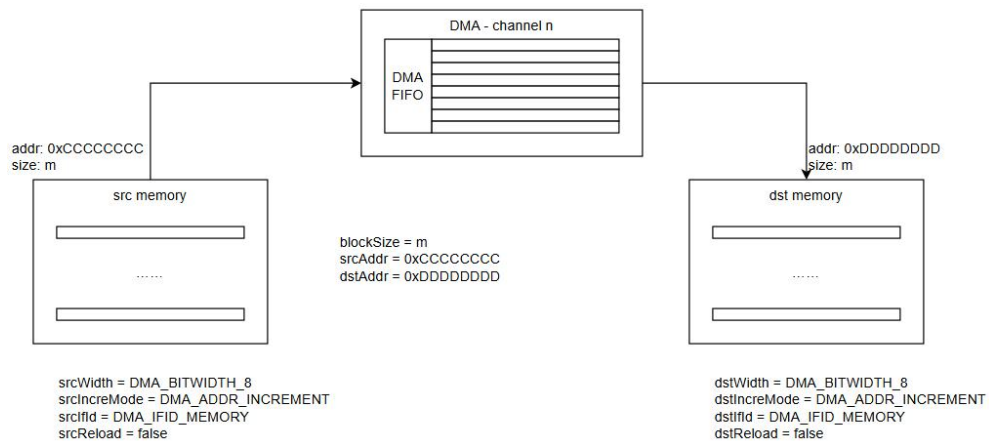


Figure 5-13 DMA Memory-to-Memory Transfer with 8-bit Width

UART Rx to Memory

For peripheral-to-memory DMA transfers, note the following:

1. Set both source and destination widths to DMA_BITWIDTH_8.
2. Set burstNum to 1 so that one byte in the UART RX FIFO triggers a DMA transfer, ensuring that all data is moved.

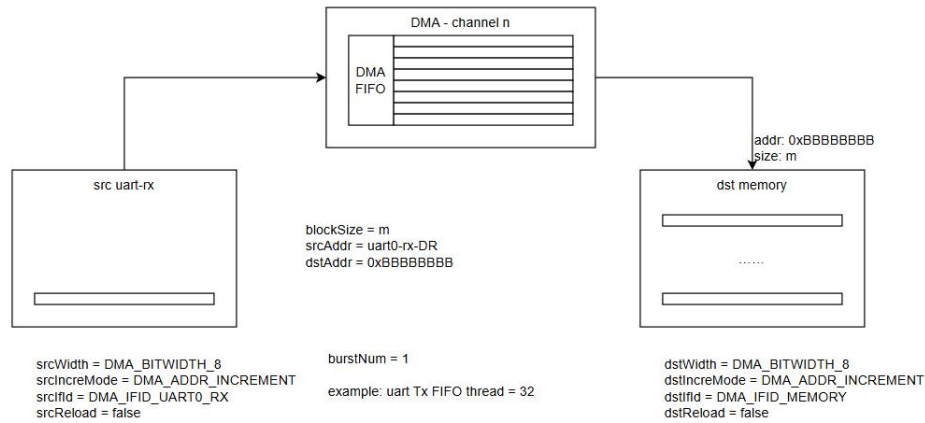


Figure 5-14 DMA UART0-RX-to-Memory Transfer

Memory to UART Tx

For memory-to-peripheral DMA transfers, note the following:

1. Set both source and destination widths to DMA_BITWIDTH_8.
2. burstNum depends on the peripheral TX FIFO trigger level. If the TX FIFO is set to 64 - n, burstNum must be less than n to avoid FIFO overflow during one DMA trigger. If burstNum is 1 and the TX FIFO trigger is 1, the FIFO may empty before DMA moves data to UART. Therefore, use the largest practical value below TX FIFO size minus trigger level; for example, with a TX FIFO level of 32, set burstNum to 8.

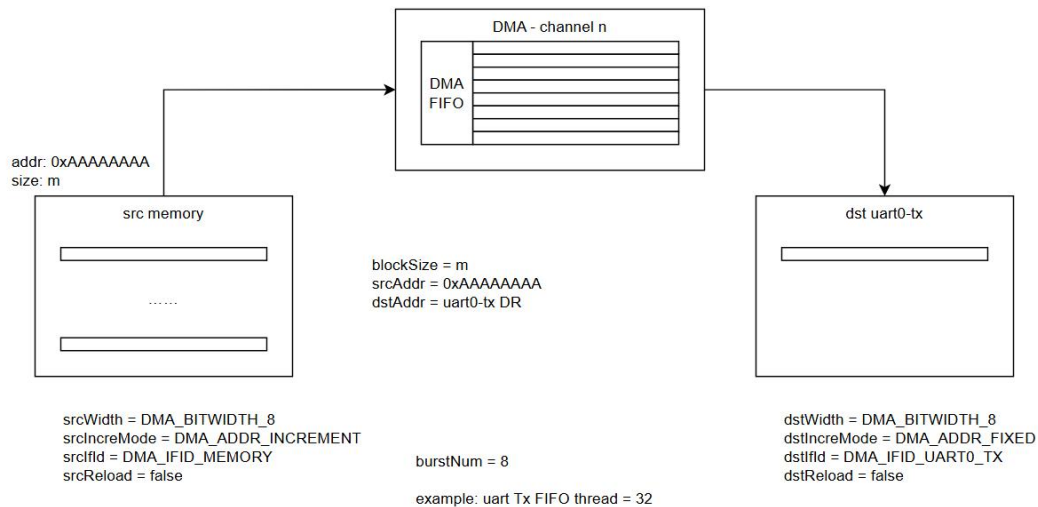


Figure 5-15 DMA UART0-TX-to-Memory Transfer

5.5.1.2 Linked-list Transfer

The linked-list transfer workflow is shown below.

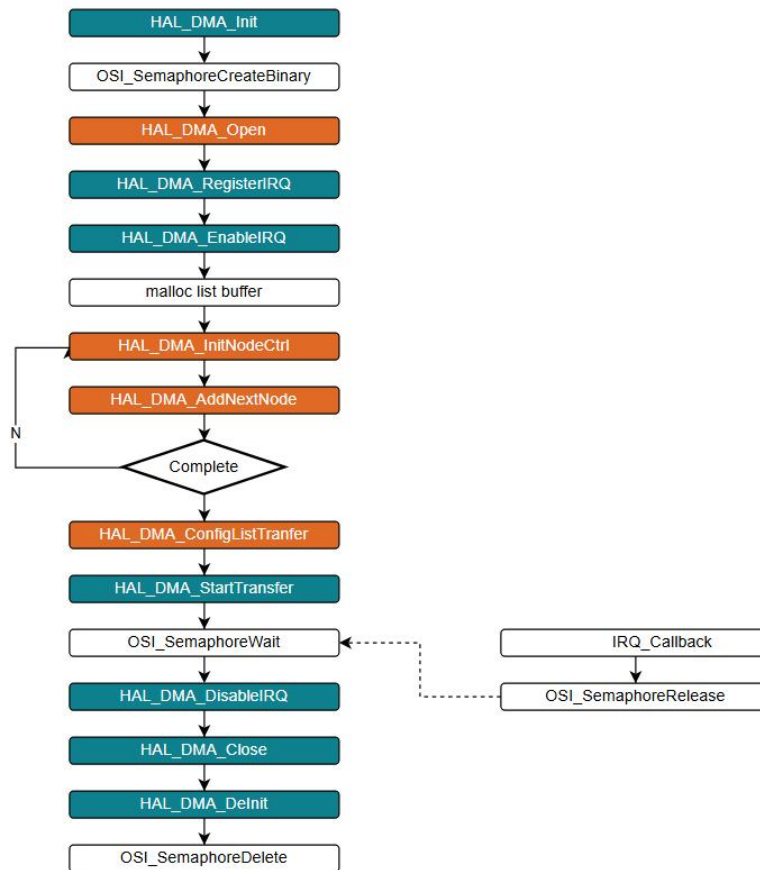


Figure 5-16 DMA Linked-List Transfer Workflow

Linked-list transfer stores DMA configurations in SRAM and organizes multiple configurations into a list. After DMA starts, it executes each item in sequence. HAL_DMA_Open configuration is the same as for block transfer; the difference is list initialization and organization. HAL_DMA_InitNodeCtrl initializes one list item, HAL_DMA_AddNextNode links the items, and HAL_DMA_ConfigListTransfer configures list transfer.

5.5.2 Interface

5.5.2.1 HAL_DMA_Init

Table 5-112 HAL_DMA_Init

Interface	Description
Function	HAL_Dev_t *HAL_DMA_Init(DMA_ID_t id);
Description	Initialize an I2C device, allocate system resources, and obtain a device control handle.
Parameters	DMA_ID_t id DMA peripheral ID. typedef enum { DMA0_ID = (0x0U), DMA_NUM, } DMA_ID_t;
Considerations	None.
Return Value	DMA device control handle (HAL_Dev_t *).

5.5.2.2 HAL_DMA_DeInit

Table 5-113 HAL_DMA_DeInit

Interface	Description
Function	HAL_Status_t HAL_DMA_DeInit(HAL_Dev_t *dma);
Description	Deinitializes the DMA device and releases system resources.
Parameters	HAL_Dev_t *dma DMA device instance handle.
Considerations	None.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.5.2.3 HAL_DMA_Open

Table 5-114 HAL_DMA_Open

Interface	Description
Function	int32_t HAL_DMA_Open(HAL_Dev_t *dma, const DMA_ChannelConf_t *pchanCfg);
Description	Configure the parameters and start a DMA channel.
Parameters	HAL_Dev_t *dma DMA device control handle. const DMA_ChannelParam_t *pchanCfg Channel configuration parameters. typedef struct { uint32_t direction : 4; uint32_t priority : 4; uint32_t srcIncrMode : 4; uint32_t dstIncrMode : 4; uint32_t srcBurstNum : 4; uint32_t dstBurstnum : 4; uint32_t srcWidth : 4; uint32_t dstWidth : 4; uint16_t srcHandShake : 4; uint16_t dstHandShake : 4; uint16_t srcEnReload : 1; uint16_t dstEnReload : 1; uint16_t srcEnStatus : 1; uint16_t dstEnStatus : 1; uint16_t pad : 4; } DMA_ChannelConf_t;
Considerations	None.
Return Value	A nonnegative return value is the allocated channel ID. A negative return value indicates allocation failure; HAL_STATUS_* identifies the failure type.

5.5.2.4 HAL_DMA_Close

Table 5-115 HAL_DMA_Close

Interface	Description
Function	HAL_Status_t HAL_DMA_Close(HAL_Dev_t *dma, uint32_t chId);
Description	Close an allocated DMA channel and release its resources.
Parameters	HAL_Dev_t *dma DMA device instance handle. uint32_t chId ID of the allocated channel to close.
Considerations	The channel ID to close must be an ID successfully allocated by HAL_DMA_Open .
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.5.2.5 HAL_DMA_ConfigBlockTransfer

Table 5-116 HAL_DMA_ConfigBlockTransfer

Interface	Description
Function	HAL_Status_t HAL_DMA_ConfigBlockTransfer(HAL_Dev_t *dma, uint32_t chId, const DMA_BlockConf_t *pblockCfg);
Description	Configure block-transfer parameters and set the channel to block-transfer mode.
Parameters	HAL_Dev_t *dma DMA device instance handle. uint32_t chId Successfully allocated channel ID. const DMA_BlockConf_t *pblockCfg Address of the structure containing the source address, destination address, and total block size. typedef struct { uint32_t srcAddr; uint32_t dstAddr; uint32_t blockSize; } DMA_BlockConf_t;
Considerations	1. The configured channel ID must be an ID successfully allocated by HAL_DMA_Open . 2. For the same channel, parameters configured by this function override those from other HAL_DMA_Config** functions and are mutually exclusive with HAL_DMA_ConfigListTransfer configuration.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.5.2.6 HAL_DMA_InitNodeCtrl

Table 5-117 HAL_DMA_InitNodeCtrl

Interface	Description
Function	HAL_Status_t HAL_DMA_InitNodeCtrl(HAL_Dev_t *dma, uint32_t chId, DMA_NodeConf_t *pcurrentNode);
Description	Initialize one linked-list node and configure its nodeCtrlx parameters.
Parameters	HAL_Dev_t *dma DMA device instance handle. uint32_t chId Successfully allocated channel ID. DMA_NodeConf_t *pcurrentNode Address of the allocated node storage. <pre>typedef struct DMA_Node { uint32_t srcAddr; uint32_t dstAddr; struct DMA_Node *pNextNode; uint32_t nodeCtrl0; uint32_t blockSize :12; uint32_t nodeCtrl1 :20; uint32_t nodeCtrl2; uint32_t nodeCtrl3; } DMA_NodeConf_t;</pre>
Considerations	1. The configured channel ID must be an ID successfully allocated by HAL_DMA_Open. 2. For a channel with the same ID, this function's parameters override other HAL_DMA_A_Config** settings and are mutually exclusive with HAL_DMA_ConfigListTransfer and HAL_DMA_ConfigScatterGatherTransfer configurations. 3. The supplied storage pointer must not be NULL. 4. Assign values to srcAddr, dstAddr, and blockSize before passing the object to this function to initialize the node. 5. For multiple nodes, call this function repeatedly to configure nodeCtrlx for each node.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.5.2.7 HAL_DMA_AddNextNode

Table 5-118 HAL_DMA_AddNextNode

Interface	Description
Function	void HAL_DMA_AddNextNode(DMA_NodeConf_t *pcurrentNode, DMA_NodeConf_t *pNextListNode)
Description	Link the current node to the next node.

Interface	Description
Parameters	DMA_NodeConf_t *pcurrentNode Address of the current-node structure. DMA_NodeConf_t *pNextNode Address of the next-node structure.
Considerations	1. The configured channel ID must be an ID successfully allocated by HAL_DMA_Open . 2. Before calling, ensure that <code>HAL_DMA_InitNodeCtrl</code> has configured the <code>nodeCtrlx</code> parameters in both node structures.
Return Value	Void

5.5.2.8 HAL_DMA_ConfigListTranfer

Table 5-119 HAL_DMA_ConfigListTranfer

Interface	Description
Function	<code>HAL_Status_t HAL_DMA_ConfigListTranfer(HAL_Dev_t *dma, uint32_t chId, const DMA_NodeConf_t *pheadlist);</code>
Description	Link the current node to the next node.
Parameters	HAL_Dev_t *dma DMA device instance handle. uint32_t chId Successfully allocated channel ID. DMA_NodeConf_t *pheadlist Address of the initialized list's head-node structure.
Considerations	1. The configured channel ID must be an ID successfully allocated by HAL_DMA_Open . 2. Record the address of the first allocated list node.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; <code>HAL_STATUS_*</code> identifies the failure type.

5.5.2.9 HAL_DMA_ConfigScatterGatherTranfer

Table 5-120 HAL_DMA_ConfigScatterGatherTranfer

Interface	Description
Function	<code>HAL_Status_t HAL_DMA_ConfigScatterGatherTranfer(HAL_Dev_t *dma, uint32_t chId, const DMA_SGConf_t *psgCfg);</code>
Description	Link the current node to the next node.
Parameters	HAL_Dev_t *dma DMA device instance handle. uint32_t chId Successfully allocated channel ID.

Interface	Description
	const DMA_SGConf_t *psgCfg Address of the scatter-gather configuration structure. Source/destination addresses and transfer size; number of source-gather blocks and destination-scatter blocks; source-gather and destination-scatter address intervals; source-gather and destination-scatter enables. <pre>typedef struct { uint32_t srcAddr; uint32_t dstAddr; uint32_t blockSize; uint32_t srcGatherCount; uint32_t srcGatherInter; uint32_t dstScatterCount; uint32_t dstScatterInter; uint16_t enSrcGather; uint16_t enDstScatter; } DMA_SGConf_t;</pre>
Considerations	1. The configured channel ID must be an ID successfully allocated by HAL_DMA_Open. 2. Enable enSrcGather/enDstScatter as required. They are disabled by default; enable the corresponding flag when configuring its parameters.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.5.2.10 HAL_DMA_StartTransfer

Table 5-121 HAL_DMA_StartTransfer

Interface	Description
Function	HAL_Status_t HAL_DMA_StartTransfer(HAL_Dev_t *dma, uint32_t chId)
Description	Start data transfer on a configured channel.
Parameters	HAL_Dev_t *dma DMA device instance handle. uint32_t chId Successfully allocated channel ID.
Considerations	1. The configured channel ID must be an ID successfully allocated by HAL_DMA_Open. 2. Configure the channel ID as required before calling HAL_DMA_StartTransfer.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.5.2.11 HAL_DMA_AbortTransfer

Table 5-122 HAL_DMA_AbortTransfer

Interface	Description
Function	HAL_Status_t HAL_DMA_AbortTransfer(HAL_Dev_t *dma, uint32_t chId)
Description	Stop data transfer on a channel.
Parameters	HAL_Dev_t *dma DMA device instance handle. uint32_t chId Successfully allocated channel ID.
Considerations	The configured channel ID must be an ID successfully allocated by HAL_DMA_Open .
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.5.2.12 HAL_DMA_RegisterIRQ

Table 5-123 HAL_DMA_RegisterIRQ

Interface	Description
Function	HAL_Status_t HAL_DMA_RegisterIRQ(HAL_Dev_t *dma, uint32_t chId, DMA_Event_t event, HAL_Callback_t *eventCb);
Description	Register the callback invoked by the interrupt service routine.
Parameters	HAL_Dev_t *dma DMA device instance handle. uint32_t chId Successfully allocated channel ID. DMA_Event_t event Register interrupt events that trigger the callback. Transfer-complete, list-node/block-complete, and error events. typedef enum { DMA_EVENT_DONE = (HAL_BIT(0)), DMA_EVENT_BLOCK_DONE = (HAL_BIT(1)), DMA_EVENT_ERR = (HAL_BIT(2)), DMA_EVENT_ALL = (DMA_EVENT_DONE DMA_EVENT_BLOCK_DONE DMA_EVENT_ERR), } DMA_Event_t; HAL_Callback_t *eventCb Address of the registered interrupt-callback structure.
Considerations	1. The configured channel ID must be an ID successfully allocated by HAL_DMA_Open . 2. When configuring event to register an interrupt callback, do not select DMA_EVENT

Interface	Description
	_ALL.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.5.2.13 HAL_DMA_UnRegisterIRQ

Table 5-124 HAL_DMA_UnRegisterIRQ

Interface	Description
Function	HAL_Status_t HAL_DMA_UnRegisterIRQ(HAL_Dev_t *dma);
Description	Unregister the callback invoked by the interrupt service routine.
Parameters	HAL_Dev_t *dma DMA device instance handle.
Considerations	The configured channel ID must be an ID successfully allocated by HAL_DMA_Open .
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.5.2.14 HAL_DMA_EnableIRQ

Table 5-125 HAL_DMA_EnableIRQ

Interface	Description
Function	HAL_Status_t HAL_DMA_EnableIRQ(HAL_Dev_t *dma, uint32_t chId, DMA_Event_t event);
Description	Enable the trigger-event interrupt.
Parameters	HAL_Dev_t *dma DMA device instance handle. uint32_t chId Successfully allocated channel ID. DMA_Event_t event Register interrupt events that trigger the callback.
Considerations	The configured channel ID must be an ID successfully allocated by HAL_DMA_Open .
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.5.2.15 HAL_DMA_DisableIRQ

Table 5-126 HAL_DMA_DisableIRQ

Interface	Description
Function	HAL_Status_t HAL_DMA_DisableIRQ(HAL_Dev_t *dma, uint32_t chId, DMA_Event_t event);
Description	Disable the trigger-event interrupt.
Parameters	HAL_Dev_t *dma DMA device instance handle. uint32_t chId Successfully allocated channel ID. DMA_Event_t event Register interrupt events that trigger the callback.
Considerations	The configured channel ID must be an ID successfully allocated by HAL_DMA_Open .
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.5.2.16 HAL_DMA_GetChanStatus

Table 5-127 HAL_DMA_DisableIRQ

Interface	Description
Function	int32_t HAL_DMA_GetChanStatus(HAL_Dev_t *dma, uint32_t chId, uint32_t*doneBlockNum);
Description	Get channel status.
Parameters	HAL_Dev_t *dma DMA device instance handle. uint32_t chId Successfully allocated channel ID. uint32_t*doneBlockNum Address of the variable receiving the number of successfully completed blocks.
Considerations	1. The configured channel ID must be an ID successfully allocated by HAL_DMA_Open. 2. In single-block mode, the maximum block count is 1.
Return Value	A nonnegative return value indicates channel state: 0 = idle; 1 = busy. A negative return value indicates that the function failed; HAL_STATUS_* identifies the failure type.

5.6 FLASH

5.6.1 Function Overview

Flash Control operates SPI flash. It supports XIP over dual and quad SPI, and single-line SPI read, erase, and write by default. The default erase unit is 4 KB.

A flash support list provides compatibility with different SPI flash devices. It stores the basic properties of multiple device types, as follows.

```
typedef struct {
uint32_t    flashSize;      /* Total flash capacity, in bytes */
uint32_t    spiFreqMax;     /* Maximum supported SPI frequency, in Hz */
uint8_t     spiFormatMax;   /* Maximum supported SPI format: 0 = single; 1 = dual; 2 = quad */
uint8_t     qerType;        /* If quad SPI is selected, quad-enable format in the flash SR register */
/*
uint8_t     sleepSupport;    /* Whether the flash supports deep sleep */
uint8_t     sleepLatency;    /* Flash sleep latency, in  $\mu$ s */
uint8_t     resumeLatency;   /* Flash resume latency, in  $\mu$ s */
uint8_t     chipId[3];      /* Flash Chip Id, 3 bytes */
} FLASH_ChipInfo_t;
```

The GD25WQ80E SPI flash is used below to illustrate how to obtain and enter each parameter.

Chip ID

Flash devices are indexed in the Flash List by a 3-byte chip ID. The ID normally identifies the manufacturer and device type and is read with command 9FH. HAL_FLASH_Open reads the chip ID and searches the list. If found, it uses that entry to configure the SPI flash; otherwise, it can use default parameters or fail and exit.

GD25WQ80E

Operation Code	MID7-MID0	ID15-ID8	ID7-ID0
9FH	C8	65	14
90H	C8		13
ABH			13

Figure 5-17 Flash Chip ID Example

SPI Format

SPI Format specifies whether XIP uses 2-line dual SPI or 4-line quad SPI. As shown below, dual SPI uses SI and SO for data, while quad SPI uses IO0 through IO3.

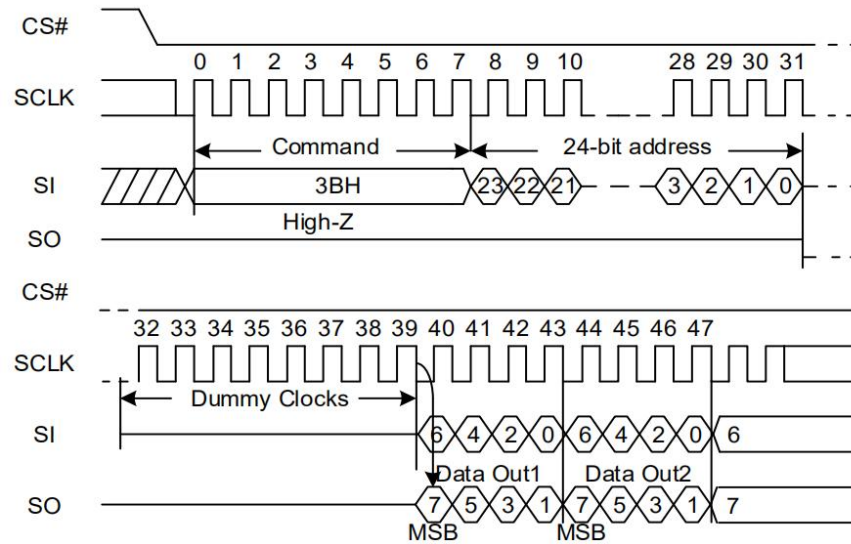


Figure 5-18 Dual SPI Transfer Example

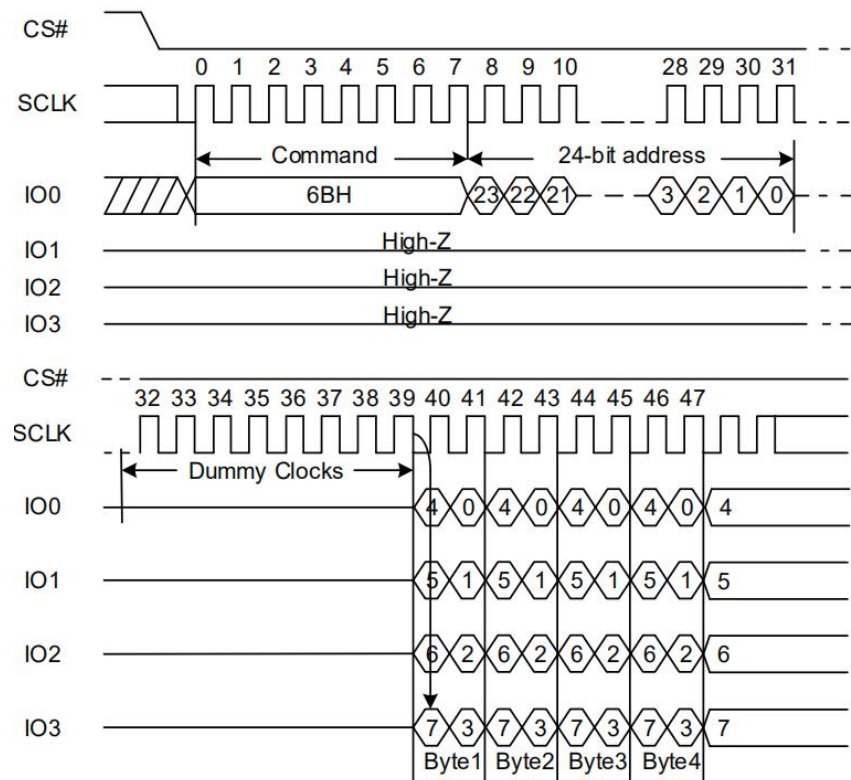


Figure 5-19 Quad SPI Transfer Example

XIP uses dual SPI by default. Quad SPI requires all three of the following conditions:

1. The hardware must be designed for quad SPI, and the spiFormat passed to HAL_FLASH_Open must match the hardware.
2. The SPI Flash must support Quad SPI, and spiFormatMax in ChipInfo must be set to 2 according to the Flash specification.
3. qerType in ChipInfo in the Flash List must be set correctly. Flash Quad SPI mode is enabled by sending a command to configure a register in the Flash. QerType identifies the location of the enable bit.

The Features section of an SPI flash specification normally states whether quad SPI is supported, as shown below.

Standard, Dual, Quad SPI

- Standard SPI: SCLK, CS#, SI, SO, WP#, HOLD#
- Dual SPI: SCLK, CS#, IO0, IO1, WP#, HOLD#
- Quad SPI: SCLK, CS#, IO0, IO1, IO2, IO3

Figure 5-20 SPI Format Support in the Specification

QER identifies the Quad Enable bit location in a status register. A bit at Status Register 2, bit 6 is S2B6 and corresponds to value 2.

Table 5. Status Register-SR No.2

No.	Name	Description	Note
S15	SUS	Suspend Bit	Volatile, read only
S14	CMP	Complement Protect Bit	Non-volatile writable
S13	Reserved	Reserved	Reserved
S12	DC	Dummy Configuration Bit	Non-volatile writable
S11	LB1	Security Register Lock Bit	Non-volatile writable (OTP)
S10	LB0	Security Register Lock Bit	Non-volatile writable (OTP)
S9	QE	Quad Enable Bit	Non-volatile writable
S8	SRP1	Status Register Protection Bit	Non-volatile writable

Figure 5-21 QER Example in the Specification

QER is defined below. Byte bit positions range from 0 to 7, not 1 to 8.

Table 5-128 QER Type

Type	Value	Description
NONE	0	Quad SPI is not supported
S2B1	1	Quad SPI is enabled by bit 1 of Status Register 2.
S2B6	2	Quad SPI is enabled by bit 6 of Status Register 2.
S2B7	3	Quad SPI is enabled by bit 6 of Status Register 2.

Note: The SPI Format above applies to XIP instruction execution and data reads. HAL_FLASH_Read, HAL_FLASH_Write, and HAL_FLASH_Erase use single-line SPI by default.

SPI Max Freq

Maximum SPI speed affects XIP efficiency and is normally determined by three factors:

1. The hardware layout determines the maximum operating frequency. Configure clockFreq passed to HAL_FLASH_Open according to the hardware.
2. The SoC imposes a maximum frequency. For example, the RS6240 maximum is AHB divided by 2. Use HAL_CLOCK_GetFreq to obtain the actual system frequency. See BOARD_InitXIP.
3. The SPI flash imposes its own maximum frequency. Enter the specification value as spiFreqMax in the Flash List.

Note: The maximum frequency in the SPI flash specification depends on dual/quad SPI mode and operating voltage.

(T=-40°C~85°C, VCC=1.65~3.6V, C_L=30pf)

Symbol	Parameter	Min.	Typ.	Max.	Unit.
F _{C1}	Serial Clock Frequency For: all commands except Read (03H), DC=1 on 2.3-3.6V power supply			104	MHz
F _{C2}	Serial Clock Frequency For: all commands except Read (03H), DC=1 on 1.65-2.3V power supply			80	MHz
f _C	Serial Clock Frequency For: all commands except Read (03H), DC=0			66	MHz
F _R	Serial Clock Frequency For: Read (03H)			50	MHz

Figure 5-22 Maximum SPI Frequency in the Specification

Flash Size

OTA and the KVF file system require the total SPI Flash size. flashSize in the Flash List specifies the device capacity. HAL_FLASH_Open automatically detects the capacity by wraparound comparison.

Sleep

In low-power applications, SPI Flash must enter deep sleep. Normally, command B9H puts the Flash to sleep and command ABH wakes it. A delay is required for each state transition. sleepSupport, sleepLatency, and resumeLatency in the Flash List specify these parameters.

t _{DP}	CS# High To Deep Power-Down Mode			3	μs
t _{RES1}	CS# High To Standby Mode Without Electronic Signature Read			30	μs
t _{RES2}	CS# High To Standby Mode With Electronic Signature Read			30	μs

Figure 5-23 Sleep Latency in the Specification

With power management enabled, the flash driver automatically sleeps and wakes the device. Waking SPI flash consumes extra power, and frequent transitions add save/restore overhead. If the application does not continuously need XIP, use HAL_FLASH_ExtControl to disable automatic sleep/wake, call HAL_FLASH_Resume before XIP use, and call HAL_FLASH_Suspend afterward.

5.6.2 Interface

5.6.2.1 HAL_FLASH_Init

Table 5-129 HAL_FLASH_Init

Interface	Description
Function	HAL_Dev_t * HAL_FLASH_Init(FLASH_ID_t id);
Description	Initialize the FLASH driver and allocate system resources without initializing hardware.
Parameters	FLASH_ID_t id FLASH peripheral ID. By default, only FLASH_DEFT_ID (0) is available.
Considerations	1. Initialization is idempotent; repeated calls only return the control handle. 2. The system initializes it by default in Board.c ; no user call is required.
Return Value	FLASH control handle, or NULL if initialization fails.

5.6.2.2 HAL_FLASH_DeInit

Table 5-130 HAL_FLASH_DeInit

Interface	Description
-----------	-------------

Interface	Description
Function	HAL_Status_t HAL_FLASH_DeInit(HAL_Dev_t * flash);
Description	Deinitializes FLASH and releases system resources.
Parameters	HAL_Dev_t * flash FLASH handle.
Considerations	1. Before deinitialization, ensure no module is using FLASH. This API is normally not used.
Return Value	None.

5.6.2.3 HAL_FLASH_Open

Table 5-131 HAL_FLASH_Open

Interface	Description
Function	HAL_Status_t HAL_FLASH_Open(HAL_Dev_t * flash, FLASH_InitParam_t * pInitParam);
Description	Open and initialize Flash Ctrl.
Parameters	HAL_Dev_t * flash FLASH handle. FLASH_InitParam_t * pInitParam Configuration parameters typedef struct { uint32_t clockFreq; /* SPI operating frequency */ FLASH_ClockSrc_t clockSrc; /* Flash Ctrl clock source */ FLASH_SpiFormat_t spiFormat; /* SPI format: standard/dual/quad */ bool psramEnable; /* PSRAM enable */ } FLASH_InitParam_t;
Considerations	Input configuration is limited by the Flash List entry; the selected settings cannot exceed the listed capabilities.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.6.2.4 HAL_FLASH_Close

Table 5-132 HAL_FLASH_Close

Interface	Description
Function	HAL_Status_t HAL_FLASH_Close(HAL_Dev_t * flash);
Description	Disables Flash Ctrl.
Parameters	HAL_Dev_t * flash FLASH handle.
Considerations	After closing, the flash cannot be read, erased, written, or used to execute XIP code.

Interface	Description
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.6.2.5 HAL_FLASH_Read

Table 5-133 HAL_FLASH_Read

Interface	Description
Function	HAL_Status_t HAL_FLASH_Read(const HAL_Dev_t *flash, uint32_t addr, uint8_t *pdata, uint32_t size);
Description	Read data from flash.
Parameters	HAL_Dev_t * flash FLASH handle. uint32_t addr Read start address, in the range 0 to flash size. uint8_t *pdata Buffer for storing the data read. uint32_t size Length of data to read.
Considerations	1. Reading fails if flash is not open or is asleep. 2. The read range must not exceed flash size; otherwise, wraparound occurs.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.6.2.6 HAL_FLASH_Write

Table 5-134 HAL_FLASH_Write

Interface	Description
Function	HAL_Status_t HAL_FLASH_Write(const HAL_Dev_t *flash, uint32_t addr, const uint8_t *pdata, uint32_t size);
Description	Write data to flash.
Parameters	HAL_Dev_t * flash FLASH handle. uint32_t addr Write start address, in the range 0 to flash size. uint8_t *pdata Buffer containing the data to write.

Interface	Description
	uint32_t size Length of the data to write.
Considerations	1. Writing fails if flash is not open or is asleep. 2. The write range must not exceed flash size; otherwise, wraparound occurs. 3. Erase the corresponding area before writing.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.6.2.7 HAL_FLASH_Erase

Table 5-135 HAL_FLASH_Erase

Interface	Description
Function	HAL_Status_t HAL_FLASH_Erase(const HAL_Dev_t *flash, uint32_t addr, uint32_t size);
Description	Erase a specified flash area.
Parameters	HAL_Dev_t * flash FLASH handle. uint32_t addr Erase start address, in the range 0 to flash size and aligned to 4 KB. uint32_t size Erase length, automatically aligned to 4 KB.
Considerations	1. Erasing fails if flash is not open or is asleep. 2. The erase start address must be 4 KB aligned.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.6.2.8 HAL_FLASH_ExtControl

Table 5-136 HAL_FLASH_ExtControl

Interface	Description
Function	HAL_Status_t HAL_FLASH_ExtControl(HAL_Dev_t *flash, FLASH_Attr_t attr, void * arg, uint32_t argSize);
Description	Flash extended control
Parameters	HAL_Dev_t * flash FLASH handle. FLASH_Attr_t attr Parameter type typedef enum { FLASH_GET_FLASH_SIZE = 0, // Get total flash size FLASH_GET_PAGE_SIZE = 1, // Get flash page size; default 2

Interface	Description
	<pre> 56 FLASH_GET_BLOCK_SIZE = 2, // Get flash block size; default 4 KB FLASH_GET_PSRAM_SIZE = 3, // Get PSRAM size; valid when PSRAM i s enabled FLASH_GET_CHIP_ID = 4, // Get the Flash chip ID FLASH_EXT_GET_CLK_SRC = 5, // Get the clock source used by Flash Ctrl FLASH_EXT_GET_CLK_DIV = 6, // Get the Flash Ctrl clock divider FLASH_EXT_GET_CLK_FREQ = 7, // Get the SPI operating frequenc y FLASH_EXT_GET_SPI_FORMAT = 8, // Get the SPI format FLASH_EXT_GET_SLEEP_STATE = 9, // Get sleep-control state FLASH_EXT_SET_SLEEP_STATE = 10, // Enable or disable automatic sl eep FLASH_EXT_GET_SLEEP_LATENCY = 11, // Get sleep latency FLASH_EXT_SET_SLEEP_LATENCY = 12, // Set sleep latency FLASH_EXT_GET_RESUME_LATENCY = 13, // Get resume latency FLASH_EXT_SET_RESUME_LATENCY = 14, // Set resume latency FLASH_EXT_GET_AUTO_SLEEP_STATE = 15, // Set automatic-sleep mode state FLASH_EXT_SET_AUTO_SLEEP_STATE = 16, // Get automatic-sleep mode state FLASH_EXT_GET_LIST_SIZE = 17, // Get the Flash List size FLASH_EXT_GET_CHIP_INFO_DEFAULT = 18, // Get the default Flash List FLASH_EXT_GET_CHIP_INFO_LIST = 19, // Get the Flash List FLASH_EXT_GET_CHIP_INFO_DETECH = 20, // Get automatically detected chi p information FLASH_EXT_GET_CHIP_INFO_MODE = 21, // Get chip-info mode FLASH_EXT_SET_CHIP_INFO_MODE = 22, // Set chip-info mode FLASH_EXT_GET_USED_CHIP_INFO_MODE = 23, // Get the current mode } FLASH_Attr_t; void * arg Parameter value to set or get. uint32_t argSize Length of the parameter to set or get. </pre>
Considerations	1. Control and get operations can be used before Open, but some parameters, such as flash size, are valid only after Open.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.6.2.9 HAL_FLASH_Suspend

Table 5-137 HAL_FLASH_Suspend

Interface	Description
Function	HAL_Status_t HAL_FLASH_Suspend(HAL_Dev_t * flash);
Description	Puts Flash to sleep.
Parameters	HAL_Dev_t * flash FLASH handle.
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.6.2.10 HAL_FLASH_Resume

Table 5-138 HAL_FLASH_Resume

Interface	Description
Function	HAL_Status_t HAL_FLASH_Resume(HAL_Dev_t * flash);
Description	Wakes Flash.
Parameters	HAL_Dev_t * flash FLASH handle.
Considerations	None
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.7 WDG

5.7.1 Function Overview

A watchdog monitors abnormal system hangs and triggers corrective action such as a reset. It is a down-counter that triggers one of the following three actions at zero. Before zero is reached, restore the counter value by "feeding" the watchdog.

1. SYS Reset: resets the entire SoC hardware.
2. CPU Reset: resets the CPU without resetting other SoC hardware.
3. IRQ: generates an interrupt.

As shown below, first call HAL_WDG_Init to obtain driver resources. Then call HAL_WDG_Open to configure the watchdog timeout in ms and trigger mode (SYS/CPU/IRQ). Register an interrupt callback for IRQ mode. The watchdog is then active. Call HAL_WDG_Feed before the timeout. If abnormal software fails to do so, a reset or interrupt is triggered.

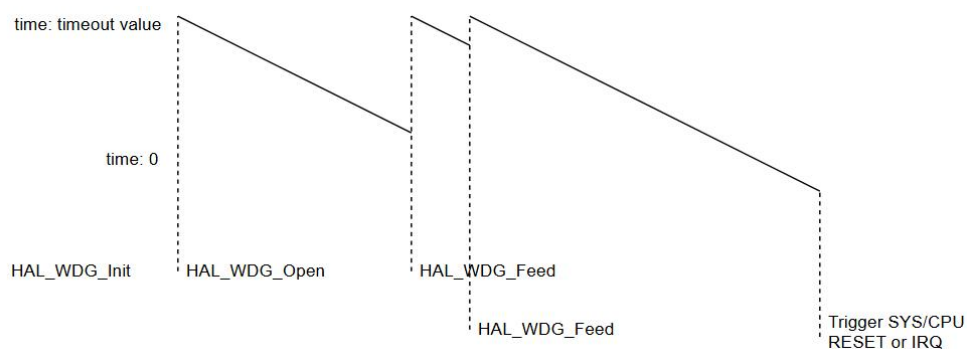


Figure 5-24 Watchdog Usage

Additional notes:

1. `HAL_WDG_Close` disables the watchdog. After it is disabled, `HAL_WDG_Feed` returns success but performs no hardware operation.
2. `HAL_WDG_Close` disables the hardware, while `HAL_WDG_DeInit` releases driver resources. Disable the hardware before releasing its resources.
3. The configuration parameter passed to `HAL_WDG_Open` can be NULL to reopen the watchdog with the previous Open parameters.
4. Watchdog hardware powers down during sleep. With power management enabled, the driver restores its configuration automatically.

See the sample project [project/driver/wdg](#).

5.7.2 Interface

5.7.2.1 HAL_WDG_Init

Table 5-139 HAL_WDG_Init

Interface	Description
Function	<code>HAL_Dev_t * HAL_WDG_Init(WDG_ID_t id);</code>
Description	Initialize WDG and allocate system resources without initializing hardware.
Parameters	WDG_ID_t id WDG peripheral ID. By default, only <code>WDG_DEFT_ID</code> (0) is available.
Considerations	
Return Value	WDG control handle, or NULL if initialization fails.

5.7.2.2 HAL_WDG_DeInit

Table 5-140 HAL_WDG_Deinit

Interface	Description
Function	<code>HAL_Status_t HAL_WDG_DeInit(HAL_Dev_t *wdg);</code>
Description	Deinitializes the watchdog and releases system resources.
Parameters	HAL_Dev_t * wdg WDG handle.
Considerations	
Return Value	<code>HAL_STATUS_OK</code> indicates that the function executed successfully.

Interface	Description
	A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.7.2.3 HAL_WDG_Open

Table 5-141 HAL_WDG_Open

Interface	Description
Function	HAL_Status_t HAL_WDG_Open(HAL_Dev_t *wdg, WDG_InitParam_t * pInitParam);
Description	Enables the watchdog.
Parameters	HAL_Dev_t * wdg WDG handle. WDG_InitParam_t * pInitParam Configuration parameters <pre>typedef struct { WDG_Mode_t mode; //Mode: reset CPU, reset system, or generate interrupt only /* ms as unit */ uint32_t timeout; // Watchdog timeout, in ms /* take effect when mode is interrupt. */ HAL_CbFunc_t callback; //Interrupt-mode callback } WDG_InitParam_t;</pre> <pre>typedef enum { WDG_MODE_RESET_CPU = 0, WDG_MODE_RESET_SYS = 1, WDG_MODE_INTERRUPT = 2, WDG_MODE_MAX, } WDG_Mode_t;</pre>
Considerations	If pInitParam is NULL, use the parameters from the previous Open call.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.7.2.4 HAL_WDG_Close

Table 5-142 HAL_WDG_Close

Interface	Description
Function	HAL_Status_t HAL_WDG_Close(HAL_Dev_t *wdg);
Description	Disables the watchdog.
Parameters	HAL_Dev_t * wdg WDG handle.
Considerations	None

Interface	Description
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

5.7.2.5 HAL_WDG_Feed

Table 5-143 HAL_WDG_Feed

Interface	Description
Function	HAL_Status_t HAL_WDG_Feed(HAL_Dev_t *wdg);
Description	Feed the watchdog.
Parameters	HAL_Dev_t * wdg WDG handle.
Considerations	Feed the watchdog before the WDG counter reaches zero.
Return Value	HAL_STATUS_OK indicates that the function executed successfully. A negative return value indicates that the function failed. See HAL_STATUS_* for details.

Disclaimer and Copyright Notice

The information provided by our company in this document is accurate to the best of our knowledge, but we do not guarantee that the content is completely error-free. All liability for any damages arising from the use of this document (including but not limited to indirect, incidental, or special losses) or any improper use resulting from non-compliance with the instructions herein (including but not limited to over-voltage, over-frequency, or over-temperature operations) is explicitly disclaimed.

The content in this document, including URL references, may differ from the actual content due to product version upgrades or other reasons, and is subject to change without notice.

THIS DOCUMENT IS PROVIDED "AS IS" WITH NO WARRANTIES. NO WARRANTY IS PROVIDED TO THIS DOCUMENT FOR ITS MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR ANY PARTICULAR PURPOSE, NOR DOES ANY WARRANTY OTHERWISE ARISING OUT OF ANY PROPOSAL, SPECIFICATION OR SAMPLE.

All liability, including liability for infringement of any patent rights, relating to use of information in this document is disclaimed. No licenses express or implied, by estoppel or otherwise, to any intellectual property rights are granted herein.

Copyright © 2026 Possumic Technology Co., Ltd. All rights reserved.